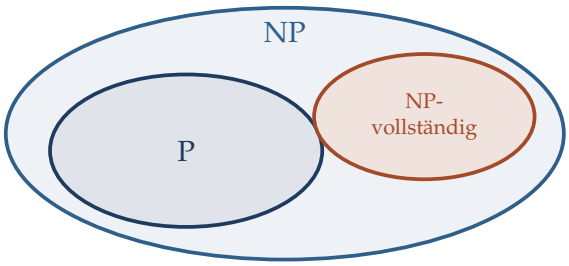
Informatik von Grund auf — Band 3

Der Preis der Berechnung

Eine vollständige Einführung in die Komplexitätstheorie



Von P und NP bis an die Grenzen der Beweisbarkeit

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 3
Der Preis der Berechnung · Komplexitätstheorie

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 3: Der Preis der Berechnung*. Fassung 1.0.1. `https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 166 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort **1**

I Ressourcen messen **9**

1 Zeit und Platz **11**

2 Die Klasse P **19**

II Nichtdeterminismus **29**

3 Die Klasse NP **31**

4 Reduktionen und NP-Vollständigkeit **39**
4.1 Der Satz von Cook und Levin 41

5 Der Zoo der vollständigen Probleme **49**

III Struktur **59**

6 coNP und die Hierarchie **61**
6.1 Die Polynomialzeithierarchie 63

7 Platz als Ressource **69**

8 Rechnen fast ohne Speicher **77**

9 Hierarchiesätze und die Mitte **83**
9.1 Ladners Satz 84

IV Wege um die Härte **91**

10 Näherungsverfahren **93**
10.1 Der PCP-Satz 95

11 Zufall als Ressource **103**

12 Parametrisierte Komplexität **113**

13 Härte als Werkzeug **123**

V Grenzen der Methode **131**

14 Warum die Frage offen ist **133**
14.1 Die Relativierungsbarriere 133
14.2 Natürliche Beweise 135

VI Aktueller Stand **143**

15 Wo die Theorie heute steht **145**
15.1 Warum die Praxis anders aussieht 145

VII Anhang **151**

A Übersicht und Formelsammlung **153**
A.1 Die Landkarte der Klassen 153
A.2 Die Beweistechniken 154
A.3 Die Landkarte des Bandes 156
A.4 Glossar 157

B Kommentierte Literatur **159**

Vorwort: Die andere Grenze

Band 2 handelte von einer Grenze, die bewiesen ist. *Dieser Band handelt von einer, die niemand bewiesen hat – und von der fast alle glauben, dass es sie gibt.*

Zwei Sorten von Unmöglichkeit

Band 2	Dieser Band
„Es gibt *kein* Verfahren.“	„Es gibt vermutlich kein *schnelles* Verfahren.“
Bewiesen, endgültig.	Offen seit 1971.
Gilt für jede denkbare Maschine.	Gilt vielleicht gar nicht.

> ***Der Unterschied ist unangenehm.*** Beim Halteproblem weiß man, woran man ist. **Bei** P **gegen** NP **arbeitet die gesamte praktische Informatik seit fünfzig Jahren mit einer *Annahme*,** *deren Beweis aussteht.*

Und es ist keine Kleinigkeit. *Wäre* $\mathrm{P} = \mathrm{NP}$*, so bräche jede eingesetzte Verschlüsselung zusammen* (Kapitel 13) **– und zugleich ließen sich Tausende praktisch wichtiger Optimierungsprobleme effizient lösen.**

Das Clay-Institut hat 2000 eine Million Dollar auf die Antwort ausgesetzt. **Sie ist nicht abgeholt.**

Worum es konkret geht

Ein Beispiel, an dem man die ganze Frage sehen kann.

Aufgabe: *Gegeben eine Landkarte mit n Ländern. Lässt sie sich mit drei Farben so färben, dass keine zwei Nachbarn dieselbe Farbe haben?*

Eine Lösung finden	Niemand kennt ein schnelles Verfahren. Alle bekannten brauchen im schlechtesten Fall exponentielle Zeit.
Eine Lösung prüfen	Trivial: Man geht die Grenzen durch und vergleicht. $\mathcal{O}(m)$.

Die Frage* P *gegen* NP *lautet in einem Satz:

Ist Finden grundsätzlich schwerer als Prüfen?

Jeder, der je eine Prüfung geschrieben oder eine Beweisaufgabe gelöst hat, hält das für offensichtlich. **Beweisen kann es niemand.**

Warum das mehr ist als eine Fingerübung

Der Punkt ist nicht die Landkarte. *Es ist die Tatsache, dass* ***Tausende*** *scheinbar unverwandter Probleme* dasselbe *Problem sind.*

Stundenpläne erstellen	Register zuteilen (Band 1, Kap. 12)
Schaltungen minimieren	Proteine falten
Rundreisen planen	Beweise suchen
Bausteine verpacken	Sudoku lösen

Alle diese Probleme sind ineinander überführbar. Wer eines davon schnell löst, löst alle. **Wer für eines beweist, dass es nicht schnell geht, hat es für alle bewiesen.**

Das ist der Befund von Cook *(1971),* Levin *(1973) und* Karp *(1972)* – **und er ist der eigentliche Inhalt dieses Bandes** (Kapitel 4 und 5).

Was dieses Buch will

Erstens: Die Klassen aufbauen, nicht aufzählen.

Die Komplexitätstheorie hat den Ruf, ein Zoo von Abkürzungen zu sein – **es gibt einen Katalog mit über 500 Klassen.** *Dieser Band behandelt etwa fünfzehn und begründet für jede,* ***warum*** *sie eingeführt wurde.*

Klasse	Ressource	Leitfrage
L, NL	Platz $\log n$	Was geht ohne Speicher?
P	Zeit, polynomiell	Was heißt „effizient“?
NP	Zertifikate	Prüfen gegen Finden
coNP	Widerlegungen	Ist Nein-Sagen schwerer?
PSPACE	Platz, polynomiell	Spiele, Quantoren
BPP	Zufall	Hilft Würfeln?
EXP	Zeit, exponentiell	bewiesen $\neq$ P

Zweitens: Zeigen, was man tut, wenn ein Problem schwer ist.

NP-vollständig heißt nicht: unlösbar. Es heißt: Der Weg über den allgemeinen Fall ist versperrt. **Teil IV behandelt die vier Auswege, die es gibt.**

Näherung	Nicht optimal, aber beweisbar nahe dran (Kap. 10)
Zufall	Fast immer richtig statt immer (Kap. 11)
Parameter	Schnell, wenn eine Größe klein bleibt (Kap. 12)
Heuristik	Kein Beweis, funktioniert trotzdem (Kap. 15)

Drittens: Ehrlich sein über den Zustand des Gebiets.

Kapitel 14 handelt davon, warum P *gegen* NP *seit fünfzig Jahren offen ist* – **und das ist die interessanteste Frage des ganzen Bandes.** *Es gibt nämlich* Sätze *darüber, welche Beweismethoden* ***nicht*** *funktionieren können.*

Schranke	Jahr	schließt aus
Relativierung	1975	Diagonalisierung
Natürliche Beweise	1994	Kombinatorik
Algebrisierung	2008	algebraische Verfahren

Drei große Beweisfamilien sind nachweislich unzureichend. **Was übrig bleibt, weiß niemand.**

Drei Warnungen

Erstens: „Polynomiell" ist nicht dasselbe wie „praktikabel".
Ein Verfahren mit n^{100} Schritten ist polynomiell und vollkommen unbrauchbar. **Kapitel 2 sagt ausdrücklich, warum man die Klasse P trotzdem so definiert** – *und wo die Definition lügt.*
Zweitens: Die Analyse betrachtet den *schlechtesten* Fall.

> *SAT ist NP-vollständig – und moderne SAT-Löser bearbeiten routinemäßig Instanzen mit Millionen Variablen.* **Das ist kein Widerspruch, sondern ein Hinweis darauf, dass der schlechteste Fall in der Praxis selten vorkommt.**

Kapitel 15 behandelt diesen Zwiespalt ausführlich.
Drittens: Von hier an sind Aussagen bedingt.
Sehr viele Sätze dieses Bandes beginnen mit „Falls $\mathrm{P} \neq \mathrm{NP}\ldots$*"* – **und ich sage jedes Mal dazu, worauf sich eine Aussage stützt.** *Wer das übersieht, hält Vermutungen für Ergebnisse.*

Zum Aufbau

- **Teil I** (Kap. 1–2): Zeit und Platz als Ressourcen; die Klasse P und was „effizient" bedeuten soll.
- **Teil II** (Kap. 3–5): NP als Klasse der prüfbaren Probleme; Reduktionen, der Satz von Cook und Levin, und der Zoo der vollständigen Probleme.
- **Teil III** (Kap. 6–9): coNP und die Polynomialzeithierarchie; Platzkomplexität; die Klassen L und NL; Hierarchiesätze und Ladners Satz.
- **Teil IV** (Kap. 10–13): Näherung, Randomisierung, parametrisierte Komplexität und Kryptographie – die vier Wege, mit Härte umzugehen.
- **Teil V** (Kap. 14–15): Warum die Frage offen ist, und der Stand im August 2026.

Ein Brief, der fünfzehn Jahre unentdeckt blieb

Am 20. März 1956 schrieb KURT GÖDEL einen Brief an JOHN VON NEUMANN, der im Sterben lag.

> *Gödel beschrieb darin ein Problem: Gegeben eine Formel und eine Zahl n – gibt es einen Beweis der Formel mit höchstens n Zeilen?* **Er stellte fest, dass man das durch Durchprobieren aller Möglichkeiten lösen kann, in etwa 2^n Schritten.**

Und dann fragte er die entscheidende Frage:

> *„Es wäre interessant zu wissen,* […] *wie stark im allgemeinen bei finiten kombinatorischen Problemen die Anzahl der Schritte gegenüber dem blossen Probieren verringert werden kann."*

Gödel fragte ausdrücklich, ob es ein Verfahren mit $\mathcal{O}(n)$ oder $\mathcal{O}(n^2)$ Schritten geben könne **– und bemerkte, dies hätte „Folgerungen von der grössten Bedeutung":**

> *„Es würde offenbar bedeuten, dass man trotz der Unlösbarkeit des Entscheidungsproblems die Denkarbeit des Mathematikers bei ja-oder-nein Fragen vollständig durch Maschinen ersetzen könnte."*

> ***Das ist*** P ***gegen*** NP **– *fünfzehn Jahre vor Cook, in einem Privatbrief, präzise formuliert.*** Und Gödel hatte bereits gesehen, was die eigentliche Konsequenz wäre: nicht bloß schnellere Algorithmen, sondern die **Automatisierung des mathematischen Findens.**

Von Neumann starb im Februar 1957 und hat vermutlich nie geantwortet. *Der Brief lag in Gödels Nachlass und wurde erst 1988 von* JURIS HARTMANIS *entdeckt und veröffentlicht.*

> *Hätte von Neumann geantwortet und die Frage weitergegeben, so wäre die Komplexitätstheorie vielleicht ein Jahrzehnt früher entstanden.*

Gödel schrieb übrigens auch, er halte ein schnelles Verfahren für unwahrscheinlich – *aber die Frage sei „keineswegs aussichtslos".* **Beide Einschätzungen gelten bis heute.**

Sechzig Jahre Komplexitätstheorie

1956	Gödel	Brief an von Neumann
1960	Rabin	erste Ansätze einer Aufwandstheorie
1964	Cobham	polynomielle Zeit als Begriff
1965	**Edmonds**	**„gute Algorithmen“; Matching in** P
1965	**Hartmanis, Stearns**	**Zeithierarchiesatz; Name des Gebiets**
1970	Savitch	$\mathrm{NSPACE}(f) \subseteq \mathrm{SPACE}(f^2)$
1971	**Cook**	**SAT ist NP-vollständig**
1972	**Karp**	**21 vollständige Probleme**
1973	Levin	unabhängig dasselbe (UdSSR)
1975	Ladner	Zwischenklassen existieren
1975	**Baker, Gill, Solovay**	**Relativierungsschranke**
1976	Miller, Rabin	randomisierter Primzahltest
1977	Solovay, Strassen	ebenso
1978	Rivest, Shamir, Adleman	RSA – Härte als Werkzeug
1979	Garey, Johnson	„Computers and Intractability“
1987	Immerman, Szelepcsényi	$\mathrm{NL} = \mathrm{coNL}$
1990	Shamir	$\mathrm{IP} = \mathrm{PSPACE}$
1992	**Arora u. a.**	**PCP-Satz**
1994	**Razborov, Rudich**	**natürliche Beweise**
1995	Impagliazzo	die fünf Welten

1999	Downey, Fellows	parametrisierte Komplexität
2000	**Clay-Institut**	P **gegen** NP **als Millenniumsproblem**
2001	Håstad	optimale Nichtapproximierbarkeit
2001	Impagliazzo, Paturi	Exponentialzeithypothese
2002	**Agrawal, Kayal, Saxena**	**Primzahltest in** P
2004	Reingold	Zusammenhang in L
2008	Aaronson, Wigderson	Algebrisierungsschranke
2011	Williams	untere Schranke für ACC
2020	Ji u. a.	$\mathrm{MIP}^* = \mathrm{RE}$
2026	—	P gegen NP weiterhin offen

Zwei Zeilen verdienen Hervorhebung.
1965, Edmonds. JACK EDMONDS *gab in seiner Arbeit über Matchings den ersten ausdrücklichen Grund dafür, warum polynomielle Zeit der richtige Begriff für „effizient" sei* – **und formulierte nebenbei die Vermutung, dass das Rundreiseproblem keinen solchen Algorithmus hat.**
1994, Razborov und Rudich. *Sie bewiesen, dass die Beweistechnik, mit der man in den 1980er Jahren große Fortschritte gemacht hatte,* ***prinzipiell nicht ausreicht*** *– sofern es Einwegfunktionen gibt.* **Ein Satz darüber, warum man einen Satz nicht beweisen kann.**

Wenn P = NP *wäre, dann wäre die Welt ein völlig anderer Ort, als wir gewöhnlich annehmen. Es gäbe keinen besonderen Wert in* kreativen Sprüngen, *keine grundsätzliche Kluft zwischen dem Lösen eines Problems und dem Erkennen der Lösung.*

— SCOTT AARONSON, 2006.

Part I

Ressourcen messen

1 Zeit und Platz

Was gemessen wird, und woran

*Band 2 fragte, **ob** eine Maschine anhält. Dieser Band fragt, **nach wie vielen Schritten**.* **Dafür braucht man ein festgelegtes Maschinenmodell** – *denn die Schrittzahl hängt davon ab, was ein Schritt ist.*

Definition 1.1. Für eine Turingmaschine M, die auf jeder Eingabe hält:

$t_M(w)$	Zahl der Schritte auf Eingabe w
$s_M(w)$	Zahl der besuchten Bandfelder
$T_M(n)$	$\max\{t_M(w) : \lvert w\rvert = n\}$ – **schlechtester Fall**
$S_M(n)$	$\max\{s_M(w) : \lvert w\rvert = n\}$

Definition 1.2.

$$\mathrm{DTIME}(f) := \{L : \exists M \text{ mit } L(M) = L,\ T_M(n) = \mathcal{O}(f(n))\}$$

und entsprechend NTIME, SPACE, NSPACE.

*Beim **Platz** zählt man üblicherweise nur das **Arbeitsband*** – **das Eingabeband darf gelesen, aber nicht beschrieben werden.** *Sonst wären Klassen unterhalb von $\mathcal{O}(n)$ sinnlos, denn die Eingabe belegt ja schon n Felder.*

Warum das Modell überhaupt festgelegt werden muss

Bei der Berechenbarkeit war das gleichgültig – *alle vernünftigen Modelle erkennen dieselben Sprachen* (Band 2, Kapitel 7).

*Bei der **Laufzeit** ist es das nicht.*

Problem	1 Band	2 Bänder
Palindrome erkennen	$\Theta(n^2)$	$\Theta(n)$

Mit einem Band muss der Kopf ständig zwischen Anfang und Ende hin- und herlaufen – **das kostet quadratisch.** *Mit zwei Bändern kopiert man die Eingabe rückwärts auf das zweite und vergleicht in einem Durchgang.*

Die untere Schranke $\Omega(n^2)$ für ein Band ist bewiesen (HENNIE, 1965) – *sie gehört zu den wenigen scharfen unteren Schranken, die man überhaupt hat.*

Warum die Modellabhängigkeit trotzdem verschwindet

*Der Ausweg besteht darin, die Klassen **grob genug** zu wählen, dass die Unterschiede zwischen Modellen darin untergehen.*

Übergang	Zeitverlust	polynomiell?
k Bänder $\to$ 1 Band	quadratisch	ja
2-dim. Band $\to$ 1 Band	polynomiell	ja
Registermaschine $\to$ TM	polynomiell	ja
Nichtdeterministisch $\to$ det.	exponentiell	**nein**

Die ersten drei Zeilen sind der Grund, warum man in Polynomen rechnet. Ein Polynom eines Polynoms ist wieder ein Polynom – **also ist die Klasse P *modellunabhängig.***

Die vierte Zeile ist der offene Punkt. **Sie ist P gegen NP.**

Man nennt die ersten drei Zeilen die erweiterte Church-Turing-These:

> *„Jedes realistische Rechenmodell lässt sich von einer Turingmaschine mit höchstens polynomiellem Zeitverlust simulieren."*

Der Quantenrechner stellt diese These infrage

SHORS Algorithmus (1994) faktorisiert Zahlen in polynomieller Zeit auf einem Quantenrechner – *klassisch kennt man nur subexponentielle Verfahren.*

> ***Wäre das ein Gegenbeispiel?*** Nur, wenn Faktorisieren klassisch *beweisbar* schwer ist – und das weiß niemand. **Die These ist also nicht widerlegt, sondern erschüttert.**

*Man hat sie deshalb aufgespalten: Die **klassische** erweiterte These gilt als fraglich, die **quantenerweiterte** (mit Quantenrechnern als Maßstab) gilt als plausibel.* **Band 15 behandelt das ausführlich.**

Wichtig: Die ursprüngliche *Church-Turing-These – über **Berechenbarkeit**, nicht über Zeit – ist davon nicht betroffen.* **Ein Quantenrechner kann das Halteproblem so wenig lösen wie ein Abakus** (Band 2, Kapitel 7).

Zwei Sätze, die zeigen, dass Konstanten nichts bedeuten

Satz 1.3 (Lineare Beschleunigung)**.** *Ist* $L \in \mathrm{DTIME}(f(n))$ *und* $c > 0$*, so ist* $L \in \mathrm{DTIME}(c \cdot f(n) + n + 2)$.

Die Beweisidee.

> *Man vergrößert das Bandalphabet: Ein neues Symbol fasst k alte zusammen.* **Die Maschine liest dann k Zeichen auf einmal und erledigt in einem Schritt, wofür sie vorher k brauchte.**

Der Preis ist ein exponentiell größeres Alphabet und eine viel größere Zustandsmenge – **beides zählt in diesem Modell nicht.**

Was dieser Satz wirklich sagt

Er sagt nicht, dass Rechnen beliebig schnell geht. *Er sagt, dass* ***das Modell*** *zu grob ist, um konstante Faktoren zu erfassen.*

> ***Das ist die Rechtfertigung für die $\mathcal{O}$-Notation*** (Band 1, Kapitel 1): Konstanten sind im Modell bedeutungslos, also lässt man sie weg. **Nicht aus Bequemlichkeit, sondern weil sie keine Information tragen.**

In der Praxis tragen sie natürlich sehr wohl Information – **das ist der Preis der Abstraktion, und Kapitel 15 kommt darauf zurück.**

Satz 1.4 (Zeit gegen Platz)**.**

$$\mathrm{DTIME}(f) \subseteq \mathrm{SPACE}(f) \subseteq \mathrm{DTIME}\big(2^{\mathcal{O}(f)}\big).$$

Die linke Inklusion. *In f Schritten kann der Kopf höchstens f Felder besuchen.* ✓
Die rechte. *Eine Maschine mit Platz $f(n)$ hat höchstens*

$$\underbrace{|Q|}_{\text{Zustand}} \cdot \underbrace{f(n)}_{\text{Kopfposition}} \cdot \underbrace{|\Gamma|^{f(n)}}_{\text{Bandinhalt}} = 2^{\mathcal{O}(f(n))}$$

verschiedene ***Konfigurationen****.*

> ***Wiederholt sich eine Konfiguration, so läuft die Maschine in einer Schleife und hält nie.*** Eine haltende Maschine kann also nicht mehr Schritte machen, als es Konfigurationen gibt. ✓

Das Konfigurationsargument ist eines der meistbenutzten Werkzeuge dieses Bandes – **es kommt in Kapitel 7 und 8 mehrfach wieder.**

Die daraus folgende Kette:

$$\mathrm{L} \subseteq \mathrm{NL} \subseteq \mathrm{P} \subseteq \mathrm{NP} \subseteq \mathrm{PSPACE} \subseteq \mathrm{EXP} \subseteq \mathrm{NEXP}.$$

> ***Von diesen sieben Inklusionen weiß man, dass mindestens zwei echt sind*** – $\mathrm{L} \neq \mathrm{PSPACE}$ und $\mathrm{P} \neq \mathrm{EXP}$, beide aus den Hierarchiesätzen (Kapitel 9). **Aber man weiß von *keiner einzelnen*, ob sie echt ist.**

Aufgabe 1.1.

(a) Bestimmen Sie Zeit- und Platzbedarf konkreter Turingmaschinen und zeigen Sie die Bandzahlabhängigkeit.

(b) Zählen Sie Konfigurationen und leiten Sie die Inklusionskette vollständig her.

(c) Untersuchen Sie, warum die Kodierung der Eingabe die Komplexität ändern kann.

Lösung Schritt für Schritt.

(a) Konkrete Maschinen.

Maschine 1 – Palindrome mit einem Band.

Verfahren: Vergleiche das erste mit dem letzten Zeichen, streiche beide, wiederhole.

Durchgang	Kopfweg	Rest
1	n hin, n zurück	$n-2$
2	$n-2$ hin und zurück	$n-4$
k	$\approx 2(n-2k)$	

$$T(n) = \sum_{k=0}^{n/2} 2(n-2k) \approx 2 \cdot \frac{n}{2} \cdot \frac{n}{2} = \frac{n^2}{2} = \Theta(n^2).$$

$$S(n) = \Theta(n) \quad \text{(nur die Eingabe).}$$

Maschine 2 – Palindrome mit zwei Bändern.

Phase 1	Eingabe auf Band 2 kopieren: n Schritte
Phase 2	Kopf 1 an den Anfang, Kopf 2 ans Ende: n Schritte
Phase 3	Beide gleichzeitig gegenläufig, Zeichen vergleichen: n Schritte

$$T(n) = \Theta(n). \qquad \checkmark$$

Ein Faktor n allein durch ein zweites Band. Bei $n = 10^6$ sind das 10^6 gegen 10^{12} Schritte – Millisekunden gegen Viertelstunden.

Maschine 3 – $\{0^n 1^n\}$ mit einem Band, aber geschickter.

*Statt einzeln zu streichen, streiche man in jedem Durchgang **jedes zweite** Zeichen und prüfe die Paritäten.*

Nach jedem Durchgang hat sich die Zahl der Zeichen halbiert. **Also gibt es nur $\mathcal{O}(\log n)$ Durchgänge zu je $\mathcal{O}(n)$ Schritten:**

$$T(n) = \mathcal{O}(n \log n).$$

Und das ist optimal für ein Band – **Hennie bewies 1965 die untere Schranke $\Omega(n \log n)$.** *Wieder eine der seltenen scharfen Schranken.*

(b) Konfigurationen zählen.
Schritt 1 – Was eine Konfiguration ist.

Bestandteil	**Möglichkeiten**	**Beitrag**
Zustand	$\lvert Q \rvert$	konstant
Kopfposition Eingabe	$n+1$	$\mathcal{O}(n)$
Kopfposition Arbeitsband	$s(n)$	
Inhalt Arbeitsband	$\lvert \Gamma \rvert^{s(n)}$	**exponentiell**

$$\#\text{Konf}(n) = |Q| \cdot (n+1) \cdot s(n) \cdot |\Gamma|^{s(n)}.$$

Schritt 2 – Zwei wichtige Spezialfälle.
Für $s(n) = \mathcal{O}(\log n)$:

$$\#\text{Konf} = \mathcal{O}\big(n \cdot \log n \cdot 2^{c \log n}\big) = \mathcal{O}(n^{c+2}) = n^{\mathcal{O}(1)}.$$

$$\boxed{\text{L} \subseteq \text{P} \quad \text{und} \quad \text{NL} \subseteq \text{P}}$$

Logarithmischer Platz erlaubt nur polynomiell viele Konfigurationen – also kann man den gesamten Konfigurationsgraphen aufbauen und darin suchen. **Das ist der Beweis, und er ist kürzer als die Formulierung.**

Für $s(n) = n^{\mathcal{O}(1)}$:

$$\#\text{Konf} = 2^{n^{\mathcal{O}(1)}} \Longrightarrow \text{PSPACE} \subseteq \text{EXP}. \qquad \checkmark$$

Schritt 3 – Die vollständige Kette.

$\text{L} \subseteq \text{NL}$	Determinismus ist Sonderfall
$\text{NL} \subseteq \text{P}$	Konfigurationsgraph, Erreichbarkeit
$\text{P} \subseteq \text{NP}$	Determinismus ist Sonderfall
$\text{NP} \subseteq \text{PSPACE}$	**alle Zertifikate durchprobieren**
$\text{PSPACE} \subseteq \text{EXP}$	Konfigurationszählung

Zur vierten Zeile, weil sie am wenigsten offensichtlich ist.

> *Ein Zertifikat hat polynomielle Länge (Kapitel 3). Man kann* ***alle*** $2^{p(n)}$ *Zertifikate der Reihe nach durchprobieren* – **und braucht dafür nur den Platz für** ***eines*** **davon**, *weil man den Speicher nach jedem Versuch wiederverwendet.*

Exponentielle ***Zeit****, polynomieller* ***Platz****.* ✓ **Das ist die wichtigste Eigenschaft von Platz: Er ist wiederverwendbar, Zeit nicht.**

(c) Die Kodierung ändert alles.
Beispiel 1 – Zahlen unär oder binär.
Aufgabe: Ist n eine Primzahl?

Kodierung	**Eingabelänge**	**Probedivision**
unär (1^n)	n	$\mathcal{O}(\sqrt{n})$ – **polynomiell**
binär	$\log_2 n$	$\mathcal{O}(\sqrt{n}) = \mathcal{O}(2^{\ell/2})$ – **exponentiell**

> ***Dasselbe Verfahren ist bei einer Kodierung effizient und bei der anderen nicht*** – weil sich die Eingabelänge exponentiell unterscheidet.

Man rechnet deshalb ***immer*** *mit der Binärkodierung.* **Unäre Kodierung wäre ein Betrug: Man bläht die Eingabe auf, um die relative Laufzeit klein aussehen zu lassen.**
Beispiel 2 – Das Rucksackproblem.
Der Algorithmus aus Band 1 (Kapitel 14) läuft in $\mathcal{O}(nW)$.

	Eingabelänge	**Laufzeit**
W unär	$\mathcal{O}(W)$	polynomiell
W binär	$\mathcal{O}(\log W)$	exponentiell

Man nennt solche Verfahren ***pseudopolynomiell*** (Band 1) – **polynomiell im Zahlen*wert*, exponentiell in der Eingabe*länge*.**

Stark und schwach NP-vollständig

Der Unterschied hat einen Namen bekommen.

schwach	Es gibt einen pseudopolynomiellen Algorithmus – **Rucksack**, Partition
stark	Auch bei unärer Kodierung noch NP-vollständig – **Rundreise**, 3-Färbbarkeit, Cliquen

Der Unterschied ist praktisch bedeutsam: ***Schwach NP-vollständige Probleme sind lösbar, solange die Zahlen klein bleiben.*** Bei stark NP-vollständigen hilft das nicht.

GAREY *und* JOHNSON *führten die Unterscheidung 1978 ein*[9]. **Sie ist der Grund, warum das Rucksackproblem in der Praxis oft leicht und das Rundreiseproblem oft schwer ist** – *obwohl beide NP-vollständig sind.*

Beispiel 3 – Graphen als Matrix oder Liste.

Darstellung	Länge	Bemerkung
Adjazenzmatrix	$\Theta(n^2)$	
Adjazenzliste	$\Theta(n+m)$	bei dünnen Graphen viel kürzer

Hier ist der Unterschied nur polynomiell ($m \leq n^2$) – **für die Klasse** P **also gleichgültig.** *Für die konkrete Laufzeit natürlich nicht* (Band 1, Kapitel 10).

Die Regel lautet: *Solange zwei Kodierungen polynomiell ineinander überführbar sind, ändert sich an der Klassenzugehörigkeit nichts.* **Nur bei unär gegen binär ist der Sprung exponentiell – und deshalb ist genau das verboten.**

Sipser[24] und Arora und Barak[2] entwickeln die Modelle mit unterschiedlicher Tiefe; Papadimitriou[21] ist bei den Maschinenmodellen besonders sorgfältig; Wegener[26] ist die knappe deutschsprachige Darstellung.

2 Die Klasse P

Was „effizient“ heißen soll

Definition 2.1.

$$\mathrm{P} := \bigcup_{k \geq 1} \mathrm{DTIME}(n^k).$$

Eine Sprache liegt in P, wenn es eine deterministische Maschine gibt, die sie in polynomieller Zeit entscheidet.

> *Diese Definition ist eine **Setzung**, und sie ist angreifbar.* **Der Rest dieses Kapitels begründet sie – und sagt, wo sie versagt.**

Die vier Argumente dafür.

Abgeschlossenheit	Polynome sind unter Summe, Produkt und Einsetzung abgeschlossen – ein Unterprogramm in P, polynomial oft aufgerufen, bleibt in P
Modellunabhängigkeit	Alle vernünftigen Modelle simulieren einander polynomiell (Kap. 1)
Kodierungsunabhängigkeit	Vernünftige Kodierungen unterscheiden sich polynomiell
Erfahrung	Probleme in P haben meist Exponenten ≤ 4

> ***Das erste Argument ist das stärkste.*** Es bedeutet, dass man Algorithmen **zusammensetzen** darf, ohne die Klasse zu verlassen – und genau das tut man beim Programmieren ständig.

*Man nennt die Setzung die **These von Cobham und Edmonds**.* **Alan Cobham formulierte sie 1964, Jack Edmonds 1965 unabhängig – und Edmonds’ Fassung war die wirkungsvollere.**

„Ein guter Algorithmus“ – Edmonds 1965

JACK EDMONDS veröffentlichte 1965 eine Arbeit mit dem Titel „Paths, Trees, and Flowers“. *Ihr Gegenstand war ein Algorithmus für das Matchingproblem in allgemeinen Graphen* – **ein Problem, für das man bis dahin nur exponentielle Verfahren kannte.**

> *Das Bemerkenswerte an der Arbeit war nicht der Algorithmus, sondern der* ***Abschnitt 2****, in dem Edmonds erklärte, warum er ihn für gut hielt.*

Er schrieb:

> *„Für ein praktisches Verfahren* [...] *sollte die Zahl der Schritte nicht nur endlich, sondern* ***algebraisch beschränkt*** *sein* [...] *als Funktion der Größe der Aufgabe.“*

Und er begründete, warum ***exponentiell*** *nicht genügt:*

> *„Ein endlicher Algorithmus, der eine astronomische Zahl von Schritten braucht, ist für praktische Zwecke nicht besser als gar keiner.“*

Edmonds sprach von ***good algorithms*** – **der Begriff „polynomiell“ setzte sich erst später durch.**

> *Im selben Aufsatz steht auch eine Vermutung, die er beiläufig äußerte: Für das Rundreiseproblem gebe es* ***vermutlich keinen*** *guten Algorithmus* – **sechs Jahre vor Cooks Satz.**

Edmonds hat später erzählt, die Gutachter hätten den philosophischen Abschnitt streichen wollen. *Er setzte sich durch.* **Der Aufsatz gilt heute als Gründungsdokument der Komplexitätstheorie – wegen dieses Abschnitts, nicht wegen des Algorithmus.**

Wo die Definition lügt

Polynomiell heißt nicht praktikabel

Laufzeit	$n = 100$	**bei** 10^9 **Op./s**
n^2	10^4	10 µs
n^4	10^8	0,1 s
n^{10}	10^{20}	3000 Jahre
n^{100}	10^{200}	jenseits allem

Alle vier Zeilen sind „polynomiell" und liegen in P. Die letzten zwei sind vollkommen unbrauchbar.

Umgekehrt gibt es brauchbare exponentielle Verfahren:

Simplexverfahren	exponentiell im schlechtesten Fall, in der Praxis linear
SAT-Löser	exponentiell, lösen Millionen Variablen
$2^{n/2}$ bei $n = 50$	$3 \cdot 10^7$ – Sekunden

Warum man die Definition trotzdem behält.

Weil sie ***robust*** *ist und die Alternativen es nicht sind.* **Eine Klasse „$\mathcal{O}(n^3)$" wäre nicht abgeschlossen unter Einsetzung und würde vom Maschinenmodell abhängen.** *Man kauft die Ungenauigkeit für die Robustheit.*

Und die Erfahrung stützt sie.

Problem	**Exponent**	**Jahr**
Matching	$n^{2,5}$	1965
Lineares Programmieren	$n^{3,5}$	1979
Primzahltest (AKS)	n^6	2002
Maximaler Fluss	$n\,m$	2013

Es gibt praktisch keine natürlichen Probleme mit Exponent 20 oder 50. Das ist ein empirischer Befund ohne Erklärung – aber ein sehr stabiler.

Und wenn doch ein hoher Exponent auftritt, sinkt er meist rasch: **Der AKS-Primzahltest begann 2002 bei n^{12} und wurde binnen Monaten auf n^6 gebracht.**

Was alles in P liegt

Problem	Aufwand	seit
Sortieren, Suchen	$n \log n$	antik
Kürzeste Wege	$m + n \log n$	1959
Maximaler Fluss	$n\,m$	1956
Matching (allgemein)	$n^{2,5}$	**1965**
Lineares Programmieren	$n^{3,5}$	**1979**
Primzahltest	n^6	**2002**
Graphisomorphie	offen	quasipoly. 2015
Faktorisieren	offen	—

Die Zeilen 5 und 6 waren jahrzehntelang offen und wurden entschieden. Das ist der Grund, warum niemand die Hoffnung aufgibt, dass auch die letzten beiden fallen.

Drei Inder und der Primzahltest

Am 6. August 2002 verschickten MANINDRA AGRAWAL und seine beiden Studenten NEERAJ KAYAL und NITIN SAXENA vom IIT Kanpur einen Aufsatz per E-Mail.

Titel: „PRIMES is in P“. **Neun Seiten.**

> *Kayal und Saxena hatten wenige Wochen zuvor ihren Bachelor gemacht.* **Sie lösten ein Problem, an dem seit den 1970er Jahren gearbeitet wurde** – *und zwar mit elementaren Mitteln, ohne unbewiesene Annahmen.*

Der Kern ist eine Verallgemeinerung des kleinen Satzes von Fermat (Mathematik-Band 1): *n ist genau dann prim, wenn*

$$(x + a)^n \equiv x^n + a \pmod{n}$$

als Polynomkongruenz gilt. **Das direkt zu prüfen kostet exponentiell viel** – *das Polynom hat n Koeffizienten.*

> *Ihr Kunstgriff: Man rechnet zusätzlich modulo $x^r - 1$ für ein geschickt gewähltes kleines r.* **Dann hat das Polynom nur noch *r* Koeffizienten** – *und der Beweis zeigt, dass die Aussage trotzdem charakteristisch bleibt.*

Die Reaktion war ungewöhnlich. *Binnen einer Woche hatten mehrere Fachleute den Beweis geprüft und für richtig befunden.* **Die New York Times berichtete am 8. August.**

> *Die Autoren erhielten 2006 gemeinsam den Gödel-Preis und den Fulkerson-Preis.* **Es kommt selten vor, dass Bachelor-Absolventen einen Preis dieser Art bekommen.**

Die praktische Wirkung ist gleich null. *Kryptographische Software benutzt weiterhin den randomisierten Miller-Rabin-Test* (Kapitel 11) – **er ist um Größenordnungen schneller und irrt mit einer Wahrscheinlichkeit, die man beliebig klein machen kann.**

Der Wert des Ergebnisses ist theoretisch: ***Es zeigt, dass Primalität keinen Zufall braucht*** – *und das ist ein Datenpunkt in der Frage, ob Zufall überhaupt Rechenkraft verleiht* (Kapitel 11).

Aufgabe 2.1.

(a) Zeigen Sie die Abgeschlossenheit von P unter den üblichen Operationen.
(b) Untersuchen Sie das lineare Programmieren als Grenzfall.
(c) Diskutieren Sie, warum P nicht dasselbe ist wie „praktisch lösbar“.

Lösung Schritt für Schritt.

(a) Abgeschlossenheit.
Behauptung 1 – P ist unter Komplement abgeschlossen.
Sei M eine Maschine für L mit Laufzeit $p(n)$. Man vertausche die Antworten. **Da M auf *jeder* Eingabe hält, ist das wohldefiniert.**

$$\overline{L} \in \mathrm{P}. \qquad \checkmark$$

Das Wort „hält immer“ trägt den Beweis. Bei den aufzählbaren Sprachen (Band 2, Kapitel 10) versagt genau dieses Argument – und deshalb sind sie nicht unter Komplement abgeschlossen.

Ob NP *unter Komplement abgeschlossen ist, ist* ***offen*** **– das ist die Frage** $\mathrm{NP} = \mathrm{coNP}$ (Kapitel 6).

Behauptung 2 – Vereinigung und Schnitt.
Man lasse beide Maschinen nacheinander laufen:

$$p_1(n) + p_2(n) \quad \text{ist wieder ein Polynom.} \qquad \checkmark$$

Behauptung 3 – Einsetzung (die wichtigste).

Ein Algorithmus A laufe in $p(n)$ Schritten und rufe dabei ein Unterprogramm B auf, das in $q(m)$ Schritten läuft, wobei m die Länge seiner Eingabe ist.

Zwei Beobachtungen:

1. A kann höchstens $p(n)$ Aufrufe machen – jeder Aufruf kostet mindestens einen Schritt.
2. Jede Eingabe an B hat höchstens Länge $p(n)$ – A kann in $p(n)$ Schritten nicht mehr hinschreiben.

$$T(n) \leq p(n) \cdot q\big(p(n)\big).$$

Ein Polynom, in ein Polynom eingesetzt, mal ein Polynom **– wieder ein Polynom.** ✓

Das ist die Eigenschaft, die* P *zur richtigen Klasse macht. Man darf modular programmieren, ohne die Effizienzgarantie zu verlieren – und das gilt für keine engere Klasse.

Probe: $p(n) = n^2$, $q(m) = m^3$:

$$T(n) \leq n^2 \cdot (n^2)^3 = n^8. \qquad \checkmark$$

Der Exponent wächst dabei rasch

Die Rechnung zeigt zugleich das Problem.
Drei Schachtelungsebenen von je n^2 ergeben $n^{2\cdot2\cdot2} = n^8$ – und bei vier Ebenen n^{16}.

> ***Die Theorie sagt „polynomiell" und meint es ernst; die Praxis sagt „unbrauchbar" und meint es auch ernst.*** Beide haben recht – sie messen Verschiedenes.

(b) Lineares Programmieren als Grenzfall.
Aufgabe: Maximiere $c^\top x$ unter $Ax \leq b$, $x \geq 0$.
Die Geschichte in vier Zeilen.

1947	Dantzig	**Simplexverfahren**
1972	Klee, Minty	Simplex ist exponentiell im schlechtesten Fall
1979	**Khatschijan**	**Ellipsoidverfahren: polynomiell**
1984	Karmarkar	Innere-Punkte-Verfahren: polynomiell *und* praktisch

Das Merkwürdige daran.

Verfahren	**schlechtester Fall**	**Praxis**
Simplex	exponentiell	**sehr schnell**
Ellipsoid	polynomiell	**unbrauchbar**
Innere Punkte	polynomiell	schnell

> ***Die ersten beiden Zeilen sind das Gegenbeispiel zur These von Cobham und Edmonds in Reinform.*** Das polynomielle Verfahren war jahrelang das schlechtere.

Warum Simplex in der Praxis gewinnt.
*Das Klee-Minty-Beispiel von 1972 ist ein verzerrter Würfel in n Dimensionen, bei dem Simplex **alle** 2^n Ecken abläuft.* **Solche Instanzen kommen in der Praxis nicht vor.**

> SPIELMAN *und* TENG *lieferten 2001 die Erklärung mit der* ***geglätteten Analyse****: Man betrachtet den schlechtesten Fall* ***plus eine kleine zufällige Störung****.*
> **Unter dieser Analyse ist Simplex polynomiell.**

*Die schlechten Instanzen sind also **isoliert** – eine winzige Störung macht sie leicht.*
Spielman und Teng erhielten dafür den Gödel-Preis 2008.

Und die ganzzahlige Variante?

Variante	Komplexität	Bemerkung
$x \in \mathbb{R}^n$	P	seit 1979
$x \in \mathbb{Z}^n$	**NP-vollständig**	Karps Liste (Kap. 5)
$x \in \{0,1\}^n$	NP-vollständig	

Eine einzige zusätzliche Forderung – Ganzzahligkeit – macht aus einem lösbaren Problem ein hartes. Das ist typisch und wird uns in Kapitel 5 mehrfach begegnen.

(c) P gegen „praktisch lösbar".
Die vier Abweichungen.
Abweichung 1 – hohe Exponenten. *Siehe oben.* n^{100} **ist in P und unbrauchbar.**

Ein reales Beispiel: Der Satz von Robertson *und* Seymour *liefert für viele Graphenprobleme Algorithmen mit Laufzeit* $\mathcal{O}(n^3)$ – **aber mit einer Konstanten, die ein Turm von Exponentialen ist.** *Der Algorithmus ist polynomiell und wurde nie ausgeführt.*

Abweichung 2 – nichtkonstruktive Ergebnisse.
*Derselbe Satz garantiert für gewisse Graphenklassen die **Existenz** eines polynomiellen Verfahrens, ohne es anzugeben.*

Man weiß, dass ein endlicher Test existiert, kennt ihn aber nicht. **Ein polynomieller Algorithmus, den niemand hinschreiben kann.**

Abweichung 3 – schlechtester Fall gegen typischen Fall.

Problem	Klasse	Praxis
SAT	NP-vollständig	Millionen Variablen
Simplex	exponentiell	Standardwerkzeug
Rundreise	NP-vollständig	10^5 Städte optimal

Alle drei Zeilen sind Fälle, in denen die Theorie „schwer“ sagt und die Praxis „geht“. Der Grund ist stets derselbe: Die harten Instanzen sind selten, künstlich oder von besonderer Bauart.

Abweichung 4 – Speicher und Parallelität.
Ein Verfahren mit $\mathcal{O}(n^2)$ Zeit und $\mathcal{O}(n^2)$ Speicher ist bei $n = 10^6$ unbrauchbar – **es bräuchte Terabytes.** *Die Klasse* P *sagt darüber nichts.*

Was man daraus *nicht* folgern sollte

Manche schließen daraus, die Komplexitätstheorie sei praktisch bedeutungslos. *Das ist falsch, und zwar aus drei Gründen.*

1. **Sie spart Arbeit.** Wer weiß, dass ein Problem NP-vollständig ist, sucht nicht mehr nach einem exakten schnellen Verfahren, sondern gleich nach einer Näherung.
2. **Sie leitet die Suche.** Die Reduktion auf ein bekanntes Problem zeigt oft, *woran* die Schwierigkeit hängt – und welche Sonderfälle leicht sind.
3. **Sie ist die Grundlage der Kryptographie.** Dort will man Härte, und die Theorie sagt, worauf man sie stützen kann (Kap. 13).

Die richtige Haltung ist die von* Garey *und* Johnson[9]*: Ein Härtebeweis ist kein Todesurteil, sondern eine **Wegbeschreibung** – er sagt, in welcher Richtung man nicht weitersuchen soll.

Ihr Buch von 1979 beginnt mit dem Bild eines Angestellten, der seinem Chef sagen muss, dass er das Problem nicht lösen kann. **Die Pointe: Statt „Ich kann es nicht“ sagt er „Ich kann es nicht, aber all diese berühmten Leute konnten es auch nicht“** – *und das ist eine ganz andere Auskunft.*

Edmonds’ Aufsatz[7] ist das Gründungsdokument; Arora und Barak[2] und Goldreich[10] begründen die Klassenwahl ausführlich; Kleinberg und Tardos[17] zeigen die Praxisseite.

Part II

Nichtdeterminismus

3 Die Klasse NP

Zwei Definitionen, dieselbe Klasse

NP *lässt sich auf zwei ganz verschiedene Weisen einführen.* **Die eine ist maschinennah, die andere anschaulich** – *und sie sind äquivalent.*

Definition 3.1 (Maschinenfassung)**.**

$$\mathrm{NP} := \bigcup_{k \geq 1} \mathrm{NTIME}(n^k),$$

also die Sprachen, die eine **nichtdeterministische** Turingmaschine in polynomieller Zeit entscheidet.

Definition 3.2 (Zertifikatfassung)**.** $L \in \mathrm{NP}$, wenn es ein Polynom p und eine Sprache $V \in \mathrm{P}$ gibt mit

$$w \in L \iff \exists\, c \text{ mit } |c| \leq p(|w|) \text{ und } (w, c) \in V.$$

Das Wort c heißt **Zertifikat** oder **Zeuge**.

In Worten: *L liegt in* NP*, wenn sich jede Ja-Antwort durch einen* ***kurzen Beleg*** *rechtfertigen lässt, den man* ***schnell prüfen*** *kann.*

Problem	**Zertifikat**	**Prüfung**
SAT	eine Belegung	Formel auswerten
3-Färbbarkeit	die Färbung	Kanten durchgehen
Clique	die Eckenmenge	Paare prüfen
Rundreise ($\leq k$)	die Tour	Längen addieren
Faktorisieren	ein Faktor	eine Division
Hamiltonkreis	der Kreis	Kanten prüfen

Der Beweis der Äquivalenz.

Zertifikat → Maschine. Die nichtdeterministische Maschine **rät** das Zertifikat Zeichen für Zeichen – $p(n)$ Schritte – und prüft es dann deterministisch. ✓
Maschine → Zertifikat. Das Zertifikat ist die **Folge der getroffenen Entscheidungen**. Sie hat höchstens die Länge der Rechnung, also $p(n)$. Der Prüfer simuliert die Maschine entlang dieser Entscheidungen – deterministisch und in polynomieller Zeit. ✓

> ***Die zweite Richtung erklärt, was Nichtdeterminismus* ist:** nicht eine geheimnisvolle Fähigkeit zu raten, sondern die Frage, ob es **irgendeinen** Rechenweg zum Erfolg gibt.

NP heißt nicht „nicht polynomiell“

Das ist der verbreitetste Irrtum des Gebiets.

NP *steht für* ***nichtdeterministisch polynomiell.***

> *Und* P $\subseteq$ NP *ist trivial: Wer ein Problem deterministisch lösen kann, braucht gar kein Zertifikat* – **man nimmt das leere Wort und prüft direkt.** *Jedes Problem in* P *liegt also in* NP.

Die Aussage „Dieses Problem ist NP“ ist außerdem meist zu schwach gemeint. *Gemeint ist üblicherweise „NP-schwer“ oder „NP-vollständig“* (Kapitel 4) **– und das ist ein ganz anderer Sachverhalt.**

Die Asymmetrie zwischen Ja und Nein

> *Die Definition ist* ***einseitig****: Sie verlangt einen Beleg für die* ***Ja****-Antwort und sagt über die Nein-Antwort nichts.*

„Diese Formel ist erfüllbar“	Beleg: die Belegung. **Kurz und prüfbar.**
„Diese Formel ist *nicht* erfüllbar“	Beleg: ? *Man müsste alle* 2^n *Belegungen ausschließen.*

> ***Diese Asymmetrie ist keine Nachlässigkeit der Definition, sondern ihr Kern*** – und aus ihr entsteht die Klasse coNP und die Frage, ob sie mit NP zusammenfällt (Kapitel 6).

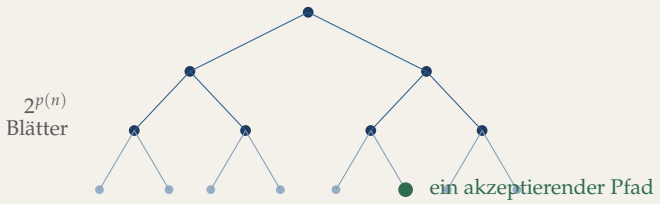
Ein Beispiel, an dem man die Asymmetrie überwinden kann.
„Ist n zusammengesetzt?“ liegt in NP – *Zertifikat: ein Faktor.*
„Ist n prim?“ liegt ***auch*** *in* NP – **aber der Beleg ist keineswegs offensichtlich.**

VAUGHAN PRATT *zeigte 1975, dass es ein kurzes Primalitätszertifikat gibt: eine Primitivwurzel modulo n zusammen mit der Faktorisierung von $n-1$ – und für jeden Faktor rekursiv dasselbe.* **Die Prüfung ist eine Kette von Potenzrechnungen.**

Damit lag Primalität in NP ∩ coNP – **ein starkes Indiz dafür, dass sie in** P **liegt, und tatsächlich wurde das 2002 bewiesen** (Kapitel 2).

Faustregel: *Probleme in* NP ∩ coNP *sind fast nie NP-vollständig und liegen erfahrungsgemäß oft in* P. **Faktorisieren ist der prominenteste noch offene Fall.**

Der Suchbaum – warum „Raten" nichts kostet und alles kostet



Eine nichtdeterministische Maschine akzeptiert, wenn ***mindestens ein*** *Blatt akzeptierend ist.* **Die *Tiefe* des Baums ist polynomiell, seine *Breite* exponentiell.**

NP **misst**	die **Tiefe** – die Länge *eines* Pfades
Die Simulation kostet	die **Breite** – man muss alle Pfade absuchen

Die naive deterministische Simulation braucht $\mathcal{O}(2^{p(n)})$ *Zeit* – **also** NP ⊆ EXP**.** *Ob es besser geht, ist die offene Frage.*

Die nichtdeterministische Maschine ist keine Maschine

Man kann sie nicht bauen. *Sie ist ein* ***Definitionsmittel****, kein Gerät.*

> *Es gibt eine verbreitete Fehlvorstellung, ein Quantenrechner sei so etwas wie eine nichtdeterministische Maschine, die alle Pfade gleichzeitig verfolgt.* **Das ist falsch.**

Ein Quantenrechner hat zwar alle Pfade gleichzeitig in Superposition – aber beim Messen erhält man ***einen zufälligen****, nicht den akzeptierenden.* **Der ganze Aufwand der Quantenalgorithmik besteht darin, durch Interferenz die falschen Pfade auszulöschen**, *und das gelingt nur bei sehr besonderer Struktur* (Band 15).

> *Es wird allgemein vermutet, dass* NP $\not\subseteq$ BQP – **Quantenrechner lösen NP-vollständige Probleme vermutlich *nicht* schnell.** *Grovers Algorithmus liefert nur den Faktor* $\sqrt{2^n}$*, also eine Halbierung des Exponenten.*

Aufgabe 3.1.

(a) Weisen Sie für mehrere Probleme die Zugehörigkeit zu NP formal nach.
(b) Untersuchen Sie Probleme, bei denen das Zertifikat nicht offensichtlich ist.
(c) Zeigen Sie, dass NP $\subseteq$ PSPACE und NP $\subseteq$ EXP, und diskutieren Sie die Schranken.

Lösung Schritt für Schritt.

(a) Zugehörigkeit zu NP **nachweisen.**

> *Das Schema ist immer dasselbe:* ***Zertifikat angeben, Länge abschätzen, Prüfverfahren angeben, dessen Laufzeit abschätzen.*** Vier Zeilen.

Problem 1 – SAT.

Zertifikat	Belegung $c \in \{0,1\}^n$ der n Variablen
Länge	$n \leq \lvert w\rvert$ ✓
Prüfung	Formel auswerten: jedes Literal nachschlagen, Klauseln prüfen
Aufwand	$\mathcal{O}(\lvert w\rvert)$ ✓

Problem 2 – Clique. *„Hat G eine Clique der Größe $\geq k$?“*

Zertifikat	Eckenmenge S mit $\lvert S\rvert = k$
Länge	$\mathcal{O}(n \log n)$ ✓
Prüfung	alle $\binom{k}{2}$ Paare auf Kante prüfen
Aufwand	$\mathcal{O}(n^2)$ ✓

Problem 3 – Hamiltonkreis.

Zertifikat	Eckenfolge $v_1, \ldots, v_n$
Prüfung	(i) alle Ecken genau einmal, (ii) $(v_i, v_{i+1}) \in E$, (iii) $(v_n, v_1) \in E$
Aufwand	$\mathcal{O}(n \log n)$ mit Sortieren ✓

Eulerkreis gegen Hamiltonkreis

Zwei Probleme, die sich fast gleich anhören und völlig verschieden sind.

	Eulerkreis	**Hamiltonkreis**
durchläuft	**jede Kante** einmal	**jede Ecke** einmal
Kriterium	alle Grade gerade	keines bekannt
Komplexität	$\mathcal{O}(m)$, in P	**NP-vollständig**
Jahr	**Euler 1736**	offen bis heute

Euler löste seines 1736 mit einem lokalen Kriterium: Man schaut jede Ecke einzeln an. Beim Hamiltonkreis gibt es kein solches Kriterium **– und es ist ein Kernbefund der Komplexitätstheorie, dass es vermutlich keines geben kann.**

Der Unterschied zwischen „jede Kante“ und „jede Ecke“ entscheidet über polynomiell und exponentiell. **Man sieht ihn dem Problem nicht an.**

(b) Nichtoffensichtliche Zertifikate.
Fall 1 – Primalität (Pratt, 1975).
Zu zeigen: „n ist prim“ hat ein kurzes Zertifikat.

***Grundlage** (Lucas-Test): n ist genau dann prim, wenn es ein a gibt mit*

$$a^{n-1} \equiv 1 \pmod{n} \quad \text{und} \quad a^{(n-1)/q} \not\equiv 1 \pmod{n}$$

für jeden Primfaktor q von $n-1$.

Das Zertifikat besteht also aus:

1. der Zahl a,
2. der Faktorisierung von $n-1$ in Primfaktoren $q_1, \ldots, q_r$,
3. **rekursiv** einem Primalitätszertifikat für jedes q_i.

Warum das kurz bleibt.
Sei $C(n)$ die Zertifikatlänge. Die Faktoren q_i erfüllen $\prod q_i^{e_i} = n-1$, also

$$\sum_i \log q_i \le \log n.$$

Die rekursiven Teilzertifikate haben zusammen also nicht mehr „Bitmasse“ als $\log n$ – **und per Induktion ergibt sich** $C(n) = \mathcal{O}(\log^2 n)$**.** ✓

Die Prüfung besteht aus $\mathcal{O}(\log n)$ modularen Potenzierungen, jede in $\mathcal{O}(\log^3 n)$ – **polynomiell.** ✓

Ein Beispiel für $n = 7$: Nimm $a = 3$. Dann $n - 1 = 6 = 2 \cdot 3$.

$$3^6 = 729 = 104 \cdot 7 + 1 \equiv 1. \quad ✓$$

$$3^{6/2} = 27 \equiv 6 \not\equiv 1, \qquad 3^{6/3} = 9 \equiv 2 \not\equiv 1. \quad ✓$$

Also ist 7 prim – und der Beleg besteht aus drei Zeilen Rechnung. ✓

Fall 2 – Nichtisomorphie von Graphen.

„$G_1 \cong G_2$“ liegt in NP *– Zertifikat ist die Permutation.* **Aber „$G_1 \ncong G_2$“?** *Man kennt kein kurzes Zertifikat.*

Es ist offen, ob Graphnichtisomorphie in NP *liegt.* **Bekannt ist aber etwas Erstaunliches:** *Sie hat ein **interaktives** Beweissystem* (Goldreich, Micali, Wigderson, 1986).

Der Prüfer wählt zufällig einen der beiden Graphen, permutiert ihn zufällig und fragt den Beweiser, welcher es war. **Sind die Graphen nichtisomorph, kann der Beweiser das immer sagen; sind sie isomorph, rät er.**

Nach k Runden irrt man mit Wahrscheinlichkeit 2^{-k}. **Ein Beleg, den man nicht aufschreiben, aber erfragen kann.**

(c) Die oberen Schranken.
Behauptung 1 – NP $\subseteq$ PSPACE**.**
Sei $L \in$ NP *mit Zertifikatlänge* $p(n)$ *und Prüfer* V*.*

```
für jedes c aus {0,1}^p(n) do
  if V(w,c) akzeptiert then akzeptiere;
  lösche c und gehe zum nächsten
verwirf
```

Der Platz für c ***wird wiederverwendet.*** Man braucht $p(n)$ Bits für das aktuelle Zertifikat plus den Platz von V – **beides polynomiell.** ✓

Die Zeit ist $2^{p(n)}$*, aber danach fragt niemand.* **Das ist die zentrale Eigenschaft von Platz: Er ist wiederverwendbar.**

Behauptung 2 – NP $\subseteq$ EXP**.**
Dieselbe Schleife, nur die Zeit gezählt:

$$2^{p(n)} \cdot q(n) = 2^{\mathcal{O}(n^k)}. \qquad \checkmark$$

Die vollständige Lage.

$$\mathrm{P} \subseteq \mathrm{NP} \subseteq \mathrm{PSPACE} \subseteq \mathrm{EXP}.$$

Inklusion	**echt?**	**Status**
P $\subseteq$ NP	offen	Millenniumsproblem
NP $\subseteq$ PSPACE	offen	
PSPACE $\subseteq$ EXP	offen	
P $\subsetneq$ EXP	**ja**	Zeithierarchiesatz

Die letzte Zeile ist der einzige harte Befund. Man weiß, dass die Kette *irgendwo* echt ist – aber nicht wo. **Mindestens eine der drei oberen Inklusionen muss echt sein**, *und niemand weiß, welche.*

Warum das so unbefriedigend ist

Man kann beweisen, dass ein Unterschied besteht, ohne ihn lokalisieren zu können.

> *Das ist eine ungewöhnliche Situation in der Mathematik.* **Sie rührt daher, dass der Beweis von** P $\neq$ EXP **ein *Diagonalargument* ist** (Kapitel 9) *– und Diagonalargumente sind grob: Sie trennen weit auseinanderliegende Klassen, aber keine benachbarten.*

Kapitel 14 zeigt, dass das kein Zufall ist: BAKER, GILL *und* SOLOVAY *bewiesen 1975, dass Diagonalisierung* P *gegen* NP ***grundsätzlich nicht*** *entscheiden kann.*

Die Methode, die P $\neq$ EXP *liefert, ist beweisbar unfähig,* P $\neq$ NP *zu liefern.* **Und eine bessere Methode kennt niemand.**

Arora und Barak[2] und Sipser[24] führen beide Definitionen sorgfältig ineinander über; Papadimitriou[21] betont die Suchproblem-Sicht; Aaronson[1] ist die beste allgemeinverständliche Übersicht über die Frage.

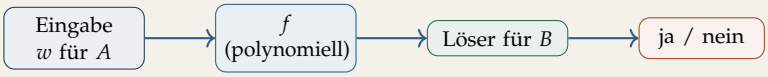
4 Reduktionen und NP-Vollständigkeit

Härte übertragen

Definition 4.1. A ist **polynomiell reduzierbar** auf B, geschrieben $A \leq_{\mathrm{p}} B$, wenn es eine in polynomieller Zeit berechenbare Funktion f gibt mit

$$w \in A \iff f(w) \in B \qquad \text{für alle } w.$$

In Worten: *Wer B lösen kann, kann auch A lösen* – **man übersetzt die Frage und stellt sie neu.** *Also ist B* ***mindestens so schwer*** *wie A.*



Satz 4.2 (Grundeigenschaften).

$A \leq_{\mathrm{p}} B$ *und* $B \in \mathrm{P}$	$\Longrightarrow A \in \mathrm{P}$
$A \leq_{\mathrm{p}} B$ *und* $B \in \mathrm{NP}$	$\Longrightarrow A \in \mathrm{NP}$
$A \leq_{\mathrm{p}} B$ *und* $B \leq_{\mathrm{p}} C$	$\Longrightarrow A \leq_{\mathrm{p}} C$ (***transitiv***)
$A \leq_{\mathrm{p}} B$ *und* $A \notin \mathrm{P}$	$\Longrightarrow B \notin \mathrm{P}$

Beweis der ersten Zeile. *Sei f in $p(n)$ berechenbar und B in $q(m)$ entscheidbar. Dann ist $|f(w)| \leq p(|w|)$, und der Gesamtaufwand beträgt*

$$p(n) + q\big(p(n)\big) \quad \text{– ein Polynom.} \qquad \checkmark$$

Die vierte Zeile ist die praktisch benutzte. Man liest die Reduktion **rückwärts**: Wer zeigen will, dass B schwer ist, reduziert ein bekanntes schweres A darauf.

Die Richtung ist die häufigste Fehlerquelle

$A \leq_p B$ bedeutet: B ist mindestens so schwer wie A.

B ist schwer zeigen	Reduziere **von** bekanntem schwerem A **auf** B
B ist leicht zeigen	Reduziere **von** B **auf** bekanntes leichtes C

Wer die Richtung vertauscht, beweist gar nichts. **Merkregel: Man übersetzt das *alte* Problem in die Sprache des *neuen*** – *nicht umgekehrt.*

Zweite Fallgrube: *Die Reduktion muss in **beide** Richtungen korrekt sein.* **Aus $w \in A \Longrightarrow f(w) \in B$ allein folgt nichts** – *man muss auch zeigen, dass $f(w) \in B \Longrightarrow w \in A$.*

In der Praxis vergisst man fast immer die zweite Richtung, und dort steckt fast immer die eigentliche Arbeit.

Die schwersten Probleme in NP

Definition 4.3. Eine Sprache B heißt

NP-schwer	wenn $A \leq_p B$ für **alle** $A \in \mathrm{NP}$
NP-vollständig	wenn zusätzlich $B \in \mathrm{NP}$

NP-schwer heißt: mindestens so schwer wie alles in NP. Ein NP-schweres Problem muss nicht selbst in NP liegen – das Halteproblem ist NP-schwer und liegt weit darüber.

Satz 4.4. *Ist B NP-vollständig, so gilt*

$$B \in \mathrm{P} \iff \mathrm{P} = \mathrm{NP}.$$

„$\Leftarrow$“ ist trivial. „$\Rightarrow$“: Sei $B \in \mathrm{P}$ und $A \in \mathrm{NP}$ beliebig. Wegen $A \leq_p B$ und der ersten Zeile des Satzes oben folgt $A \in \mathrm{P}$. ✓

Das ist die eigentliche Bedeutung der NP-Vollständigkeit. Alle diese Probleme stehen oder fallen **gemeinsam**. *Wer eines von ihnen schnell löst, hat alle gelöst – und die Millennium-Million verdient.*

*Umgekehrt: Seit fünfzig Jahren versuchen Tausende, für **eines** dieser Tausenden Probleme ein schnelles Verfahren zu finden.* **Das ist das stärkste (nicht beweisende) Argument für** $\mathrm{P} \neq \mathrm{NP}$**.**

4.1 Der Satz von Cook und Levin

Das erste vollständige Problem

*Die Definition der NP-Vollständigkeit ist zunächst leer: Man muss **alle** Probleme in* NP *auf eines reduzieren.* **Dass das überhaupt gelingen kann, ist überraschend.**

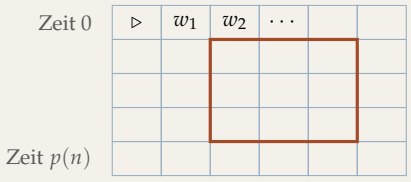
Definition 4.5. SAT ist die Sprache aller aussagenlogischen Formeln in konjunktiver Normalform, die eine erfüllende Belegung besitzen.

Satz 4.6 (Cook 1971, Levin 1973)**.** SAT *ist NP-vollständig.*

Die Beweisidee – die Rechnung als Formel.

Sei $A \in \mathrm{NP}$ mit nichtdeterministischer Maschine M und Laufzeit $p(n)$. Man baut zu jeder Eingabe w eine Formel φ_w, die genau dann erfüllbar ist, wenn M die Eingabe w akzeptiert.

Der Kunstgriff: das Rechentableau.
Eine Rechnung von M auf w ist eine Folge von $p(n)$ Konfigurationen, jede der Länge $p(n)$. Man schreibt sie als Tabelle:



Die Variablen.

$$x_{i,j,s} = 1 \iff \text{Feld } j \text{ zur Zeit } i \text{ trägt das Symbol } s.$$

Das sind $\mathcal{O}(p(n)^2)$ Variablen – polynomiell viele. ✓

Die vier Klauselgruppen.

φ_{Zelle}	Jedes Feld trägt **genau ein** Symbol
φ_{Start}	Die erste Zeile ist die Startkonfiguration mit Eingabe w
φ_{Ende}	Irgendwo steht ein akzeptierender Zustand
φ_{Schritt}	**Jedes 2×3-Fenster ist legal**

Warum die vierte Gruppe genügt – der eigentliche Kern.

> *Ein Rechenschritt ändert nur* ***lokal*** *etwas: an der Kopfposition und höchstens einem Nachbarfeld.* **Also lässt sich die Korrektheit des Übergangs *Fenster für Fenster* prüfen** *– mit einem Fenster der Höhe 2 und Breite 3.*

Man listet alle legalen Fensterbelegungen auf (endlich viele, da $|\Gamma|$ und $|Q|$ fest sind) und schreibt für jedes Fenster eine Klausel: „es ist eine der legalen".

> ***Sind alle Fenster legal, so ist das ganze Tableau eine korrekte Rechnung.*** Das ist nicht selbstverständlich und muss bewiesen werden – aber es stimmt, weil ein Schritt außerhalb der Kopfumgebung nichts ändert.

Die Bilanz.

Größe	**Anzahl**	**Bemerkung**
Variablen	$\mathcal{O}(p(n)^2)$	
Fenster	$\mathcal{O}(p(n)^2)$	
Klauseln je Fenster	$\mathcal{O}(1)$	konstant viele
Formellänge	$\mathcal{O}(\mathbf{p(n)^2})$	**polynomiell ✓**

> *Und die Formel ist in polynomieller Zeit* ***hinschreibbar*** **– man muss nur die Übergangstabelle von M kennen.**

Eine erfüllende Belegung von φ_w ist genau ein akzeptierendes Rechentableau **– und umgekehrt. ✓**

Zwei Entdeckungen im Kalten Krieg

Stephen Cook trug seinen Satz im Mai 1971 auf einem Symposium in Shaker Heights, Ohio, vor. *Der Aufsatz heißt „The Complexity of Theorem-Proving Procedures" und ist elf Seiten lang.*

> *Cook hatte zwei Jahre zuvor in Berkeley* ***keine Festanstellung*** *bekommen –* **man hielt seine Arbeit für zu wenig relevant.** *Er ging nach Toronto und blieb dort sein Leben lang.*

Berkeley hat sich später öffentlich für diese Entscheidung entschuldigt. **Cook erhielt 1982 den Turing-Award.**

> *Unabhängig und ohne Kenntnis davon fand* Leonid Levin *in Moskau dasselbe Ergebnis.* **Sein Aufsatz erschien 1973 auf zwei Seiten in „Problemy Peredatschi Informazii"**, *unter dem Titel „Universelle Suchprobleme".*

Levins Fassung ist in mancher Hinsicht die stärkere: *Er behandelte* ***Suchprobleme*** *statt Entscheidungsprobleme und gab gleich* ***sechs*** *vollständige Probleme an.* **Sein Zugang über „universelle Suche“ ist konstruktiver.**

Levin durfte in der Sowjetunion nicht promovieren – er galt als politisch unzuverlässig und war Schüler von KOLMOGOROW*, der selbst in Ungnade stand.* **Er emigrierte 1978 in die USA und lehrt seither in Boston.**

Der Satz heißt heute nach beiden. **Es ist einer der wenigen Fälle, in denen sich die westliche Fachwelt die Mühe gemacht hat, eine sowjetische Parallelentdeckung nachträglich anzuerkennen.**

Cook stellte in seinem Aufsatz übrigens ausdrücklich die Frage, ob $\mathrm{P} = \mathrm{NP}$ – *mit anderer Notation, aber unmissverständlich.* **Er hielt es für unwahrscheinlich und schrieb, ein Beweis wäre „ein bedeutender Durchbruch“.**

Aufgabe 4.1.

(a) Führen Sie die Reduktion $\textsc{Sat} \leq_{\mathrm{p}}$ 3-SAT vollständig durch.
(b) Reduzieren Sie 3-SAT auf das Cliquenproblem.
(c) Untersuchen Sie, warum 2-SAT in P liegt – und wo die Grenze verläuft.

Lösung Schritt für Schritt.

(a) Von SAT zu 3-SAT.

3-SAT *ist* SAT *mit der Einschränkung, dass jede Klausel* ***genau drei*** *Literale hat.*

Schritt 1 – Was zu tun ist.
Man muss jede Klausel beliebiger Länge durch mehrere Klauseln der Länge 3 ersetzen, ohne die Erfüllbarkeit zu ändern.
Schritt 2 – Klauseln der Länge $k \geq 4$**.**
Aus

$$(\ell_1 \vee \ell_2 \vee \ell_3 \vee \cdots \vee \ell_k)$$

wird mit neuen Hilfsvariablen $y_1, \ldots, y_{k-3}$*:*

$$(\ell_1 \vee \ell_2 \vee y_1) \wedge (\overline{y_1} \vee \ell_3 \vee y_2) \wedge (\overline{y_2} \vee \ell_4 \vee y_3) \wedge \cdots \wedge (\overline{y_{k-3}} \vee \ell_{k-1} \vee \ell_k).$$

Schritt 3 – Warum das äquivalent ist.
„⇒“: Sei die Originalklausel erfüllt, etwa durch ℓ_j*. Man setze*

$$y_1 = \cdots = y_{j-2} = 1, \qquad y_{j-1} = \cdots = y_{k-3} = 0.$$

Dann sind die Klauseln ***vor*** ℓ_j *durch ihr* y_i *erfüllt, die Klausel* ***mit*** ℓ_j *durch* ℓ_j *selbst, und die* ***danach*** *durch ihr* $\overline{y_i}$*.* ✓

„⇐“: Sei kein ℓ_j wahr. Die erste Klausel erzwingt $y_1 = 1$, die zweite dann $y_2 = 1$, und so fort bis $y_{k-3} = 1$. **Die letzte Klausel lautet dann** $(0 \vee \ell_{k-1} \vee \ell_k)$ **und ist unerfüllt. Widerspruch. ✓**

> ***Die Hilfsvariablen wirken wie eine Kette von Dominosteinen:*** Sobald ein echtes Literal wahr wird, bricht die Kette ab.

Schritt 4 – Klauseln der Länge 1 und 2.

(ℓ)	$(\ell \vee z_1 \vee z_2) \wedge (\ell \vee \overline{z_1} \vee z_2) \wedge (\ell \vee z_1 \vee \overline{z_2}) \wedge (\ell \vee \overline{z_1} \vee \overline{z_2})$
$(\ell_1 \vee \ell_2)$	$(\ell_1 \vee \ell_2 \vee z) \wedge (\ell_1 \vee \ell_2 \vee \overline{z})$

In beiden Fällen kommen alle Belegungen der Hilfsvariablen vor **– also muss** ℓ **bzw.** $\ell_1 \vee \ell_2$ **selbst wahr sein. ✓**

Schritt 5 – Aufwand.
Eine Klausel der Länge k wird zu $k-2$ Klauseln mit $k-3$ neuen Variablen. **Die Gesamtlänge wächst höchstens linear:**

$$|f(\varphi)| = \mathcal{O}(|\varphi|). \qquad \checkmark$$

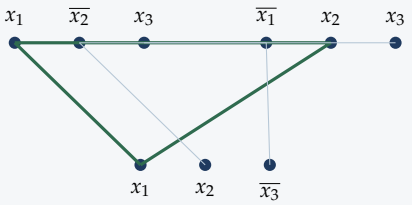
> *Diese Reduktion ist **sehr** effizient – linear statt bloß polynomiell.* **Deshalb überträgt sie auch feinere Härteaussagen**, *etwa die Exponentialzeithypothese* (Kapitel 12).

(b) Von 3-SAT zum Cliquenproblem.
Gegeben eine Formel φ mit m Klauseln zu je drei Literalen. Gesucht ein Graph G und eine Zahl k mit: φ erfüllbar $\iff$ G hat eine Clique der Größe k.
Schritt 1 – Die Konstruktion.

Ecken	Für jedes Literalvorkommen eine Ecke – also $3m$ Stück, in m Dreiergruppen
Kanten	Zwischen u und v genau dann, wenn (i) sie in **verschiedenen** Klauseln liegen **und** (ii) sie **nicht** komplementär sind
Schwelle	$k := m$

Schritt 2 – Ein Beispiel.

$$\varphi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}).$$



Clique der Größe 3

Die grüne Clique besteht aus x_1 (Klausel 1), x_2 (Klausel 2), x_1 (Klausel 3). **Belegung:** $x_1 = x_2 = 1$. ✓

Schritt 3 – Der Beweis, Richtung „⇒“.
*Sei φ erfüllbar. Wähle in jeder Klausel **ein** wahres Literal und nimm dessen Ecke.*

m Ecken	je eine pro Klausel ✓
verschiedene Klauseln	nach Konstruktion ✓
nicht komplementär	beide sind **wahr** unter derselben Belegung, können also nicht x und $\overline{x}$ sein ✓

Also sind alle Paare verbunden – **eine Clique der Größe** m. ✓

Schritt 4 – Richtung „⇐“.
*Sei C eine Clique der Größe m. Da Ecken derselben Klausel nie verbunden sind, enthält C aus **jeder** Klausel genau eine Ecke.*

*Man setze jedes Literal in C auf **wahr**.* **Das ist widerspruchsfrei, weil** C **keine komplementären Paare enthält.** *Restliche Variablen beliebig.*

Jede Klausel enthält nun ein wahres Literal. φ **ist erfüllt.** ✓

Schritt 5 – Aufwand.

$3m$ Ecken, $\mathcal{O}(m^2)$ Kanten, konstruierbar in $\mathcal{O}(m^2)$. ✓

Damit ist Clique NP-schwer – und da es offensichtlich in NP liegt (Kapitel 3), **NP-vollständig.** *Kapitel 5 setzt diese Kette fort.*

(c) Warum 2-SAT leicht ist.

Der Sprung von 2 auf 3 Literale je Klausel ist der Sprung von P *zu NP-vollständig.* **Das ist bemerkenswert, und der Grund ist strukturell.**

Schritt 1 – Die Klausel als Implikation.
*Eine Klausel mit zwei Literalen ist äquivalent zu **zwei** Implikationen:*

$$(\ell_1 \vee \ell_2) \equiv (\overline{\ell_1} \to \ell_2) \equiv (\overline{\ell_2} \to \ell_1).$$

Das ist der ganze Trick. Eine Zweierklausel sagt: „Wenn das eine falsch ist, muss das andere wahr sein." **Bei drei Literalen gibt es keine solche eindeutige Folgerung** – *man erhält $\overline{\ell_1} \to (\ell_2 \vee \ell_3)$, und die rechte Seite ist keine Zuweisung, sondern wieder eine Wahl.*

Schritt 2 – Der Implikationsgraph.
Ecken: die $2n$ Literale $x_1, \overline{x_1}, \ldots, x_n, \overline{x_n}$. Kanten: die Implikationen.
Beispiel: $(x_1 \vee x_2) \wedge (\overline{x_1} \vee x_3) \wedge (\overline{x_2} \vee \overline{x_3})$

Klausel	Kante 1	Kante 2
$(x_1 \vee x_2)$	$\overline{x_1} \to x_2$	$\overline{x_2} \to x_1$
$(\overline{x_1} \vee x_3)$	$x_1 \to x_3$	$\overline{x_3} \to \overline{x_1}$
$(\overline{x_2} \vee \overline{x_3})$	$x_2 \to \overline{x_3}$	$x_3 \to \overline{x_2}$

Satz 4.7 (Aspvall, Plass, Tarjan 1979)**.** *φ ist genau dann unerfüllbar, wenn es eine Variable x gibt, für die x und $\overline{x}$ in **derselben** starken Zusammenhangskomponente des Implikationsgraphen liegen.*

Warum das stimmt.

*Liegen x und $\overline{x}$ in derselben Komponente, so gilt $x \to \overline{x}$ **und** $\overline{x} \to x$.* **Beide Belegungen führen auf einen Widerspruch** – *unerfüllbar.*

*Umgekehrt: Liegen sie stets getrennt, so kann man eine Belegung konstruieren, indem man die Komponenten topologisch sortiert und jedem Literal den Wert der **späteren** Komponente gibt.* ✓

Schritt 3 – Der Aufwand.

Schritt	Aufwand	Verfahren
Graph aufbauen	$\mathcal{O}(n+m)$	
Starke Komponenten	$\mathcal{O}(n+m)$	Tarjan (Band 1)
Prüfung	$\mathcal{O}(n)$	
gesamt	$\mathcal{O}(\mathbf{n+m})$	**linear**

*Nicht bloß polynomiell – **linear**.* **2-SAT ist eines der leichtesten interessanten Probleme überhaupt.**

Die Grenze verläuft nicht bei der Klauselläng allein

Wo genau liegt der Übergang? *Die Antwort ist überraschend scharf.*

Variante	Komplexität	Bemerkung
2-SAT	$\mathcal{O}(n+m)$	Implikationsgraph
Horn-SAT	$\mathcal{O}(n+m)$	**höchstens ein positives Literal**
XOR-SAT	$\mathcal{O}(n^3)$	Gaußelimination über $\mathbb{F}_2$
3-SAT	**NP-vollständig**	
Max-2-SAT	**NP-vollständig**	*obwohl 2-SAT leicht ist!*

> ***Die vorletzte Zeile ist die lehrreichste:*** *Zu fragen, ob* alle *Klauseln erfüllbar sind, ist leicht – zu fragen, wie viele man* höchstens *erfüllen kann, ist schwer.*

THOMAS SCHAEFER bewies 1978 einen Dichotomiesatz: *Jede Variante von* SAT *mit festgelegten erlaubten Klauseltypen ist* ***entweder*** *in* P ***oder*** *NP-vollständig* – **und er gab genau sechs Familien an, die leicht sind.**

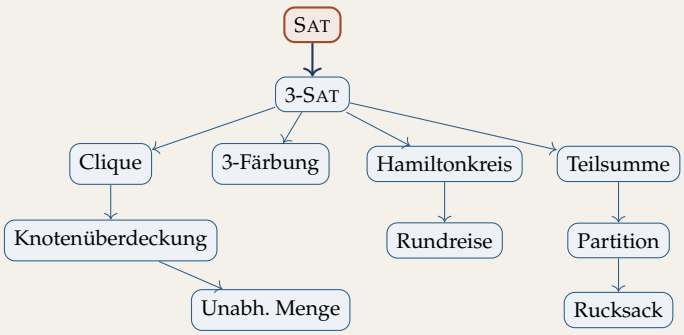
Dazwischen gibt es nichts. **Das ist bemerkenswert, weil Ladners Satz** (Kapitel 9) **zeigt, dass es *im Allgemeinen* sehr wohl Zwischenklassen gibt** – *nur eben nicht bei den* SAT*-Varianten.*

Cooks Aufsatz[4] und Levins zweiseitige Arbeit[19] sind beide lesenswert; Hopcroft, Motwani und Ullman[13] liefern die formalsprachlichen Grundlagen; Sipser[24] führt den Tableau-Beweis am klarsten aus; Arora und Barak[2] geben die moderne Fassung.

5 Der Zoo der vollständigen Probleme

Karps einundzwanzig

*Cooks Satz lieferte **ein** vollständiges Problem.* **Erst Richard Karps Arbeit von 1972 machte daraus ein Werkzeug** – *indem sie zeigte, dass die Vollständigkeit sich in alle Richtungen ausbreitet.*



Jeder Pfeil ist eine polynomielle Reduktion. Weil Reduktionen transitiv sind (Kapitel 4), erbt jedes Problem im Baum die Vollständigkeit von SAT – **man muss nur *einen* Pfeil neu beweisen.**

Das ist der Grund, warum die Liste so schnell wuchs. **1972 waren es 21 Probleme; 1979 sammelten Garey und Johnson über 300; heute sind es viele Tausend.**

Ein Aufsatz, der ein Fach begründete

Richard Karp war 1972 Professor in Berkeley. *Er hatte Cooks Vortrag von 1971 gehört und erkannt, was daraus folgt.*

*Sein Aufsatz „Reducibility Among Combinatorial Problems" zeigte, dass **einundzwanzig** bekannte Probleme aus der Optimierung, Graphentheorie und Logik allesamt NP-vollständig sind.*

Die Wirkung war unmittelbar.

Vorher hatte jede dieser Fachgemeinden ihr eigenes ungelöstes Problem und hielt es für eine örtliche Schwierigkeit. **Karp zeigte, dass alle dasselbe Problem hatten.**

Karp hat später erzählt, er habe vor allem Probleme ausgewählt, an denen ***er selbst oder Kollegen jahrelang gescheitert*** *waren* – **das Rundreiseproblem, die Knotenüberdeckung, die Cliquensuche.**

„Es war eine große Erleichterung zu erfahren, dass wir nicht bloß zu dumm gewesen waren."

Karp erhielt 1985 den Turing-Award. *In der Begründung hieß es, seine Arbeit habe „der Theorie der NP-Vollständigkeit ihre zentrale Stellung verschafft".*

Ein Nebenbefund: Karp zeigte auch, welche Probleme ***nicht*** *in die Liste gehören.* **Das Matchingproblem etwa – Edmonds hatte 1965 gezeigt, dass es in** P **liegt** (Kapitel 2)*, obwohl es dem Cliquenproblem täuschend ähnlich sieht.*

Solche Paare – ein leichtes und ein schweres Problem, die einander gleichen – sind das Lehrreichste am ganzen Gebiet. **Aufgabe (c) sammelt sie.**

Die Standardliste

Problem	Frage
SAT, 3-SAT	Ist die Formel erfüllbar?
Clique	Gibt es k paarweise verbundene Ecken?
Unabhängige Menge	Gibt es k paarweise unverbundene Ecken?
Knotenüberdeckung	Decken k Ecken alle Kanten ab?
3-Färbbarkeit	Genügen drei Farben?
Hamiltonkreis	Gibt es einen Kreis durch alle Ecken?
Rundreise	Gibt es eine Tour der Länge $\leq B$?
Teilsumme	Summiert eine Teilmenge zu t?
Partition	Lässt sich die Menge halbieren?
Rucksack	Wert $\geq W$ bei Gewicht $\leq G$?
Behälterpacken	Genügen k Behälter?
Mengenüberdeckung	Decken k Mengen alles ab?
Ganzz. Programmieren	Gibt es eine ganzzahlige Lösung?

Die Optimierungsfassung ist nicht dasselbe

Alle Probleme oben sind *Entscheidungs*probleme – sie fragen nach ja oder nein.

Entscheidung	„Gibt es eine Clique der Größe $\geq k$?“
Suche	„Nenne eine solche Clique.“
Optimierung	„Wie groß ist die größte Clique?“

Die Komplexitätstheorie arbeitet mit der ersten Fassung, weil Sprachen Ja-Nein-Mengen sind (Band 2, Kapitel 1). **Die anderen beiden sind aber nicht wesentlich schwerer.**

Von Entscheidung zu Optimierung: *binäre Suche über k* – $\mathcal{O}(\log n)$ **Aufrufe.** ✓
Von Entscheidung zu Suche: *Man baut die Lösung Stück für Stück, indem man jeweils fragt, ob es noch eine Lösung gibt, wenn man ein Element festlegt* (**Selbstreduzierbarkeit**) – $\mathcal{O}(n)$ **Aufrufe.** ✓

Alle drei Fassungen sind also ***polynomiell äquivalent*** **– man darf sie im Alltag verwechseln, muss den Unterschied aber kennen.**

Aufgabe 5.1.

(a) Führen Sie die Kette Clique → Unabhängige Menge → Knotenüberdeckung durch.
(b) Reduzieren Sie 3-SAT auf die Teilsumme und rechnen Sie ein Beispiel.
(c) Stellen Sie Paare ähnlicher Probleme gegenüber, von denen eines leicht und eines schwer ist.

Lösung Schritt für Schritt.

(a) Drei Probleme, eine Konstruktion.
Schritt 1 – Clique und Unabhängige Menge.

Definition 5.1. Eine **unabhängige Menge** ist eine Eckenmenge ohne Kanten dazwischen.

Die Reduktion ist die Komplementbildung des Graphen.

Sei $\overline{G}$ der Graph mit denselben Ecken und genau den ***nicht*** *vorhandenen Kanten.*

$$S \text{ ist Clique in } G \iff S \text{ ist unabhängig in } \overline{G}.$$

Denn: „alle Paare verbunden in G“ heißt „kein Paar verbunden in $\overline{G}$“. ✓

$$f(G,k) := (\overline{G},k), \qquad \text{berechenbar in } \mathcal{O}(n^2). \qquad ✓$$

Schritt 2 – Unabhängige Menge und Knotenüberdeckung.

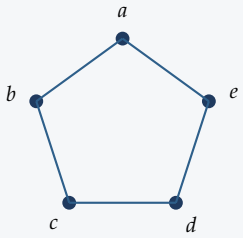
Definition 5.2. Eine **Knotenüberdeckung** ist eine Eckenmenge, die von **jeder** Kante mindestens einen Endpunkt enthält.

Satz 5.3. *S ist genau dann unabhängig, wenn $V \setminus S$ eine Knotenüberdeckung ist.*

Der Beweis in beide Richtungen.
„⇒“: Sei S unabhängig und (u,v) eine Kante. Wären beide in S, gäbe es eine Kante innerhalb von S – Widerspruch. Also liegt mindestens einer in $V \setminus S$. ✓
„⇐“: Sei $V \setminus S$ eine Überdeckung und $u,v \in S$. Wäre (u,v) eine Kante, so müsste einer der beiden in $V \setminus S$ liegen – Widerspruch. ✓

$$f(G,k) := (G,\ n-k). \qquad ✓$$

Schritt 3 – Ein Beispiel, das alles zusammenfasst.
G: fünf Ecken a,b,c,d,e; Kanten ab, bc, cd, de, ea – ein Fünfeck.



Größe	Wert	Beispiel
größte Clique in G	2	$\{a,b\}$
größte unabh. Menge	2	$\{a,c\}$
kleinste Überdeckung	3	$\{b,d,e\}$

Probe: $2+3=5=n$. ✓ *Und $\{b,d,e\}$ deckt tatsächlich alle fünf Kanten: ab* (b), *bc* (b), *cd* (d), *de* (d), *ea* (e). ✓

Drei äquivalente Probleme – mit sehr verschiedenen Näherungseigenschaften

Die drei Probleme sind polynomiell äquivalent. *In der* ***Näherbarkeit*** *unterscheiden sie sich dramatisch:*

Problem	**Näherung**	**Schranke**
Knotenüberdeckung	Faktor 2	nicht besser als 1,36
Unabhängige Menge	$n^{1-\varepsilon}$	**praktisch nichts**
Clique	$n^{1-\varepsilon}$	ebenso

> ***Das ist auf den ersten Blick paradox:*** *Die Probleme sind dasselbe, und trotzdem lässt sich eines gut nähern und das andere nicht.*

Die Auflösung: *Die Reduktion $S \mapsto V \setminus S$ erhält die* ***Optima****, aber nicht die* ***Verhältnisse****.* **Eine Überdeckung, die doppelt so groß wie das Optimum ist, entspricht einer unabhängigen Menge, die um ein Vielfaches zu klein sein kann.**

> *Beispiel: $n = 100$, Optimum der Überdeckung $= 49$, unabhängige Menge $= 51$.*
> **Eine 2-Näherung der Überdeckung liefert 98 Ecken – und damit eine unabhängige Menge der Größe 2 statt 51.** *Faktor 25 statt Faktor 2.*
>
> *Deshalb braucht die Näherungstheorie einen* ***eigenen*** *Reduktionsbegriff –* **die approximationserhaltende Reduktion** (Kapitel 10).

(b) Von 3-SAT zur Teilsumme.

Definition 5.4. Teilsumme: Gegeben Zahlen $a_1, \ldots, a_m$ und t – gibt es eine Teilmenge mit Summe genau t?

> *Diese Reduktion ist die schönste der Liste, weil sie Logik in* ***Arithmetik*** *übersetzt.*
> **Der Kunstgriff besteht darin, Zahlen in Dezimaldarstellung zu bauen, deren Stellen die Klauseln zählen.**

Schritt 1 – Die Konstruktion.
Sei φ eine Formel mit n Variablen und m Klauseln. Jede Zahl hat $n + m$ Dezimalstellen:

Stellen $1 \dots n$	je eine pro Variable
Stellen $n{+}1 \dots n{+}m$	je eine pro Klausel

Für jede Variable x_i zwei Zahlen:

v_i	1 an Stelle i; 1 an jeder Klauselstelle, in der x_i **positiv** vorkommt
$\overline{v_i}$	1 an Stelle i; 1 an jeder Klauselstelle, in der $\overline{x_i}$ vorkommt

*Dazu für jede Klausel j zwei **Füllzahlen**:*

s_j, s'_j mit einer 1 bzw. einer 1 an Klauselstelle j.

Und die Zielsumme:

$$t := \underbrace{1\,1\dots1}_{n \text{ Stellen}}\ \underbrace{3\,3\dots3}_{m \text{ Stellen}}.$$

Schritt 2 – Warum das funktioniert.

Variablenstellen	Die Ziffer 1 erzwingt: aus jedem Paar $\{v_i, \overline{v_i}\}$ genau eine Zahl. **Das ist die Belegung.**
Klauselstellen	Die Zielziffer 3 muss erreicht werden. Die beiden Füllzahlen liefern zusammen höchstens 2, also muss der Beitrag der Literalzahlen mindestens **1** sein – *die Klausel ist erfüllt*.

Genauer: Eine Klausel hat drei Literale, also tragen die gewählten Literalzahlen zwischen 0 und 3 bei. **Mit zwei Füllzahlen von je 1 lässt sich die Zielziffer 3 genau dann erreichen, wenn der Literalbeitrag zwischen 1 und 3 liegt** *– man wählt so viele Füllzahlen, wie zum Auffüllen nötig sind.*

Ist der Literalbeitrag dagegen 0 – die Klausel also unerfüllt –, so kommt man mit zwei Füllzahlen höchstens auf 2 und verfehlt die 3. **Die Zielziffer 3 kodiert also genau die Erfüllung der Klausel.** ✓

Kein Übertrag kann auftreten, weil jede Stelle höchstens $3 + 3 = 6 < 10$ erreicht.
Deshalb die Dezimaldarstellung. ✓

Schritt 3 – Ein durchgerechnetes Beispiel.

$$\varphi = (x_1 \vee x_2) \wedge (\overline{x_1} \vee x_2).$$

Zwei Variablen, zwei Klauseln – also vierstellige Zahlen $x_1\,x_2 \mid K_1\,K_2$.

Zahl	x_1	x_2	K_1	K_2	**Wert**
v_1 (x_1)	1	0	1	0	1010
$\overline{v_1}$ $(\overline{x_1})$	1	0	0	1	1001
v_2 (x_2)	0	1	1	1	0111
$\overline{v_2}$ $(\overline{x_2})$	0	1	0	0	0100
s_1, s_1'	0	0	1	0	0010
s_2, s_2'	0	0	0	1	0001
Ziel t	1	1	3	3	**1133**

Zielziffer 3, zwei Füllzahlen je Klausel – der Literalbeitrag muss also ≥ 1 *sein.*
Eine Lösung.
Belegung $x_1 = 1$, $x_2 = 1$*: wähle* $v_1 = 1010$ *und* $v_2 = 0111$.

$$1010 + 0111 = 1121.$$

Fehlt zu 1133*: genau* 0012 *– also* $s_1 = 0010$ *und zweimal…*

Nehmen wir $s_1 = 0010$ *und* $s_2, s_2' = 0001$ *beide:*

$$1121 + 0010 + 0001 + 0001 = 1133. \qquad \checkmark$$

Probe an der Formel: $x_1 = x_2 = 1$ *erfüllt* $(x_1 \vee x_2)$ ✓*und* $(\overline{x_1} \vee x_2)$ ✓ **– die Belegung stimmt.**

Schritt 4 – Die Zahlen sind riesig.

Die konstruierten Zahlen haben $n + m$ *Dezimalstellen* **– ihr Wert ist also etwa** 10^{n+m}**.** *Aber ihre* ***Länge*** *ist nur* $n + m$*, und darauf kommt es an.* ✓

Genau hier liegt der Unterschied zwischen schwacher und starker NP-Vollständigkeit (Kapitel 1): **Teilsumme ist nur *schwach* NP-vollständig, weil der pseudopolynomielle Algorithmus** $\mathcal{O}(mt)$ **existiert** *– und* t *ist hier exponentiell groß.*

(c) Ähnliche Probleme, verschiedene Härte.

Die folgende Tabelle ist die lehrreichste des ganzen Bandes. In jeder Zeile stehen zwei Probleme, die sich fast nicht unterscheiden – und von denen eines in P liegt und das andere NP-vollständig ist.

leicht (P)	**schwer (NPC)**	**Unterschied**
Eulerkreis	**Hamiltonkreis**	Kanten statt Ecken
Kürzester Weg	**Längster Weg**	Minimum statt Maximum
2-Färbbarkeit	**3-Färbbarkeit**	eine Farbe mehr
2-SAT	3-**SAT**	ein Literal mehr
Matching	**3-dim. Matching**	Paare statt Tripel
Min. Spannbaum	**Steinerbaum**	Zusatzecken erlaubt
Lin. Programm	**Ganzz. Programm**	Ganzzahligkeit
Max. Fluss	**Mehrgüterfluss**	mehrere Warensorten
Prim/Zusammenges.	**Faktorisieren***	Zerlegung statt Test

* *Faktorisieren ist vermutlich nicht NP-vollständig, aber auch nicht in* P.

Drei Zeilen im Detail.

Zeile 2 – Kürzester gegen längster Weg.

Kürzeste Wege lassen sich schrittweise aufbauen, weil Teilwege kürzester Wege wieder kürzeste Wege sind (Band 1, Kapitel 11).

*Bei **längsten** Wegen gilt das nicht:* **Ein Teilweg eines längsten Weges ist nicht der längste zwischen seinen Endpunkten** – *sonst würde er den Gesamtweg womöglich in einen Kreis führen.*

Das Optimalitätsprinzip von Bellman versagt – **und mit ihm alle Verfahren, die darauf beruhen.**

Zeile 3 – Zwei gegen drei Farben.

2-Färbbarkeit ist entscheidbar in $\mathcal{O}(n+m)$: Man färbt eine Ecke, dann alle Nachbarn anders, und prüft auf Widerspruch – **eine Breitensuche.** *Ein Graph ist genau dann 2-färbbar, wenn er keinen Kreis ungerader Länge enthält.*

*Bei drei Farben gibt es beim Färben einer Ecke eine **Wahl*** – **und Wahlen erzeugen den Suchbaum.**

Zeile 5 – Zwei- gegen dreidimensionales Matching.

Paare zu bilden ist leicht (Edmonds 1965). ***Tripel zu bilden ist NP-vollständig*** *– es steht auf Karps Liste.*

*Anschaulich: Man kann n Personen paarweise zusammenstellen, indem man lokal verbessert (**augmentierende Pfade**).* **Bei Dreiergruppen gibt es keinen entsprechenden lokalen Verbesserungsschritt.**

Was man daraus lernt

Man sieht einem Problem seine Härte nicht an. Die Formulierungen sind fast identisch; die Komplexität ist es nicht.

Und die Übergänge folgen einem Muster:

Von 2 auf 3	Zwei Möglichkeiten erzwingen, drei erlauben Wahl
Von lokal auf global	Lokale Kriterien sind prüfbar, globale nicht
Von stetig auf ganzzahlig	Konvexität geht verloren
Von Min auf Max	Optimalitätsprinzip versagt

Diese vier Zeilen sind keine Sätze, sondern Faustregeln. **Aber sie sind erstaunlich zuverlässig** *– und sie sind das Beste, was man an Intuition hat, um ein neues Problem einzuschätzen.*

Die praktische Vorgehensweise bei einem neuen Problem:

1. In Garey und Johnson[9] nachschlagen – die Liste ist erstaunlich vollständig.
2. Nach einer Reduktion von einem der Standardprobleme suchen.
3. Prüfen, ob ein Sonderfall vorliegt, der leicht ist.
4. Erst dann nach einem exakten Algorithmus suchen.

Karps Aufsatz[15] ist erstaunlich knapp; Garey und Johnson[9] sind bis heute das Standardnachschlagewerk mit über 300 Problemen; Kleinberg und Tardos[17] zeigen die Reduktionen mit besonders guter Motivation.

Part III

Struktur

6 coNP und die Hierarchie

Die andere Seite der Antwort

Definition 6.1.

$$\text{coNP} := \{L : \overline{L} \in \text{NP}\}.$$

Ein Problem liegt in coNP***, wenn sich die* Nein*-Antwort kurz belegen lässt.*** Die Asymmetrie der NP-Definition (Kapitel 3) wird hier gespiegelt.

Problem	**Klasse**	**Zertifikat für**
SAT	**NP-vollständig**	Erfüllbarkeit
UNSAT	**coNP-vollständig**	Unerfüllbarkeit
TAUTOLOGIE	coNP-vollständig	
„n zusammengesetzt“	NP	ein Faktor
„n prim“	NP ∩ coNP	Pratt-Zertifikat

$\text{P} \subseteq \text{NP} \cap \text{coNP}$, *denn wer entscheiden kann, kann beide Antworten belegen* – **gar nicht.** ✓

Satz 6.2. $\text{NP} = \text{coNP}$ *genau dann, wenn* $\text{UNSAT} \in \text{NP}$.

Denn SAT *ist NP-vollständig, also ist* UNSAT *coNP-vollständig; und eine vollständige Sprache liegt genau dann in einer Klasse, wenn die ganze Klasse darin liegt.* ✓

coNP ist nicht das Komplement von NP

Das ist ein häufiger Fehler.

> coNP *ist die Klasse der* ***Komplemente der Sprachen aus*** NP **– nicht die Menge der Sprachen, die nicht in** NP **liegen.** *Der Schnitt* $\mathrm{NP} \cap \mathrm{coNP}$ *ist nicht leer; er enthält ganz* P.

Und die Frage $\mathrm{NP} = \mathrm{coNP}$ **ist *nicht* dieselbe wie** $\mathrm{P} = \mathrm{NP}$**.**

$\mathrm{P} = \mathrm{NP}$	$\Longrightarrow \mathrm{NP} = \mathrm{coNP}$ (denn P ist unter Komplement abgeschlossen)
$\mathrm{NP} = \mathrm{coNP}$	$\Longrightarrow$ **nichts über** $\mathrm{P} = \mathrm{NP}$

Es könnte also sein, dass $\mathrm{NP} = \mathrm{coNP}$ *und trotzdem* $\mathrm{P} \neq \mathrm{NP}$. **Beide Fragen sind offen, und beide werden verneint vermutet.**

Warum NP = coNP die Beweistheorie beträfe

> TAUTOLOGIE *ist coNP-vollständig. Wäre* $\mathrm{NP} = \mathrm{coNP}$*, so gäbe es für* ***jede*** *aussagenlogische Tautologie einen* ***polynomiell kurzen*** *Beweis.*
>
> *Man nennt das die Frage nach der Existenz eines* ***polynomiell beschränkten Beweissystems*** **– COOK und RECKHOW formulierten sie 1979.**

Beweissystem	Schranke	Jahr
Resolution	exponentiell	Haken 1985
Schnittebenen	exponentiell	Pudlák 1997
Frege-Systeme	offen	—
erweiterte Frege	offen	—

> ***Das Programm von Cook und Reckhow lautet:*** *Man beweise für immer stärkere Beweissysteme untere Schranken.* **Gelänge es für *alle*, so wäre** $\mathrm{NP} \neq \mathrm{coNP}$ **und damit** $\mathrm{P} \neq \mathrm{NP}$**.**

Nach vierzig Jahren steht man bei relativ schwachen Systemen. **Für Frege-Systeme – die gewöhnliche Aussagenlogik mit Axiomen und Modus ponens – kennt man keine einzige superpolynomielle untere Schranke.**

HAKENS *Ergebnis von 1985 betrifft das* ***Schubfachprinzip****: Die Aussage „$n+1$ Tauben passen nicht in n Fächer“ braucht in Resolution exponentiell lange Beweise* – **obwohl sie einem Menschen in einem Satz einleuchtet.**

6.1 Die Polynomialzeithierarchie

Wenn Quantoren sich abwechseln

NP *ist die Klasse der Probleme der Form „$\exists c$: $V(w,c)$“,* coNP *die der Form „$\forall c$: $V(w,c)$“.* **Was ist mit „$\exists c_1 \forall c_2$“?**

Definition 6.3.

$$\Sigma_0^{\mathrm{P}} = \Pi_0^{\mathrm{P}} := \mathrm{P},$$

$$\Sigma_k^{\mathrm{P}} := \{L : w \in L \iff \exists c_1 \forall c_2 \ldots Q c_k \; V(w, c_1, \ldots, c_k)\},$$

mit k alternierenden Quantoren über polynomiell lange Wörter und $V \in \mathrm{P}$. Π_k^{P} beginnt mit $\forall$.

$$\mathrm{PH} := \bigcup_{k \geq 0} \Sigma_k^{\mathrm{P}}.$$

Stufe	Klasse	Beispiel
$\Sigma_0 = \Pi_0$	P	Erreichbarkeit
Σ_1	NP	SAT
Π_1	coNP	TAUTOLOGIE
Σ_2		**Minimale Formel**
Π_2		

Ein natürliches Σ_2-Problem.

„Gibt es eine Formel ψ der Länge $\leq k$, die zu φ ***äquivalent*** *ist?“*

$$\exists \psi \; \forall \text{Belegung } b : \; \psi(b) = \varphi(b).$$

Der erste Quantor sucht die kurze Formel, der zweite prüft die Äquivalenz – **und beide sind unvermeidlich.** *Das Problem ist Σ_2-vollständig.*

Satz 6.4 (Kollaps). *Gilt $\Sigma_k^{\mathrm{P}} = \Pi_k^{\mathrm{P}}$ für ein k, so ist*

$$\mathrm{PH} = \Sigma_k^{\mathrm{P}}$$

– die Hierarchie ***kollabiert*** *auf Stufe k.*

> ***Insbesondere:*** $\mathrm{P} = \mathrm{NP}$ ***würde die ganze Hierarchie auf*** P ***zusammenfallen lassen.*** Und $\mathrm{NP} = \mathrm{coNP}$ ließe sie auf NP kollabieren.
>
> **Die Beweisidee.** *Ist* $\Sigma_k = \Pi_k$*, so lässt sich ein führender Quantor der Stufe* $k+1$ *in den Rumpf hineinziehen und mit dem nächsten verschmelzen* – **die Alternation bricht ab.** ✓
>
> *Man vermutet, dass die Hierarchie* ***echt unendlich*** *ist – also alle Stufen verschieden sind.*
> **Bewiesen ist davon keine einzige Trennung.**

Aufgabe 6.1.

(a) Zeigen Sie, dass TAUTOLOGIE coNP-vollständig ist, und untersuchen Sie Faktorisieren.
(b) Beweisen Sie den Kollapssatz für $k = 1$ vollständig.
(c) Ordnen Sie konkrete Probleme in die Hierarchie ein.

Lösung Schritt für Schritt.

(a) coNP-Vollständigkeit und $\mathrm{NP} \cap \mathrm{coNP}$**.**
Teil 1 – TAUTOLOGIE ist coNP-vollständig.
Eine Formel ist eine ***Tautologie****, wenn jede Belegung sie erfüllt.*

$$\varphi \text{ ist Tautologie} \iff \neg\varphi \text{ ist unerfüllbar.}$$

Also ist $\varphi \mapsto \neg\varphi$ *eine Reduktion zwischen* TAUTOLOGIE *und* UNSAT*, in beiden Richtungen.* ✓

Und UNSAT *ist coNP-vollständig, weil sein Komplement* SAT *NP-vollständig ist.* ✓

> ***Merkregel:*** *Ist* L *NP-vollständig, so ist* $\overline{L}$ *coNP-vollständig* – **denn Reduktionen übertragen sich auf Komplemente:** $A \leq_{\mathrm{p}} B \iff \overline{A} \leq_{\mathrm{p}} \overline{B}$ *mit derselben Funktion* f*.* ✓

Teil 2 – Faktorisieren liegt in $\mathrm{NP} \cap \mathrm{coNP}$**.**
Als Entscheidungsproblem: „Hat n *einen Faktor* $\leq k$*?“*
In NP***:*** Zertifikat ist der Faktor $d \leq k$; Prüfung: eine Division. ✓
In coNP***:*** Zertifikat für die Nein-Antwort ist die **vollständige Primfaktorzerlegung** von n zusammen mit Pratt-Zertifikaten für jeden Faktor.

> *Der Prüfer verifiziert: (i) das Produkt ergibt* n*, (ii) jeder Faktor ist prim (Pratt), (iii) alle Faktoren sind* $> k$*.* **Damit ist bewiesen, dass es keinen Faktor** $\leq k$ **gibt.** ✓

Warum das ein starkes Indiz ist

Satz 6.5. *Ist ein NP-vollständiges Problem in* coNP*, so gilt* NP = coNP*.*

Beweis: Sei B NP-vollständig und $B \in$ coNP. Für jedes $A \in$ NP gilt $A \leq_p B$, also $A \in$ coNP – damit NP $\subseteq$ coNP*. Die andere Inklusion folgt durch Komplementbildung.* ✓

> ***Wäre Faktorisieren NP-vollständig, so wäre*** NP = coNP – und die Polynomialzeithierarchie würde kollabieren. **Das hält fast niemand für wahrscheinlich.**

*Faktorisieren ist also vermutlich **weder** in* P ***noch** NP-vollständig* – **ein Kandidat für die Zwischenschicht, deren Existenz Ladners Satz garantiert** (Kapitel 9).

Dieselbe Lage gilt für die Graphisomorphie. **Beide sind die berühmtesten Kandidaten für „NP-intermediär".**

(b) Der Kollapssatz für $k = 1$.

Satz 6.6. *Ist* NP = coNP*, so gilt* PH = NP*.*

Schritt 1 – Was zu zeigen ist.
Es genügt, $\Sigma_2 \subseteq$ NP zu zeigen; der Rest folgt per Induktion.
Schritt 2 – Die Ausgangslage.
Sei $L \in \Sigma_2$, also

$$w \in L \iff \exists c_1\, \forall c_2 :\ V(w, c_1, c_2)$$

mit $V \in$ P und $|c_1|, |c_2| \leq p(|w|)$.
Schritt 3 – Den inneren Teil benennen.
Setze

$$M := \{(w, c_1) : \forall c_2\ V(w, c_1, c_2)\}.$$

Nach Definition ist $M \in$ coNP – **es ist eine $\forall$-Aussage mit polynomiellem Rumpf.** ✓

Schritt 4 – Die Voraussetzung anwenden.
Nach Annahme ist coNP = NP*, also $M \in$ NP.* **Es gibt somit ein $V' \in$ P und ein Polynom q mit**

$$(w, c_1) \in M \iff \exists c_3,\ |c_3| \leq q,\ V'(w, c_1, c_3).$$

Schritt 5 – Einsetzen und verschmelzen.

$$w \in L \iff \exists c_1\, \exists c_3 :\ V'(w, c_1, c_3).$$

Zwei aufeinanderfolgende $\exists$-Quantoren lassen sich zu einem zusammenfassen – man nimmt das Paar (c_1, c_3) als *ein* Zertifikat.

$$w \in L \iff \exists c : \; V''(w, c) \quad \text{mit } c = (c_1, c_3),\ |c| \le p + q.$$
$$\Longrightarrow L \in \text{NP}. \qquad \checkmark$$

Schritt 6 – Die Induktion.
Genauso zeigt man $\Sigma_{k+1} \subseteq \Sigma_k$ für jedes k, sobald $\Sigma_k = \Pi_k$ gilt. **Also fällt die ganze Hierarchie auf Stufe k zusammen.**

Der ganze Beweis besteht darin, einen Quantor umzudrehen und dann zwei gleichartige zu verschmelzen. Er ist zehn Zeilen lang und hat weitreichende Folgen.

Die praktische Verwendung.

*Der Satz wird meist **kontrapositiv** benutzt: Wer zeigt, dass eine Aussage A den Kollaps der Hierarchie erzwingt, hat gezeigt, dass A vermutlich falsch ist.*

*Man nennt das ein **Argument aus dem Nichtkollaps**.* **Beispiel: Gäbe es für SAT Schaltkreise polynomieller Größe, so kollabierte PH auf Σ_2** (KARP und LIPTON, 1980) – *was man für unwahrscheinlich hält.*

(c) Probleme einordnen.

Problem	Klasse	Quantorenform
Erreichbarkeit im Graphen	P	keine
SAT	Σ_1	$\exists$ Belegung
TAUTOLOGIE	Π_1	$\forall$ Belegung
Minimale äquiv. Formel	Σ_2	$\exists \psi\, \forall b$
Kleinster Schaltkreis	Σ_2	$\exists C\, \forall x$
QBF	PSPACE	**beliebig viele**
Gewinnstrategie im Spiel	PSPACE	$\exists\forall\exists\forall\ldots$

Der Unterschied zwischen PH und PSPACE.

*In der Hierarchie ist die Zahl der Quantoren eine **Konstante** – sie hängt nicht von der Eingabe ab.* **Bei QBF darf sie mit n wachsen** (Kapitel 7).

$$\text{PH} \subseteq \text{PSPACE},$$

und ob die Inklusion echt ist, ist offen. **Wäre PH = PSPACE, so hätte PH ein vollständiges Problem** – *und daraus folgte der Kollaps.*

Warum PH kein vollständiges Problem hat (vermutlich)

Ein hübsches Argument.
Angenommen, L wäre PH*-vollständig. Dann läge L in irgendeiner Stufe* Σ_k *– denn* PH *ist die* ***Vereinigung*** *der Stufen, und L ist ein einzelnes Problem.*

> *Da sich alles in* PH *auf L reduzieren lässt, läge* ***ganz*** PH *in* Σ_k **– also wäre die Hierarchie kollabiert.** ✓

Wenn also die Hierarchie echt unendlich ist, hat sie kein vollständiges Problem. **Und das unterscheidet sie von** P**,** NP**,** PSPACE **und** EXP**, die alle eines haben.**

Klassen ohne vollständiges Problem sind schwer zu handhaben – **man kann Aussagen über sie nicht auf ein einzelnes Problem zurückführen.** *Dasselbe gilt für* BPP (Kapitel 11).

Arora und Barak[2] und Papadimitriou[21] behandeln die Hierarchie ausführlich; Goldreich[10] betont die konzeptuelle Seite; Wigderson[27] ordnet sie in die Mathematik ein.

7 Platz als Ressource

Warum Platz sich anders verhält als Zeit

Platz ist wiederverwendbar, Zeit nicht. Aus dieser einen Asymmetrie folgt fast alles, was die Platzkomplexität von der Zeitkomplexität unterscheidet.

Definition 7.1.

$$\mathrm{PSPACE} := \bigcup_{k \geq 1} \mathrm{SPACE}(n^k), \qquad \mathrm{NPSPACE} := \bigcup_{k \geq 1} \mathrm{NSPACE}(n^k).$$

Frage	Zeit	Platz
Determinismus gegen Nichtdet.	**offen**	**gelöst 1970**
Abschluss unter Komplement	offen	**gelöst 1987**

Beide großen Fragen, die bei der Zeit offen sind, sind beim Platz beantwortet. Das ist der überraschendste Befund dieses Kapitels – und ein Hinweis darauf, dass Zeit die schwierigere Ressource ist.

Der Satz von Savitch

Satz 7.2 (Savitch, 1970)**.** *Für platzkonstruierbares $f(n) \geq \log n$ gilt*

$$\mathrm{NSPACE}\big(f(n)\big) \subseteq \mathrm{SPACE}\big(f(n)^2\big).$$

Nichtdeterminismus kostet beim Platz nur eine Quadrierung – gegenüber der exponentiellen Kluft, die man bei der Zeit vermutet. **Insbesondere:**

$$\boxed{\mathrm{PSPACE} = \mathrm{NPSPACE}}$$

denn das Quadrat eines Polynoms ist ein Polynom. ✓
Der Beweis – das Erreichbarkeitsproblem, rekursiv halbiert.

Man betrachtet den ***Konfigurationsgraphen*** *der nichtdeterministischen Maschine.* **Akzeptieren heißt: Von der Startkonfiguration gibt es einen Weg zur akzeptierenden.**

Der Graph hat $2^{\mathcal{O}(f(n))}$ *Knoten* (Kapitel 1) – **man kann ihn nicht hinschreiben.** *Aber man kann in ihm suchen, ohne ihn zu speichern.*

Definition 7.3. `ERREICHBAR(u, v, k)`: *Gibt es einen Weg von u nach v der Länge* $\leq 2^k$*?*

```
ERREICHBAR(u, v, k):
  if k = 0 then return (u = v oder u ->
v ist ein Schritt);
  für jede Konfiguration m do
    if ERREICHBAR(u, m, k-1) und
ERREICHBAR(m, v, k-1)
      then return wahr;
  return falsch
```

Der Kunstgriff: Man rät die* Mitte *des Weges statt der ganzen Folge. Ein Weg der Länge 2^k zerfällt in zwei Wege der Länge 2^{k-1} mit einem Mittelpunkt dazwischen.

Die Platzanalyse.

Größe	**Wert**	**Grund**
Rekursionstiefe	$\mathcal{O}(f(n))$	$2^{\mathcal{O}(f)}$ Konfigurationen
Platz je Ebene	$\mathcal{O}(f(n))$	eine Konfiguration m
gesamt	$\mathcal{O}(\mathbf{f(n)^2})$	

Die Zeit ist katastrophal – die Schleife über alle m läuft exponentiell oft, auf jeder Ebene. **Aber nach der Zeit fragt der Satz nicht.**

Warum derselbe Trick bei der Zeit versagt

Man könnte versuchen, dasselbe für die Zeit zu tun. *Es scheitert sofort.*

> *Der Halbierungstrick spart Platz, weil man den Speicher der ersten Hälfte für die zweite* ***wiederverwendet****.* **Zeit lässt sich nicht wiederverwenden** *– die beiden Hälften kosten zusammen die Summe, nicht das Maximum.*

Die Rekursion verdoppelt die Zeit auf jeder Ebene, bei f Ebenen also 2^f – **genau der Aufwand, den man vermeiden wollte.**

Das ist im Kern der Grund, warum P *gegen* NP *offen ist und* PSPACE $=$ NPSPACE *nicht.*

QBF und die Spiele

Definition 7.4. QBF (**quantifizierte Boolesche Formel**): Ist eine Formel der Gestalt

$$\exists x_1 \forall x_2 \exists x_3 \dots Q x_n \; \varphi(x_1, \dots, x_n)$$

wahr?

Satz 7.5. QBF *ist* PSPACE*-vollständig.*

> **SAT *ist der Sonderfall mit nur* $\exists$*-Quantoren.*** Die Alternation macht aus einem NP-vollständigen ein PSPACE-vollständiges Problem.

Warum QBF in PSPACE **liegt.**
Man wertet rekursiv aus:

$\exists x_i \dots$	Probiere $x_i = 0$; falls falsch, probiere $x_i = 1$
$\forall x_i \dots$	Probiere $x_i = 0$; falls wahr, probiere $x_i = 1$

Die Rekursionstiefe ist n, jede Ebene braucht $\mathcal{O}(1)$ Platz – **insgesamt $\mathcal{O}(n)$.** ✓

Die Spielinterpretation.

> *$\exists$ ist ein Zug von Spieler A, $\forall$ ein Zug von Spieler B.* **„Die Formel ist wahr" heißt: A hat eine Gewinnstrategie.**

Spiel	Komplexität	Grund
GEOGRAPHY	PSPACE-vollst.	
Hex ($n \times n$)	PSPACE-vollst.	
Reversi, Amazons	PSPACE-vollst.	polynomielle Partien
Schach ($n \times n$)	EXP-vollständig	Partien exponentiell lang
Go (japan. Regeln)	EXP-vollständig	

Der Unterschied liegt an der Partielänge. Ist eine Partie polynomiell lang, so ist das Spiel PSPACE-vollständig; darf sie exponentiell lang werden, wird es EXP-vollständig.

Und EXP*-Vollständigkeit ist* ***keine*** *Vermutung: Aus dem Zeithierarchiesatz folgt* $\mathrm{P} \subsetneq \mathrm{EXP}$ (Kapitel 9), **also ist verallgemeinertes Schach *beweisbar* nicht in polynomieller Zeit lösbar.**

Das ist eine der ganz wenigen bewiesenen Härteaussagen für ein natürliches Problem.

Aufgabe 7.1.

(a) Beweisen Sie $\mathrm{NP} \subseteq \mathrm{PSPACE}$ und die PSPACE-Vollständigkeit von QBF im Überblick.

(b) Untersuchen Sie den Satz von Immerman und Szelepcsényi und seine Bedeutung.

(c) Rechnen Sie ein QBF-Beispiel als Spiel durch.

Lösung Schritt für Schritt.

(a) Die Einordnung von QBF.

Teil 1 – $\mathrm{NP} \subseteq \mathrm{PSPACE}$**.** *Bereits in Kapitel 3 gezeigt: alle Zertifikate durchprobieren, Speicher wiederverwenden.* ✓

Teil 2 – QBF ist PSPACE**-schwer.**

Sei $L \in \mathrm{PSPACE}$ *mit Platzschranke* $p(n)$*. Man baut wie bei Savitch eine Formel*

$$\Phi_k(C_1, C_2) := \text{„von “ } C_1 \text{ nach } C_2 \text{ in } \leq 2^k \text{ Schritten.}$$

Der naive Ansatz und sein Scheitern.

$$\Phi_k(C_1, C_2) := \exists M \, \big[\Phi_{k-1}(C_1, M) \wedge \Phi_{k-1}(M, C_2)\big].$$

*Das ist korrekt – aber die Formel **verdoppelt** sich auf jeder Ebene.* **Bei $k = \mathcal{O}(p(n))$ Ebenen wird sie exponentiell lang.** *Die Reduktion wäre nicht polynomiell.*

Der Kunstgriff – der $\forall$-Quantor als Kopiervermeider.

$$\Phi_k(C_1, C_2) := \exists M \, \forall (A, B) \in \{(C_1, M), (M, C_2)\} : \, \Phi_{k-1}(A, B).$$

*Statt Φ_{k-1} zweimal hinzuschreiben, quantifiziert man über die beiden Paare und schreibt es **einmal**.* **Die Formel wächst nur um $\mathcal{O}(p(n))$ je Ebene, insgesamt also $\mathcal{O}(p(n)^2)$.** ✓

Die Auswahl „(A, B) ist eines der beiden Paare“ lässt sich mit einer zusätzlichen Booleschen Variablen und einer kurzen Formel ausdrücken. ✓

Das ist der Beweis von* STOCKMEYER *und* MEYER *(1973) – und der $\forall$-Quantor dient hier nicht der Logik, sondern der **Platzersparnis in der Formel.** *Ein hübscher technischer Kniff.*

(b) Immerman und Szelepcsényi.

Satz 7.6 (1987, unabhängig). *Für $f(n) \geq \log n$ gilt*

$$\mathrm{NSPACE}(f) = \mathrm{coNSPACE}(f).$$

Insbesondere $\mathrm{NL} = \mathrm{coNL}$.

Nichtdeterministische Platzklassen sind unter Komplement abgeschlossen – was bei den Zeitklassen offen ist (NP gegen coNP) und bei den aufzählbaren Sprachen sogar **falsch** (Band 2, Kapitel 10).

Das Problem, das zu lösen war.
*Um „es gibt **keinen** Weg von s nach t“ nichtdeterministisch zu zeigen, müsste man alle Wege ausschließen* – **scheinbar unmöglich mit Raten.**
Die Lösung – induktives Zählen.

*Sei N_i die **Anzahl** der von s aus in höchstens i Schritten erreichbaren Ecken.* **Kennt man N_i, so lässt sich nichtdeterministisch prüfen, ob eine gegebene Ecke v *nicht* erreichbar ist:**

1. Man durchläuft alle Ecken u und rät für jede, ob sie erreichbar ist.
2. Behauptet man „erreichbar“, so rät man den Weg und verifiziert ihn.
3. Man zählt die erfolgreich verifizierten mit.
4. Erreicht der Zähler genau N_i, so waren alle Behauptungen **richtig** – denn man kann nicht mehr als N_i verifizieren.
5. Kam v dabei nicht vor, so ist v **nicht** erreichbar. ✓

Der Kern ist Schritt 4: *Die Zahl N_i wirkt als* ***Beglaubigung****. Wer N_i Ecken tatsächlich verifiziert hat, hat sie alle gefunden –* **und darf über die übrigen die Nein-Aussage treffen.**

Und N_i selbst berechnet man induktiv aus N_{i-1} mit demselben Verfahren. **Der Platz ist $\mathcal{O}(\log n)$: nur Zähler und aktuelle Ecke.** ✓

Neil Immerman *und* Róbert Szelepcsényi *fanden das 1987 unabhängig voneinander* – Szelepcsényi war damals Student in Bratislava. *Beide erhielten 1995 gemeinsam den Gödel-Preis.*

Ein 25 Jahre altes Problem, gelöst mit einer Zählidee

Die Frage war seit 1964 offen – sie hieß das zweite LBA-Problem.

Sie fragte, ob die kontextsensitiven Sprachen (Band 2, Kapitel 4) unter Komplement abgeschlossen sind. **Diese entsprechen genau** $\mathrm{NSPACE}(n)$ *– also folgt die Antwort „ja“ unmittelbar aus dem Satz.*

Das *erste* LBA-Problem – ob $\mathrm{SPACE}(n) = \mathrm{NSPACE}(n)$ – ist bis heute offen.

Savitch liefert nur $\mathrm{NSPACE}(n) \subseteq \mathrm{SPACE}(n^2)$ **– und die Quadrierung lässt sich innerhalb einer festen Platzschranke nicht wegdiskutieren.**

(c) Ein QBF-Beispiel als Spiel.
Gegeben

$$\Psi = \exists x\, \forall y\, \exists z : \ (x \vee y \vee z) \wedge (\overline{x} \vee \overline{y} \vee z) \wedge (\overline{x} \vee y \vee \overline{z}).$$

Die Spielregeln.

Spieler A ($\exists$)	wählt x, später z; **will**, dass φ wahr wird
Spieler B ($\forall$)	wählt y; **will**, dass φ falsch wird

Die Auswertung, Zug für Zug.
Versuch $x = 0$**.**

x	y	z	K_1	K_2	K_3
0	0	0	0	1	1
0	0	1	1	1	1 ✓
0	1	0	1	1	1 ✓
0	1	1	1	1	1 ✓

Bei $x = 0$ *und* $y = 0$ *kann A mit* $z = 1$ *gewinnen* ✓*; bei* $x = 0$ *und* $y = 1$ *ebenfalls (sogar mit beiden* z*).* **Also gewinnt A mit** $x = 0$ **gegen jedes** y**.**

Ψ ist **wahr**

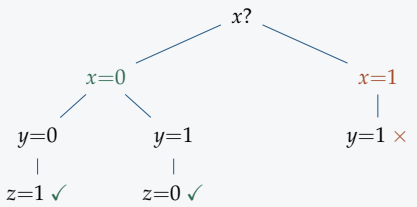
Zur Kontrolle: Versuch $x = 1$**.**

x	y	z	K_1	K_2	K_3
1	1	0	1	0	1
1	1	1	1	1	0

Bei $x = 1$ *kann B mit* $y = 1$ *antworten, und dann verliert A für* ***beide*** *Werte von* z*.* $x = 1$ **wäre also ein Fehler gewesen.** ✓

Genau das ist die Bedeutung der Quantorenalternation: A muss x wählen, **bevor** er y kennt – er muss also einen Zug finden, der gegen *jede* Antwort hält.

Der Suchbaum und der Platz.



Der Baum hat 2^n *Blätter –* ***exponentielle Zeit****.* **Aber man durchläuft ihn in Tiefensuche und merkt sich nur den aktuellen Pfad –** $\mathcal{O}(n)$ *Platz.* ✓

Das ist die Signatur von PSPACE***:*** *exponentielle Zeit, polynomieller Platz* – **und genau die Bauform jedes Spielbaumsuchers, von Schachprogrammen bis zu Alpha-Beta.**

Savitchs Arbeit[23] ist kurz und elegant; Sipser[24] führt die Spielinterpretation am anschaulichsten aus; Arora und Barak[2] beweisen Immerman-Szelepcsényi vollständig; Papadimitriou[21] verbindet es mit den Sprachklassen.

8 Rechnen fast ohne Speicher

Was mit $\log n$ Bits geht

Definition 8.1.

$$\mathrm{L} := \mathrm{SPACE}(\log n), \qquad \mathrm{NL} := \mathrm{NSPACE}(\log n).$$

Gezählt wird nur das Arbeitsband; die Eingabe darf gelesen, aber nicht beschrieben werden.

> *$\mathcal{O}(\log n)$ Bits reichen für **eine Handvoll Zeiger** in die Eingabe und ein paar Zähler* – **mehr nicht.** *Man kann sich nicht einmal die Eingabe merken.*

Problem	**Klasse**	**Bemerkung**
Palindrom erkennen	L	zwei Zeiger
Addition, Multiplikation	L	stellenweise
Zusammenhang, ungerichtet	L	**seit 2004**
Erreichbarkeit, gerichtet	NL-**vollständig**	
Bipartitheit	L	
2-SAT	NL-vollständig	

Die Kette: $\mathrm{L} \subseteq \mathrm{NL} \subseteq \mathrm{P}$ – **die zweite Inklusion aus dem Konfigurationsargument** (Kapitel 1).

Warum die Eingabe nicht mitzählt

Zählte man sie mit, wäre jede Klasse unterhalb von $\mathcal{O}(n)$ leer – *die Eingabe belegt ja schon n Felder.*

> *Das Modell mit* ***Nur-Lese-Eingabeband*** *ist keine Spitzfindigkeit, sondern beschreibt eine reale Situation:* **Ein Programm, das eine riesige Datei verarbeitet, ohne sie in den Speicher zu laden.**

Genau das ist das Modell der ***Datenströme*** – **eine der aktivsten Richtungen der praktischen Algorithmik** (Band 1, Kapitel 15).

Logspace-Reduktionen

> *Für Klassen* ***innerhalb*** *von* P *ist die polynomielle Reduktion nutzlos:* **Jedes nichttriviale Problem in** P **lässt sich auf jedes andere polynomiell reduzieren** – *man löst es einfach und gibt eine feste Ja- oder Nein-Instanz aus.*

Definition 8.2. $A \leq_{\mathrm{L}} B$, wenn die Übersetzungsfunktion in $\mathcal{O}(\log n)$ Platz berechenbar ist.

Man braucht ein ***feineres*** *Werkzeug, sobald man feinere Unterschiede messen will.* **Dieses Muster wiederholt sich in jedem Teil der Komplexitätstheorie.**

Satz 8.3. *Erreichbarkeit in gerichteten Graphen (*REACH*) ist* NL*-vollständig unter Logspace-Reduktionen.*

Warum REACH in NL liegt.
Man rät den Weg Ecke für Ecke und merkt sich stets nur die ***aktuelle*** *Ecke und einen Schrittzähler:*

$$\mathcal{O}(\log n) + \mathcal{O}(\log n) = \mathcal{O}(\log n). \qquad \checkmark$$

> ***Der Zähler ist nötig, um Endlosschleifen zu verhindern*** – nach n Schritten bricht man ab, denn ein einfacher Weg hat höchstens $n - 1$ Kanten.

Warum es NL-schwer ist.
Der Konfigurationsgraph einer NL*-Maschine hat polynomiell viele Knoten und ist* ***in Logspace konstruierbar*** – **man muss nur für jedes Konfigurationspaar prüfen, ob ein Schritt sie verbindet, und das ist eine lokale Prüfung.** ✓

> *Akzeptieren heißt dann: Die akzeptierende Konfiguration ist erreichbar.*

Der Zufall verschwindet aus dem Zusammenhang

Die Frage, ob ungerichtete Erreichbarkeit in L **liegt, war seit den 1970er Jahren offen.**

> *Die naheliegenden Verfahren – Tiefensuche, Breitensuche – brauchen einen Stapel oder eine Warteschlange und damit* ***linearen*** *Platz.* **Man darf sich nicht merken, wo man schon war.**

1979 fanden Aleliunas, Karp, Lipton, Lovász und Rackoff eine überraschende Teillösung:

> *Man läuft* ***zufällig*** *durch den Graphen.* **Ein solcher Irrfahrt besucht in einem zusammenhängenden Graphen mit** n **Ecken und** m **Kanten nach erwartet** $\mathcal{O}(nm)$ **Schritten** ***jede*** **Ecke.**

Man merkt sich nur die aktuelle Ecke und einen Zähler – $\mathcal{O}(\log n)$ **Platz, aber mit Zufall.** *Das ergab die Zugehörigkeit zur Klasse* RL.

25 Jahre lang blieb offen, ob der Zufall entbehrlich ist.

2004 zeigte Omer Reingold, dass er es ist.

> *Sein Verfahren verwandelt den Graphen schrittweise in einen* ***Expander*** *–* **einen Graphen mit sehr gutem Zusammenhang, in dem alle Wege kurz sind** *– und zwar mit dem* ***Zickzackprodukt****, das* Reingold, Vadhan *und* Wigderson *2002 eingeführt hatten.*

In einem Expander mit logarithmischem Durchmesser genügt es, alle Wege einer festen kurzen Länge durchzuprobieren – **und dafür reicht ein Zähler.**

> *Reingold erhielt dafür 2005 den Gödel-Preis.* **Das Ergebnis ist eines der wenigen Beispiele einer erfolgreichen Derandomisierung** *– ein Verfahren, das Zufall brauchte, kommt beweisbar ohne aus* (Kapitel 11).

Für ***gerichtete*** **Graphen ist die Frage weiterhin offen –** L *gegen* NL. **Und die Asymmetrie ist einleuchtend: In einem ungerichteten Graphen kann man umkehren, in einem gerichteten nicht.**

Aufgabe 8.1.

(a) Geben Sie Logspace-Algorithmen für konkrete Probleme an.
(b) Zeigen Sie NL $\subseteq$ P und die Folgerungen aus Savitch für NL.
(c) Diskutieren Sie das Verhältnis der kleinen Klassen zueinander.

Lösung Schritt für Schritt.

(a) Logspace-Algorithmen.
Problem 1 – Palindrom.

```
i := 1;  j := n;
while i < j do {
  if Eingabe[i] <> Eingabe[j] then
verwirf;
  i := i+1;  j := j-1 }
akzeptiere
```

Zwei Zähler bis $n \Longrightarrow 2\log_2 n$ **Bits.** ✓

Problem 2 – Ist die Eingabe die Binärdarstellung einer geraden Zahl? *Man springt zur letzten Stelle und liest sie.* **Ein Zähler.** ✓
Problem 3 – Multiplikation zweier Zahlen.

> *Man berechnet Bit für Bit. Für das k-te Bit des Produkts braucht man die Summe der Teilprodukte an dieser Stelle plus die Überträge.*

Der Trick: Man berechnet die Überträge ***neu****, statt sie zu speichern.* **Das kostet Zeit und spart Platz** *– die klassische Abwägung dieses Kapitels.* ✓
Problem 4 – Bipartitheit eines Graphen.

> *Ein Graph ist genau dann bipartit, wenn er keinen Kreis* ***ungerader*** *Länge enthält.* **Und Kreiserkennung lässt sich auf ungerichtete Erreichbarkeit zurückführen** *– nach Reingolds Satz also in* L. ✓

Vor 2004 wusste man nur, dass es in RL *liegt.* **Ein Beispiel dafür, wie ein einziger Satz eine Reihe von Problemen mit verschiebt.**

(b) NL $\subseteq$ P **und Savitch für** NL.
Teil 1 – NL $\subseteq$ P**.**
Sei M eine NL*-Maschine. Die Zahl der Konfigurationen ist*

$$|Q| \cdot (n+1) \cdot \mathcal{O}(\log n) \cdot |\Gamma|^{c \log n} = \mathcal{O}(n^{c'}).$$

> ***Polynomiell viele Konfigurationen*** – also kann man den Konfigurationsgraphen **explizit** aufbauen (polynomiell groß) und darin eine Breitensuche machen ($\mathcal{O}(n^{2c'})$). ✓

Der entscheidende Schritt ist $|\Gamma|^{c \log n} = 2^{c \log n \cdot \log|\Gamma|} = n^{c \log|\Gamma|}$ **– die Exponentialfunktion von einem Logarithmus ist ein Polynom.** ✓

Teil 2 – Savitch angewandt.
Mit $f(n) = \log n$ liefert Satz 7.2:

$$\mathrm{NL} = \mathrm{NSPACE}(\log n) \subseteq \mathrm{SPACE}(\log^2 n).$$

Man nennt $\mathrm{SPACE}(\log^2 n)$ *auch* L^2. **Ob** $\mathrm{NL} = \mathrm{L}$ **gilt, ist offen** – *Savitch bringt einen nur bis* $\log^2$.

Konkret für REACH.

Verfahren	**Platz**	**Zeit**
Breitensuche	$\mathcal{O}(n)$	$\mathcal{O}(n+m)$
Nichtdet. Raten	$\mathcal{O}(\log n)$	— (nichtdet.)
Savitch	$\mathcal{O}(\log^2 n)$	$n^{\mathcal{O}(\log n)}$

Savitchs Verfahren ist deterministisch, sparsam und praktisch unbrauchbar – die Laufzeit $n^{\mathcal{O}(\log n)}$ ist **quasipolynomiell**, also weder polynomiell noch exponentiell.

(c) Das Verhältnis der kleinen Klassen.

Inklusion	**Status**	**Bemerkung**
$\mathrm{L} \subseteq \mathrm{NL}$	offen	
$\mathrm{NL} \subseteq \mathrm{P}$	offen	
$\mathrm{P} \subseteq \mathrm{NP}$	offen	
$\mathrm{NP} \subseteq \mathrm{PSPACE}$	offen	
$\mathrm{NL} \subsetneq \mathrm{PSPACE}$	**bewiesen**	Platzhierarchiesatz
$\mathrm{L} \subsetneq \mathrm{PSPACE}$	**bewiesen**	ebenso
$\mathrm{P} \subsetneq \mathrm{EXP}$	**bewiesen**	Zeithierarchiesatz

Das Muster ist immer dasselbe: *Man kann Klassen trennen, die* ***weit*** *auseinanderliegen, und keine, die* ***benachbart*** *sind.*

Ein bemerkenswerter Nebenbefund.
Aus $\mathrm{L} \subsetneq \mathrm{PSPACE}$ *folgt:* ***Mindestens eine*** *der vier offenen Inklusionen oben ist echt.* **Man weiß nicht, welche.**

Was aus $\mathrm{L} = \mathrm{P}$ folgen würde

Es gibt eine überraschende Konsequenz.

Wäre $\mathrm{L} = \mathrm{P}$*, so ließe sich jedes polynomiell lösbare Problem mit logarithmischem Speicher lösen* – **und damit auch massiv parallelisieren.**

Der Zusammenhang: $\mathrm{L} \subseteq \mathrm{NC}^2$*, wobei* NC *die Klasse der* ***gut parallelisierbaren*** *Probleme ist* (Schaltkreise polylogarithmischer Tiefe).

NC	polylogarithmische Zeit auf polynomiell vielen Prozessoren
P-vollständig	**vermutlich nicht** gut parallelisierbar

Es gibt P*-vollständige Probleme – etwa die Auswertung eines Schaltkreises oder das lineare Programmieren.* **Für sie gilt: Sind sie in** NC**, so ist** $\mathrm{P} = \mathrm{NC}$ – *und alles Berechenbare wäre parallelisierbar.*

Das hält niemand für wahrscheinlich. **Manche Probleme scheinen *inhärent sequentiell* zu sein** – *jeder Schritt hängt vom vorigen ab.*

Die Frage P *gegen* NC *ist damit das Parallelrechen-Gegenstück zu* P *gegen* NP – **ebenso offen und ebenso praktisch bedeutsam.**

Arora und Barak[2] behandeln die kleinen Klassen und NC gründlich; Papadimitriou[21] entwickelt die Logspace-Reduktionen; Reingolds Ergebnis von 2004 ist in Wigderson[27] allgemeinverständlich dargestellt.

9 Hierarchiesätze und die Mitte

Mehr Zeit bringt mehr

Satz 9.1 (Zeithierarchiesatz; Hartmanis und Stearns, 1965)**.** *Für zeitkonstruierbare Funktionen mit $f(n)\log f(n) = o\big(g(n)\big)$ gilt*

$$\mathrm{DTIME}(f) \subsetneq \mathrm{DTIME}(g).$$

Mit mehr Zeit kann man echt mehr entscheiden. Das klingt selbstverständlich und ist der einzige harte Trennungssatz, den die Komplexitätstheorie besitzt.

Der Beweis – Diagonalisierung.
*Man baut eine Maschine D, die in $g(n)$ Zeit läuft und sich von **jeder** f-Zeit-Maschine unterscheidet:*

```
D bei Eingabe w:
  deute w als Kodierung einer Maschine M;
  simuliere M auf w für g(|w|) Schritte;
  falls M akzeptiert:  verwirf;
  falls M verwirft oder die Zeit abläuft:  akzeptiere
```

Sei M eine Maschine mit Laufzeit $\mathcal{O}(f(n))$. Auf der Eingabe $\langle M\rangle$ (für hinreichend großes $\langle M\rangle$) läuft die Simulation vollständig durch – **und** D **tut das Gegenteil von** M**.** *Also ist $L(D) \neq L(M)$.* ✓

Der Faktor $\log f(n)$ ist der Preis der Simulation – **eine universelle Maschine braucht einen logarithmischen Zusatzaufwand, um das Band der simulierten Maschine zu verwalten.**

Folgerung 1	$\mathrm{P} \subsetneq \mathrm{EXP}$
Folgerung 2	$\mathrm{DTIME}(n^2) \subsetneq \mathrm{DTIME}(n^3)$
Folgerung 3	$\mathrm{PSPACE} \subsetneq \mathrm{EXPSPACE}$ (Platzfassung)

Warum dieselbe Methode bei P gegen NP versagt

Die Diagonalisierung braucht zwei Zutaten:

1. Eine Aufzählung aller Maschinen der kleineren Klasse.
2. Die Fähigkeit, eine solche Maschine **effizient zu simulieren und ihr Ergebnis umzukehren**.

Punkt 2 scheitert bei NP*: Man kann eine* nichtdeterministische *Maschine nicht in polynomieller Zeit simulieren* – **und ihr Ergebnis umzukehren würde** NP **verlassen und in** coNP **führen.**

Kapitel 14 zeigt, dass das kein technisches Problem ist: **Baker, Gill und Solovay bewiesen 1975, dass *keine* diagonalisierende Methode die Frage entscheiden kann.**

9.1 Ladners Satz

Wenn P $\neq$ NP, dann gibt es eine Mitte

Eine naheliegende Vermutung wäre: Alles in NP *ist entweder leicht (in* P*) oder maximal schwer (NP-vollständig).* **Diese Vermutung ist falsch.**

Satz 9.2 (Ladner, 1975)**.** *Ist* P $\neq$ NP*, so gibt es eine Sprache* $L \in$ NP*, die*

weder in P *liegt* *noch NP-vollständig ist.*

Man nennt solche Sprachen ***NP-intermediär****.*

Die Beweisidee – Verdünnung.

Man nimmt Sat *und* ***löscht Teile davon****, sodass die Sprache zu dünn wird, um noch NP-vollständig zu sein, aber nicht dünn genug, um in* P *zu liegen.*

Konkret setzt man

$$L := \{\varphi : \varphi \in \text{Sat und } f(|\varphi|) \text{ ist gerade}\}$$

für eine geschickt gewählte, langsam wachsende Funktion f*.*
Die Konstruktion von f **– Diagonalisierung in Zeitlupe.**

f gerade	Die Sprache verhält sich wie SAT (schwer)
f ungerade	Die Sprache ist leer (leicht)

*f wird so gebaut, dass sie **umschaltet**, sobald ein Gegenbeweis droht:*

1. Solange f gerade ist, sucht man nach einer P-Maschine, die L entscheidet. Findet man eine, so erhöht man f (macht es ungerade) – **die Sprache wird dünn, und die Maschine irrt.**
2. Solange f ungerade ist, sucht man nach einer Reduktion $\text{SAT} \leq_{\text{p}} L$. Findet man eine, so erhöht man f wieder – **die Sprache wird dicht, und die Reduktion versagt.**

f wächst so langsam, dass jede Phase lange genug dauert, um den jeweiligen Gegner zu widerlegen. Und f ist selbst in polynomieller Zeit berechenbar, sodass $L \in \text{NP}$ bleibt.

*Der Beweis heißt **Verzögerungsdiagonalisierung** (delayed diagonalization)* – **man diagonalisiert gegen zwei unendliche Listen zugleich, aber immer nur ein Stück auf einmal.**

Die konstruierte Sprache ist künstlich

Ladners Sprache ist kein Problem, das jemanden interessiert. *Sie ist eigens gebaut, um zwischen den Stühlen zu sitzen.*

> *Der Satz sagt:* ***Es gibt*** *eine Mitte.* **Er sagt nicht, dass irgendein *natürliches* Problem darin liegt.**

Die Kandidaten sind trotzdem berühmt:

Faktorisieren	liegt in NP ∩ coNP; NP-vollständig würde NP = coNP erzwingen
Graphisomorphie	seit 2015 quasipolynomiell (Babai)
Diskreter Logarithmus	wie Faktorisieren
Gitterprobleme	Grundlage der Nach-Quanten-Kryptographie

Für keines davon ist bewiesen, dass es intermediär ist – **das würde** $\mathrm{P} \neq \mathrm{NP}$ **voraussetzen.** *Aber alle vier verhalten sich seit Jahrzehnten so.*

Graphisomorphie – ein Beinahe-Durchbruch

Im November 2015 kündigte László Babai in Chicago an, das Graphisomorphieproblem in *quasipolynomieller* Zeit gelöst zu haben – *in* $2^{\mathcal{O}(\log^c n)}$ *statt in* $2^{\mathcal{O}(n^{1/3})}$*, dem Stand seit 1983.*

> *Quasipolynomiell liegt* ***zwischen*** *polynomiell und exponentiell.* **Das Ergebnis brachte die Graphisomorphie sehr nahe an** P **heran** – *ohne sie hineinzubringen.*

Im Januar 2017 fand Harald Helfgott bei der Durchsicht einen Fehler. *Babai zog die Behauptung zurück.*

> *Fünf Tage später hatte er den Fehler behoben.* **Das korrigierte Ergebnis steht.**
>
> *Bemerkenswert an der Episode ist der Umgang damit: Babai veröffentlichte die Rücknahme selbst und sofort, Helfgott veröffentlichte seine Prüfung, und die Korrektur wurde öffentlich diskutiert.*

Die praktische Lage ist übrigens ganz anders als die theoretische. *Das Programm* `nauty` *von* Brendan McKay *löst Isomorphiefragen für Graphen mit Millionen Ecken in Sekunden.*

Man kennt bis heute keine Familie von Graphen, an der die praktischen Verfahren scheitern **– obwohl es sie nach der Theorie geben müsste.**

Aufgabe 9.1.

(a) Führen Sie den Zeithierarchiesatz vollständig durch und diskutieren Sie den log-Faktor.

(b) Untersuchen Sie den Platzhierarchiesatz und seine Folgerungen.

(c) Diskutieren Sie, warum Faktorisieren vermutlich intermediär ist.

Lösung Schritt für Schritt.

(a) Der Zeithierarchiesatz im Detail.

Schritt 1 – Die Aufzählung.

Jede Turingmaschine hat eine endliche Beschreibung, also gibt es eine Aufzählung $M_1, M_2, \ldots$ (Band 2, Kapitel 6). **Man vereinbart, dass jede Maschine unendlich viele Kodierungen hat** – *etwa durch angehängte Nullen* – **sodass sie beliebig lange Eingaben „trifft".**

> ***Dieser technische Punkt ist wesentlich:*** *Die Diagonalisierung muss die Maschine auf einer Eingabe erwischen, die* ***lang genug*** *ist, damit die Simulation durchläuft.*

Schritt 2 – Die Diagonalmaschine.

```
D(w):
  n := |w|;
  Zähler := g(n);      g ist zeitkonstruierbar
  simuliere M_w auf w, dekrementiere
den Zähler je Schritt;
  if Zähler = 0 then akzeptiere;
  if M_w akzeptiert then verwirf else
akzeptiere
```

Schritt 3 – Die Laufzeit von D.

Die Simulation einer Mehrbandmaschine durch eine feste universelle Maschine kostet einen Faktor $\mathcal{O}(\log)$ **– man muss die Bandinhalte verschachtelt speichern und beim Kopfbewegen umsortieren.**

$$T_D(n) = \mathcal{O}\big(g(n)\big) \quad \text{bei geeigneter Buchführung.} \qquad \checkmark$$

$$\Longrightarrow L(D) \in \mathrm{DTIME}(g).$$

Schritt 4 – $L(D) \notin \mathrm{DTIME}(f)$**.**
Angenommen, eine Maschine M mit Laufzeit $c \cdot f(n)$ entscheidet $L(D)$.
Wegen $f(n) \log f(n) = o(g(n))$ gibt es ein n_0, ab dem

$$c \cdot f(n) \cdot \log f(n) < g(n).$$

Wähle eine Kodierung w von M mit $|w| \geq n_0$. **Dann läuft die Simulation vollständig durch, und D tut das Gegenteil von M:**

$$w \in L(D) \iff M \text{ verwirft } w \iff w \notin L(M).$$

Also $L(D) \neq L(M)$ – **Widerspruch.**

Schritt 5 – Warum der log**-Faktor da steht.**

Mehrband-Modell	Faktor $\mathcal{O}(\log f)$ für die Simulation
Einband-Modell	Faktor $\mathcal{O}(f)$ – die Hierarchie wird gröber
RAM-Modell	Faktor $\mathcal{O}(1)$ – der log verschwindet

Der** log**-Faktor ist also eine Eigenschaft des Modells, nicht der Sache. Beim Platz gibt es ihn nicht – die Platzsimulation kostet nur einen konstanten Faktor.

(b) Der Platzhierarchiesatz.

Satz 9.3. *Für platzkonstruierbares g mit $f(n) = o(g(n))$ gilt*

$$\mathrm{SPACE}(f) \subsetneq \mathrm{SPACE}(g).$$

***Kein** log**-Faktor* – die Platzhierarchie ist schärfer als die Zeithierarchie.**

Folgerungen.

$\mathrm{SPACE}(\log n) \subsetneq \mathrm{SPACE}(\log^2 n)$	**also** $\mathrm{L} \subsetneq \mathrm{L}^2$
$\mathrm{L} \subsetneq \mathrm{PSPACE}$	
$\mathrm{NL} \subsetneq \mathrm{PSPACE}$	über Savitch: $\mathrm{NL} \subseteq \mathrm{L}^2 \subsetneq \mathrm{PSPACE}$
$\mathrm{PSPACE} \subsetneq \mathrm{EXPSPACE}$	

Eine hübsche Anwendung.

> *Aus* NL $\subsetneq$ PSPACE *folgt:* ***Mindestens eine*** *der Inklusionen* NL $\subseteq$ P $\subseteq$ NP $\subseteq$ PSPACE *ist echt.*

Ebenso aus P $\subsetneq$ EXP*: Mindestens eine von* P $\subseteq$ NP $\subseteq$ PSPACE $\subseteq$ EXP *ist echt.*

Man weiß also ***sicher****, dass irgendwo eine Trennung liegt* – **und nach fünfzig Jahren keine einzige davon.**

(c) Faktorisieren als Kandidat für die Mitte.
Argument 1 – Es liegt in NP $\cap$ coNP**.**
Gezeigt in Kapitel 6. **Wäre es NP-vollständig, so folgte** NP $=$ coNP **und der Kollaps der Hierarchie.** ✓
Argument 2 – Es hat Struktur, die vollständige Probleme nicht haben.

> *Faktorisieren ist* ***selbstreduzierbar*** *in einem starken Sinn und hat eine* ***Gruppenstruktur*** *im Hintergrund.* **NP-vollständige Probleme sind typischerweise strukturlos** – *gerade das macht sie vollständig.*

Argument 3 – Es ist quantenmechanisch leicht.
SHORS *Algorithmus faktorisiert in polynomieller Quantenzeit* (Band 15).

> *Wäre Faktorisieren NP-vollständig, so wäre* NP $\subseteq$ BQP – **und Quantenrechner könnten alle NP-Probleme schnell lösen.** *Dafür gibt es keinerlei Anzeichen; das Gegenteil wird vermutet.*

Argument 4 – Die subexponentiellen Verfahren.

Verfahren	**Laufzeit**	**Jahr**
Probedivision	$2^{n/2}$	antik
Quadratisches Sieb	$2^{\mathcal{O}(\sqrt{n \log n})}$	1981
Zahlkörpersieb	$2^{\mathcal{O}(n^{1/3} \log^{2/3} n)}$	1990

> ***Für NP-vollständige Probleme kennt man keine subexponentiellen Verfahren*** – und unter der Exponentialzeithypothese (Kapitel 12) gibt es auch keine. **Faktorisieren hat welche.**

Und trotzdem ist nichts bewiesen

Alle vier Argumente sind *Indizien*.

Faktorisieren in P?	offen – niemand hat es ausgeschlossen
Faktorisieren NP-vollständig?	offen – würde NP = coNP bedeuten
Faktorisieren intermediär?	offen – setzt P $\neq$ NP voraus

Auf dieser dreifach offenen Lage ruht die RSA-Verschlüsselung (Kapitel 13) **– und mit ihr ein erheblicher Teil des Zahlungsverkehrs.**

Das ist keine Nachlässigkeit, sondern der Normalzustand der Kryptographie: *Sie beruht auf Annahmen, nicht auf Sätzen* **– und Kapitel 13 zeigt, warum das so bleiben muss.**

Ladners Arbeit[18] ist technisch, aber kurz; Arora und Barak[2] geben eine zugängliche Fassung; Fortnow[8] erzählt die Geschichte der Zwischenklassen allgemeinverständlich.

Part IV

Wege um die Härte

10 Näherungsverfahren

Fast optimal statt optimal

Definition 10.1. Ein Algorithmus für ein Minimierungsproblem hat die **Güte** ρ, wenn für jede Eingabe

$$\frac{A(I)}{\mathrm{OPT}(I)} \leq \rho$$

gilt. Bei Maximierung entsprechend $\mathrm{OPT}(I)/A(I) \leq \rho$.

> ***Entscheidend ist, dass die Schranke für* jede *Eingabe gilt*** – es ist eine Garantie, keine Erfahrung. **Das unterscheidet ein Näherungsverfahren von einer Heuristik.**

Problem	**beste Güte**	**Schranke**
Knotenüberdeckung	2	$\geq 1{,}36$
Metrische Rundreise	1,5	$\geq 123/122$
Max-Cut	1,139	$\geq 17/16$
Mengenüberdeckung	$\ln n$	$\geq (1-\varepsilon)\ln n$
Rucksack	$1+\varepsilon$	beliebig gut
Clique	$n^{1-\varepsilon}$	**praktisch nichts**
Allgemeine Rundreise	keine	**unmöglich**

> ***Die Spannweite ist das Bemerkenswerte:*** *Alle diese Probleme sind NP-vollständig und damit* gleich *schwer zu lösen – aber* ***ganz verschieden*** *gut zu nähern.*

Die NP-Vollständigkeit ist also ein zu grobes Maß. **Die Näherungstheorie liefert die feinere Einteilung.**

Die Klassen der Näherbarkeit

Klasse	Bedeutung	Beispiel
FPTAS	$1+\varepsilon$ in Zeit poly in n *und* $1/\varepsilon$	Rucksack
PTAS	$1+\varepsilon$ in Zeit poly in n für jedes feste ε	euklidische Rundreise
APX	konstante Güte	Knotenüberdeckung
log-APX	Güte $\mathcal{O}(\log n)$	Mengenüberdeckung
poly-APX	Güte $n^{\mathcal{O}(1)}$	Clique

Der Unterschied zwischen PTAS und FPTAS steckt in der Abhängigkeit von ε. Ein PTAS darf $n^{1/\varepsilon}$ brauchen – bei $\varepsilon = 0{,}01$ also n^{100}. **Ein FPTAS ist auf ε polynomiell beschränkt.**

*Beispiel für den Unterschied: Arora und Mitchell gaben 1996 unabhängig ein PTAS für die **euklidische** Rundreise an – Laufzeit $n(\log n)^{\mathcal{O}(1/\varepsilon)}$.* **Bei $\varepsilon = 0{,}1$ ist der Exponent bereits jenseits des Praktischen.**

Metrisch, euklidisch, allgemein – drei Rundreiseprobleme

Variante	Näherbarkeit	Voraussetzung
allgemein	**keine**	beliebige Gewichte
metrisch	1,5 (Christofides)	**Dreiecksungleichung**
euklidisch	**PTAS**	Punkte in der Ebene

Der Beweis der ersten Zeile ist lehrreich und kurz.
Angenommen, es gäbe ein Verfahren mit Güte ρ für die allgemeine Rundreise. Man nehme einen Graphen G und baue daraus eine vollständige Instanz:

$$d(u,v) := \begin{cases} 1 & \text{falls } (u,v) \in E, \\ \rho \cdot n + 1 & \text{sonst.} \end{cases}$$

Hat G einen Hamiltonkreis, so ist $\mathrm{OPT} = n$*, und das Näherungsverfahren liefert höchstens ρn* – **also benutzt es keine teure Kante.** *Hat G keinen, so ist jede Tour $\geq \rho n + 1 > \rho n$.*

Man könnte also am Ergebnis ablesen, ob G einen Hamiltonkreis hat – **ein NP-vollständiges Problem in polynomieller Zeit.**

Der Kunstgriff ist die Lücke: *Man baut eine Instanz, bei der* gute *und* schlechte *Fälle um mehr als den Faktor ρ auseinanderliegen.* **Dann kann ein ρ-Näherungsverfahren sie unterscheiden.**

*Diese **Lückenreduktion** ist das Grundwerkzeug aller Nichtnäherbarkeitsbeweise* – **und der PCP-Satz ist die Aussage, dass es sie für SAT selbst gibt.**

10.1 Der PCP-Satz

Beweise, die man stichprobenartig prüft

Ein NP*-Zertifikat muss **vollständig** gelesen werden.* **Der PCP-Satz sagt, dass sich jeder solche Beweis so umschreiben lässt, dass *konstant viele zufällige Stichproben* genügen.**

Satz 10.2 (PCP-Satz; Arora, Lund, Motwani, Safra, Sudan, Szegedy 1992)**.**

$$\mathrm{NP} = \mathrm{PCP}\big(\mathcal{O}(\log n),\ \mathcal{O}(1)\big).$$

Jede Sprache in NP *hat ein Beweissystem, bei dem der Prüfer* $\mathcal{O}(\log n)$ *Zufallsbits verwendet und nur* ***konstant viele*** *Bits des Beweises liest.*

Vollständigkeit	Ist die Aussage wahr, gibt es einen Beweis, den der Prüfer stets akzeptiert
Korrektheit	Ist sie falsch, so wird **jeder** vorgelegte Beweis mit Wahrscheinlichkeit $\geq \frac{1}{2}$ abgelehnt

Das ist eine erstaunliche Aussage über die Natur mathematischer Beweise. Ein gewöhnlicher Beweis kann durch **einen einzigen** falschen Schritt ungültig werden – man muss ihn ganz lesen.

Ein PCP-Beweis ist so ***robust*** *kodiert, dass ein Fehler sich über den ganzen Text ausbreitet* **– und deshalb an fast jeder Stelle sichtbar wird.**

Die Verbindung zur Näherbarkeit.

Der PCP-Satz ist ***äquivalent*** *zu einer Lückenaussage:*

Es gibt ein $\varepsilon > 0$*, sodass es NP-schwer ist, zwischen „alle Klauseln erfüllbar“ und „höchstens* $(1-\varepsilon)$ *der Klauseln erfüllbar“ zu unterscheiden.*

Daraus folgt sofort: ***Max-3-SAT hat kein PTAS*****, sofern** $\mathrm{P} \neq \mathrm{NP}$**.** ✓

Die Übersetzung: Der Prüfer wird zu einer Formel, jede mögliche Zufallswahl zu einer Klausel. **Akzeptanzwahrscheinlichkeit = Anteil erfüllter Klauseln.** ✓

Zwei Jahre, in denen ein Gebiet entstand

Der PCP-Satz entstand nicht aus dem Nichts, sondern aus einer Kette von Ergebnissen zwischen 1990 und 1992.

1985	Goldwasser u. a.	interaktive Beweise
1990	Lund u. a.	$\mathrm{coNP} \subseteq \mathrm{IP}$
1990	Shamir	$\mathrm{IP} = \mathrm{PSPACE}$
1991	Feige u. a.	erste Nichtnäherbarkeit für Clique
1992	Arora, Safra	$\mathrm{NP} = \mathrm{PCP}(\log n, \log\log n)$
1992	**Arora u. a.**	$\mathrm{NP} = \mathrm{PCP}(\log n, 1)$

SHAMIRS *Ergebnis von 1990 löste die Fachwelt aus:* $\mathrm{IP} = \mathrm{PSPACE}$ **bedeutet, dass ein Prüfer mit begrenzter Rechenkraft, aber Zufall und Rückfragen, sich von einem allmächtigen Beweiser *jede* PSPACE-Aussage belegen lassen kann.**

Die Beweismethode – die ***Arithmetisierung****, bei der Boolesche Formeln in Polynome übersetzt werden – war der Schlüssel zu allem, was folgte.*

Die sechs Autoren des PCP-Satzes erhielten 2001 gemeinsam den Gödel-Preis. **Feige, Goldwasser, Lovász, Safra und Szegedy hatten ihn bereits 1996 für die Vorarbeit erhalten.**

2005 gab IRIT DINUR *einen völlig neuen, wesentlich kürzeren Beweis –* **durch schrittweise „Lückenverstärkung" statt durch algebraische Kodierung.** *Ihr Beweis passt auf etwa dreißig Seiten; der ursprüngliche brauchte über hundert.*

Aufgabe 10.1.

(a) Analysieren Sie den 2-Näherungsalgorithmus für die Knotenüberdeckung vollständig.

(b) Führen Sie den Algorithmus von Christofides durch und beweisen Sie die Güte 1,5.

(c) Untersuchen Sie den gierigen Algorithmus für die Mengenüberdeckung.

Lösung Schritt für Schritt.

(a) Knotenüberdeckung, Faktor 2.

Der Algorithmus.

```
C := leer;
while es gibt eine unbedeckte Kante
(u,v) do
  C := C + {u, v};      beide Endpunkte!
  entferne alle Kanten an u oder v
return C
```

Schritt 1 – Korrektheit.
Die Schleife endet erst, wenn keine Kante mehr unbedeckt ist. **Also ist C eine Überdeckung.** ✓
Schritt 2 – Die Güte.

> *Sei M die Menge der **gewählten Kanten**. Je zwei davon haben keinen gemeinsamen Endpunkt* – **denn nach der Wahl von (u, v) werden alle Kanten an u und v entfernt.**

$$\Longrightarrow M \text{ ist ein } \textbf{Matching}.$$

*Jede Überdeckung muss aus **jeder** Kante von M mindestens einen Endpunkt enthalten. Da die Kanten disjunkt sind:*

$$\mathrm{OPT} \geq |M| .$$

Und der Algorithmus nimmt genau $2\,|M|$ Ecken:

$$|C| = 2\,|M| \leq 2 \cdot \mathrm{OPT}. \qquad \checkmark$$

> ***Der Beweis ist bemerkenswert, weil er das Optimum nie berechnet.*** Er vergleicht mit einer **unteren Schranke**, die der Algorithmus nebenbei erzeugt – dem Matching. *Das ist die Standardbauform aller Güteanalysen.*

Schritt 3 – Ein Beispiel.
Ein Weg mit vier Ecken $a - b - c - d$.

Wahl	**Ergebnis**	**Größe**
Kante (b, c)	$C = \{b, c\}$	2
Optimum	$\{b, c\}$	2
Kante (a, b), dann (c, d)	$C = \{a, b, c, d\}$	4

Im schlechten Fall liefert er 4 statt 2 – genau Faktor 2. **Die Schranke ist also scharf.** ✓

Warum der naive gierige Ansatz *schlechter* ist

Es liegt nahe, stets die Ecke mit dem *höchsten Grad* zu nehmen.

Das ist intuitiv besser und liefert nur die Güte $\mathcal{O}(\log n)$ – **schlechter als das simple Verfahren oben.**

Der Grund: Beim gierigen Verfahren gibt es keine natürliche untere Schranke, mit der man vergleichen könnte. **Das „dumme" Verfahren erzeugt eine** – *das Matching.*

Eine der lehrreichsten Beobachtungen der Näherungstheorie: *Das Verfahren mit der besseren Garantie ist oft nicht das intuitiv klügere.*

Schritt 4 – Die untere Schranke.

Ergebnis	Schranke	Voraussetzung
Dinur, Safra 2005	1,3606	$\mathrm{P} \neq \mathrm{NP}$
Khot, Regev 2008	$2 - \varepsilon$	**Hypothese der eindeutigen Spiele**

*Unter der **Unique-Games-Hypothese** ist der Faktor 2 **optimal*** – **der triviale Algorithmus wäre dann der bestmögliche.** *Die Hypothese ist seit 2002 offen und eines der wichtigsten ungelösten Probleme des Gebiets.*

(b) Christofides' Algorithmus.
*Voraussetzung: Die Abstände erfüllen die **Dreiecksungleichung** $d(u,w) \leq d(u,v) + d(v,w)$.*
Der Algorithmus.

1. Berechne einen **minimalen Spannbaum** T (Band 1, Kapitel 12).
2. Sei U die Menge der Ecken **ungeraden Grades** in T.
3. Berechne ein **minimales perfektes Matching** M auf U.
4. $T \cup M$ hat nur gerade Grade – bestimme einen **Eulerkreis**.
5. Kürze ihn ab, indem du wiederholt besuchte Ecken überspringst.

Schritt 1 – Warum $|U|$ gerade ist.
Die Gradsumme eines Graphen ist $2m$, also gerade (Mathematik-Band 15). **Die Zahl der Ecken ungeraden Grades muss daher gerade sein** – *sonst wäre die Summe ungerade.* ✓

Also existiert ein perfektes Matching auf U. **Und es ist in polynomieller Zeit berechenbar** (Edmonds 1965). ✓

Schritt 2 – Die erste Abschätzung: $w(T) \leq \mathrm{OPT}$.

Entfernt man aus einer optimalen Rundreise eine Kante, so entsteht ein ***Weg*** *durch alle Ecken – also ein Spannbaum.* **Der minimale Spannbaum ist höchstens so schwer.** ✓

Schritt 3 – Die zweite Abschätzung: $w(M) \leq \frac{1}{2}\mathrm{OPT}$.

Man betrachte die optimale Rundreise und darauf die Ecken aus U in der Reihenfolge ihres Auftretens. **Kürzt man die Tour auf diese Ecken ab (unter Ausnutzung der Dreiecksungleichung), so erhält man einen Kreis auf U mit Gewicht $\leq \mathrm{OPT}$.**

Ein Kreis gerader Länge zerfällt in ***zwei*** *perfekte Matchings – die Kanten abwechselnd genommen.* **Das leichtere hat höchstens das halbe Gewicht:**

$$w(M) \leq \frac{1}{2}\,\mathrm{OPT}. \qquad \checkmark$$

Schritt 4 – Zusammensetzen.

$$w(\text{Eulerkreis}) = w(T) + w(M) \leq \mathrm{OPT} + \tfrac{1}{2}\mathrm{OPT} = \tfrac{3}{2}\,\mathrm{OPT}.$$

Das Abkürzen in Schritt 5 verkleinert das Gewicht wegen der Dreiecksungleichung nur:

$$\boxed{w(\text{Tour}) \leq 1{,}5 \cdot \mathrm{OPT} \qquad \checkmark}$$

NICOS CHRISTOFIDES *veröffentlichte das 1976 als internen Bericht der Carnegie Mellon University.* Er wurde nie in einer Zeitschrift publiziert – **und blieb 45 Jahre lang der beste bekannte Faktor.**

2020 verbesserten KARLIN, KLEIN *und* OVEIS GHARAN *ihn auf* $1{,}5 - 10^{-36}$ **– ein winziger, aber grundsätzlich bedeutsamer Fortschritt**, *denn er zeigte, dass 1,5 keine natürliche Schranke ist.*

(c) Mengenüberdeckung, gierig.
Gegeben ein Universum $\mathcal{U}$ mit n Elementen und Teilmengen $S_1, \ldots, S_m$. Gesucht: möglichst wenige Mengen, die alles überdecken.
Der Algorithmus.

*Nimm stets die Menge, die die **meisten noch unbedeckten** Elemente enthält.*

Satz 10.3. *Der gierige Algorithmus hat die Güte $H_n = \sum_{k=1}^{n} \frac{1}{k} \approx \ln n$.*

Der Beweis – Kosten auf Elemente verteilen.
Wählt der Algorithmus eine Menge, die k neue Elemente überdeckt, so belaste man jedes dieser Elemente mit den Kosten $\frac{1}{k}$.

$$|\text{Lösung}| = \sum_{e \in \mathcal{U}} \text{kosten}(e).$$

Sei OPT *eine optimale Überdeckung. Betrachte eine ihrer Mengen S mit $|S| = s$.*

Nummeriere die Elemente von S in der Reihenfolge, in der der gierige Algorithmus sie überdeckt: $e_1, \ldots, e_s$.

Wenn e_i überdeckt wird, sind noch mindestens $s - i + 1$ Elemente von S frei – **also hätte der Algorithmus mit S mindestens $s - i + 1$ neue Elemente bekommen.**

*Da er die **beste** Menge nimmt, überdeckt seine Wahl mindestens so viele:*

$$\text{kosten}(e_i) \leq \frac{1}{s - i + 1}.$$

$$\sum_{e \in S} \text{kosten}(e) \leq \sum_{i=1}^{s} \frac{1}{s - i + 1} = H_s \leq H_n.$$

Summiert über alle $|\text{OPT}|$ Mengen der optimalen Lösung – die zusammen alles überdecken:

$$|\text{Lösung}| \leq |\text{OPT}| \cdot H_n.$$

Die Beweistechnik heißt Kostenverteilung und ist die zweite Standardbauform neben der unteren Schranke aus Teil (a). **Man verteilt die Kosten der eigenen Lösung auf die Elemente und schätzt dann pro Menge des Optimums ab.**

Ist $\ln n$ optimal?

Ergebnis	Schranke	Voraussetzung
Lund, Yannakakis 1994	$\frac{1}{2}\log n$	
Feige 1998	$(1-\varepsilon)\ln n$	NP $\not\subseteq$ DTIME$(n^{\log\log n})$
Dinur, Steurer 2014	$(1-\varepsilon)\ln n$	**nur** P $\neq$ NP

Der gierige Algorithmus ist also optimal – man kann ihn nicht einmal um einen Faktor $(1-\varepsilon)$ verbessern, sofern P $\neq$ NP.

Das ist ein befriedigender Zustand, den man selten hat: **obere und untere Schranke stimmen überein.** *Bei der Knotenüberdeckung klafft dagegen noch die Lücke zwischen* 1,36 *und* 2.

Vazirani[25] ist das Standardwerk der Näherungsverfahren; Håstads Arbeit[12] enthält die optimalen Nichtnäherbarkeitsergebnisse; Arora und Barak[2] beweisen den PCP-Satz in Dinurs Fassung.

11 Zufall als Ressource

Die randomisierten Klassen

Definition 11.1. Eine probabilistische Turingmaschine darf Münzwürfe verwenden.

Klasse	**bei** $w \in L$	**bei** $w \notin L$
RP	akzeptiert mit Wkt. $\geq \frac{1}{2}$	**verwirft stets**
coRP	**akzeptiert stets**	verwirft mit Wkt. $\geq \frac{1}{2}$
BPP	akzeptiert mit Wkt. $\geq \frac{2}{3}$	verwirft mit Wkt. $\geq \frac{2}{3}$
ZPP	stets korrekt, Laufzeit im Mittel polynomiell	

RP ***hat einseitigen Fehler:*** *Sagt die Maschine „ja“, so stimmt es sicher.* BPP **hat zweiseitigen Fehler, dafür beide Richtungen gleich gut.**

Die Kette: $\mathrm{P} \subseteq \mathrm{ZPP} \subseteq \mathrm{RP} \subseteq \mathrm{BPP} \subseteq \mathrm{PSPACE}$, *und* $\mathrm{RP} \subseteq \mathrm{NP}$.

Warum die Konstanten gleichgültig sind

Satz 11.2 (Wahrscheinlichkeitsverstärkung)**.** *Die Schranke $\frac{2}{3}$ in der Definition von* BPP *lässt sich durch jedes $\frac{1}{2} + \frac{1}{n^c}$ ersetzen, ohne die Klasse zu ändern – und man kann den Fehler auf 2^{-n^k} drücken.*

Der Beweis – Mehrheitsentscheid.
Man lasse den Algorithmus t-mal unabhängig laufen und entscheide nach der Mehrheit. **Nach der Chernoff-Schranke** (Mathematik-Band 4) **fällt die Fehlerwahrscheinlichkeit exponentiell in t:**

$$\Pr[\text{Mehrheit irrt}] \leq \mathrm{e}^{-2t\varepsilon^2} \quad \text{bei Vorteil } \varepsilon.$$

t	**Fehler bei $\varepsilon = \frac{1}{6}$**	**Vergleich**
10	0,20	
100	0,004	
1000	10^{-25}	**Hardwarefehler**
10 000	10^{-241}	Atome im Universum: 10^{80}

Bei tausend Wiederholungen ist die Irrtumswahrscheinlichkeit kleiner als die Wahrscheinlichkeit eines Hardwarefehlers während der Rechnung. Ein randomisiertes Verfahren ist dann praktisch so verlässlich wie ein deterministisches.

Die große Wende: Zufall hilft vermutlich nicht

Bis etwa 1990 hielt man $\mathrm{P} \subsetneq \mathrm{BPP}$ *für wahrscheinlich* – **Zufall schien echte Rechenkraft zu verleihen.** *Heute glaubt fast niemand mehr daran.*

Satz 11.3 (Impagliazzo und Wigderson, 1997)**.** *Gibt es eine Sprache in* EXP*, die keine Schaltkreise der Größe $2^{\varepsilon n}$ hat, so gilt*

$$\mathrm{P} = \mathrm{BPP}.$$

In Worten: Härte lässt sich in Zufall verwandeln. Wer eine schwere Funktion kennt, kann daraus einen **Pseudozufallsgenerator** bauen, der jeden randomisierten Algorithmus täuscht – und dann probiert man alle Startwerte durch.

Übliche Vermutung	Schwere Probleme existieren
Also	Pseudozufall existiert
Also	P = BPP

Das ist eine bemerkenswerte Umkehrung: ***Je schwerer die Welt, desto weniger nützt der Zufall.***

Und es gibt empirische Belege.

Problem	**randomisiert**	**deterministisch**
Primzahltest	1976	**2002** (AKS)
Ungerichteter Zusammenhang	1979	**2004** (Reingold)
Polynomidentität	1977	offen

Zweimal wurde der Zufall nachträglich entfernt – jeweils nach 25 Jahren. **Die dritte Zeile ist der prominenteste offene Fall.**

BPP hat kein bekanntes vollständiges Problem

Das ist eine ungewöhnliche Eigenschaft. P, NP, PSPACE *und* EXP *haben alle vollständige Probleme;* BPP *nicht.*

> *Der Grund: Die Definition von* BPP *enthält eine* ***semantische*** *Bedingung* – „die Maschine irrt selten". **Ob eine gegebene Maschine das tut, ist unentscheidbar** (Band 2, Satz von Rice).

Man kann die Maschinen von BPP *also nicht aufzählen* – **und ohne Aufzählung gibt es weder ein universelles Problem noch einen Hierarchiesatz.**

Man nennt solche Klassen ***semantisch****, im Gegensatz zu* ***syntaktischen*** *wie* P *und* NP. **Semantische Klassen sind theoretisch viel unhandlicher.**

Aufgabe 11.1.

(a) Analysieren Sie den Miller-Rabin-Primzahltest vollständig.

(b) Untersuchen Sie das Schwartz-Zippel-Lemma und den Polynomidentitätstest.

(c) Diskutieren Sie Kargers Algorithmus für den minimalen Schnitt.

Lösung Schritt für Schritt.

(a) Miller-Rabin.
Schritt 1 – Die Grundlage.
Nach dem kleinen Satz von Fermat gilt für Primzahlen p und $1 \leq a < p$:

$$a^{p-1} \equiv 1 \pmod{p}.$$

Die Umkehrung gilt ***nicht*** **– es gibt Carmichael-Zahlen wie** $561 = 3 \cdot 11 \cdot 17$**, die den Test für *alle* teilerfremden a bestehen.**

Schritt 2 – Die Verschärfung.
Man schreibe $n - 1 = 2^s \cdot d$ mit ungeradem d.

> *Ist n prim, so hat die Gleichung $x^2 \equiv 1$ modulo n genau die Lösungen $x \equiv \pm 1$* – **denn $\mathbb{Z}/n\mathbb{Z}$ ist dann ein Körper** (Mathematik-Band 11).

Betrachtet man die Folge

$$a^d,\ a^{2d},\ a^{4d},\ \ldots,\ a^{2^s d} = a^{n-1},$$

so muss sie bei einer Primzahl entweder mit 1 beginnen oder irgendwo eine -1 enthalten.

```
MillerRabin(n, k):
  schreibe n-1 = 2^s * d, d ungerade;
  wiederhole k mal {
    wähle a zufällig aus {2,...,n-2};
    x := a^d mod n;
    if x = 1 oder x = n-1 then weiter;
    wiederhole s-1 mal { x := x^2 mod
n;
      if x = n-1 then weiter mit äußerer
Schleife }
    return "zusammengesetzt" }
  return "wahrscheinlich prim"
```

Schritt 3 – Ein Durchlauf mit $n = 561$.
$560 = 2^4 \cdot 35$*, also* $s = 4$, $d = 35$*. Wähle* $a = 2$.

Potenz	Wert mod 561	Prüfung
2^{35}	263	$\neq 1, \neq 560$
2^{70}	166	$\neq 560$
2^{140}	67	$\neq 560$
2^{280}	1	1, **aber Vorgänger war** 67

Hier ist der Widerspruch: $67^2 \equiv 1$, *aber* $67 \not\equiv \pm 1$ – **eine „nichttriviale Quadratwurzel der Eins".** *In einem Körper kann es die nicht geben.*

$$\Longrightarrow 561 \text{ ist zusammengesetzt.} \qquad \checkmark$$

Und tatsächlich: $\mathrm{ggT}(67-1, 561) = \mathrm{ggT}(66, 561) = 33$ – **der Test liefert nebenbei einen Faktor.**

Schritt 4 – Die Fehlerschranke.

Satz 11.4 (Rabin, 1980)**.** *Ist n zusammengesetzt, so sind höchstens $\frac{1}{4}$ aller a* ***Lügner*** *– also Zeugen, die den Test bestehen lassen.*

$$\Pr[\text{Irrtum nach } k \text{ Runden}] \leq 4^{-k}.$$

k	Schranke	Praxis
10	10^{-6}	
40	10^{-24}	**OpenSSL-Standard**
64	10^{-39}	

Der Test ist einseitig: *Sagt er „zusammengesetzt", so stimmt das* ***sicher*** – der gefundene Widerspruch ist ein Beweis. **Nur „prim" ist wahrscheinlichkeitsbehaftet.** *Also liegt Primalität in* coRP.

Warum AKS den Miller-Rabin-Test nicht verdrängt hat

Verfahren	Aufwand	Bemerkung
Miller-Rabin	$\mathcal{O}(k \log^3 n)$	randomisiert
AKS (2002)	$\mathcal{O}(\log^{12} n)$	deterministisch
AKS (verbessert)	$\mathcal{O}(\log^6 n)$	

Bei einer 2048-Bit-Zahl ist $\log^3 n \approx 10^{10}$ *gegen* $\log^6 n \approx 10^{20}$ – **ein Faktor** 10^{10}**.** *Kein kryptographisches System benutzt AKS.*

Der Wert von AKS ist theoretisch, *und zwar erheblich: Er ist der Beleg dafür, dass Zufall bei diesem Problem entbehrlich war* – **ein Datenpunkt für** $\mathrm{P} = \mathrm{BPP}$**.**

(b) Polynomidentität.
Aufgabe: Gegeben zwei Polynome in n Variablen, dargestellt als Schaltkreise – sind sie gleich?

Ausmultiplizieren hilft nicht: Ein Schaltkreis der Größe s kann ein Polynom mit 2^s *Termen darstellen.* **Etwa** $\prod_{i=1}^{n}(x_i + y_i)$ **– klein hingeschrieben, exponentiell viele Terme.**

Satz 11.5 (Schwartz-Zippel-Lemma). *Sei* $p \neq 0$ *ein Polynom vom Grad d über einem Körper und S eine endliche Teilmenge. Werden* $r_1, \ldots, r_n$ *unabhängig gleichverteilt aus S gewählt, so gilt*

$$\Pr[p(r_1, \ldots, r_n) = 0] \leq \frac{d}{|S|}.$$

Der Beweis – Induktion über n**.**
$n = 1$*: Ein Polynom vom Grad d in einer Variablen hat höchstens d Nullstellen* (Mathematik-Band 11). **Also ist die Wahrscheinlichkeit höchstens** $d/|S|$**.** ✓
Schritt: *Schreibe*

$$p(x_1, \ldots, x_n) = \sum_{i=0}^{k} x_1^i \cdot q_i(x_2, \ldots, x_n),$$

wobei k der höchste Grad in x_1 *ist und* $q_k \neq 0$*.*
Zwei Fälle für die Wahl von $r_2, \ldots, r_n$*:*

$q_k(r_2, \dots, r_n) = 0$	Wkt. $\leq \frac{d-k}{\lvert S\rvert}$ nach Induktion (q_k hat Grad $\leq d-k$)
$q_k(r_2, \dots, r_n) \neq 0$	Dann ist $p(x_1, r_2, \dots)$ ein Polynom vom Grad genau k in x_1; Wkt. einer Nullstelle $\leq \frac{k}{\lvert S\rvert}$

$$\Pr \leq \frac{d-k}{|S|} + \frac{k}{|S|} = \frac{d}{|S|}.$$

Der Algorithmus.

Um $p = q$ zu prüfen, teste man $p - q = 0$: Wähle $|S| \geq 2d$ und werte an einem zufälligen Punkt aus. **Fehlerwahrscheinlichkeit $\leq \frac{1}{2}$.** ✓

Ein einziger Auswertungspunkt – statt exponentiell vieler Terme. **Das ist einer der schlagendsten Fälle, in denen Zufall etwas leistet.**

Anwendungen.

Matrixprodukt prüfen	Ist $AB = C$? In $\mathcal{O}(n^2)$ statt $\mathcal{O}(n^{2{,}37})$ (Freivalds)
Perfektes Matching	Determinante der Tutte-Matrix
Primzahltest	Grundlage von AKS

Ob Polynomidentität in* P *liegt, ist offen – und es ist *das* Testproblem für die Derandomisierung. **KABANETS und IMPAGLIAZZO zeigten 2003: Eine Derandomisierung würde untere Schaltkreisschranken liefern** – *also genau das, woran man seit Jahrzehnten scheitert.*

(c) Kargers Algorithmus für den minimalen Schnitt.
Gesucht: Die kleinste Kantenmenge, deren Entfernung den Graphen zerfallen lässt.
Der Algorithmus (Karger, 1993).

```
while mehr als 2 Ecken übrig do
  wähle eine Kante (u,v) gleichverteilt
zufällig;
  ziehe u und v zu einer Ecke zusammen
(Mehrfachkanten bleiben,
  Schleifen werden gelöscht)
return die Kanten zwischen den beiden
Restecken
```

Verblüffend einfach – und es funktioniert, weil ein minimaler Schnitt **wenige** Kanten hat und deshalb selten getroffen wird.

Die Analyse.
Sei C ein minimaler Schnitt mit k Kanten. Dann hat jede Ecke Grad $\geq k$ – **sonst wäre sie allein ein kleinerer Schnitt.** *Also*

$$m \geq \frac{nk}{2}.$$

Die Wahrscheinlichkeit, im ersten Schritt ***keine*** *Kante aus C zu treffen:*

$$1 - \frac{k}{m} \geq 1 - \frac{2}{n}.$$

Nach i Zusammenziehungen sind $n - i$ Ecken übrig, und dasselbe Argument gilt:

$$\Pr[C \text{ überlebt}] \geq \prod_{i=0}^{n-3} \Big(1 - \frac{2}{n-i}\Big) = \prod_{j=3}^{n} \frac{j-2}{j}.$$

Das Produkt teleskopiert:

$$\frac{1}{3} \cdot \frac{2}{4} \cdot \frac{3}{5} \cdots \frac{n-2}{n} = \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

$$\boxed{\Pr[\text{Erfolg}] \geq \frac{2}{n(n-1)}}$$

Die Verstärkung.
Man wiederhole $t = \binom{n}{2} \ln n$ mal:

$$\Pr[\text{nie erfolgreich}] \leq \Big(1 - \frac{1}{\binom{n}{2}}\Big)^t \leq \mathrm{e}^{-\ln n} = \frac{1}{n}.$$

Gesamtaufwand: $\mathcal{O}(n^2 \log n)$ Läufe zu je $\mathcal{O}(n^2)$ – $\mathcal{O}(n^4 \log n)$. ✓

Karger und Stein verbesserten das 1996 auf $\mathcal{O}(n^2 \log^3 n)$, indem sie die frühen Zusammenziehungen (die selten schiefgehen) teilen und nur die späteren wiederholen – **eine rekursive Bauform.**

Ein Nebenergebnis, das kein Zufallsverfahren liefert

Aus der Analyse folgt ein rein kombinatorischer Satz.

> *Jeder minimale Schnitt überlebt mit Wahrscheinlichkeit* $\geq 1/\binom{n}{2}$. **Da sich die Ereignisse für verschiedene Schnitte ausschließen, kann es höchstens** $\binom{n}{2}$ **minimale Schnitte geben.**

$$\boxed{\text{Ein Graph hat höchstens } \binom{n}{2} = \frac{n(n-1)}{2} \text{ minimale Schnitte.}}$$

Und die Schranke ist scharf: Der Kreis mit n Ecken hat genau $\binom{n}{2}$ *minimale Schnitte –* **je zwei Kanten entfernt trennt ihn.** ✓

> ***Eine Aussage ohne Zufall, bewiesen mit Zufall.*** Das ist die **probabilistische Methode** von PAUL ERDŐS – eine der fruchtbarsten Beweistechniken der Kombinatorik (Mathematik-Band 15).

Motwani und Raghavan[20] sind das Standardwerk; Arora und Barak[2] behandeln die Derandomisierung; das Ergebnis von Impagliazzo und Wigderson ist in Wigderson[27] allgemeinverständlich dargestellt.

12 Parametrisierte Komplexität

Nicht alle Eingaben sind gleich groß

*Die klassische Theorie misst die Eingabe mit **einer** Zahl n.* **Oft gibt es aber eine zweite Größe, die klein bleibt** – *und dann ist das Problem in der Praxis leicht.*

Problem	**Parameter k**	**typische Größe**
Knotenüberdeckung	Größe der Überdeckung	klein bei dünnen Netzen
Datenbankanfrage	Länge der Anfrage	$k \leq 20$
Typprüfung	Schachtelungstiefe	$k \leq 10$
Motivsuche in Genen	Motivlänge	$k \leq 15$

Definition 12.1. Ein parametrisiertes Problem heißt **fest-parameter-handhabbar** (FPT), wenn es einen Algorithmus mit Laufzeit

$$f(k) \cdot n^{\mathcal{O}(1)}$$

gibt – f beliebig, aber **unabhängig von** n.

Der Unterschied zu $\mathcal{O}(n^k)$ ist entscheidend. Bei $f(k) \cdot n^c$ steht k **nicht im Exponenten von** n – die Abhängigkeiten sind getrennt.

	$n = 10^6, k = 10$	$n = 10^6, k = 20$	**Art**
n^k	10^{60}	10^{120}	XP
$2^k \cdot n$	10^9	10^{12}	**FPT**
$1{,}28^k \cdot n$	10^7	10^8	**FPT**

Der Unterschied zwischen Zeile 1 und Zeile 2 ist der Unterschied zwischen unmöglich und Sekunden.

Die W-Hierarchie

Nicht jedes Problem ist FPT – und man braucht wieder eine Härtetheorie, um das zu zeigen.

Klasse	Beispiel	Status
FPT	Knotenüberdeckung	lösbar
W[1]	**Clique**, Unabh. Menge	vermutlich nicht FPT
W[2]	**Mengenüberdeckung**, dominierende Menge	
XP	alles mit $n^{f(k)}$	

$$\text{FPT} \subseteq \text{W}[1] \subseteq \text{W}[2] \subseteq \cdots \subseteq \text{XP}.$$

Die Leitvermutung lautet FPT $\neq$ W[1] – das Gegenstück zu P $\neq$ NP auf dieser Ebene. **Sie folgt aus der Exponentialzeithypothese.**

Definition 12.2 (Impagliazzo und Paturi, 2001)**.**

ETH	3-SAT ist nicht in Zeit $2^{o(n)}$ lösbar
SETH	Für jedes $\varepsilon > 0$ gibt es ein k, sodass k-SAT nicht in $2^{(1-\varepsilon)n}$ lösbar ist

Die ETH ist stärker als P $\neq$ NP***:*** *Sie behauptet nicht nur, dass es kein polynomielles Verfahren gibt, sondern dass auch kein* subexponentielles *existiert.*

Aus ihr folgen ***scharfe*** *untere Schranken* – **etwa: Clique braucht $n^{\Omega(k)}$, ist also nicht FPT.**

Warum man Hypothesen statt Sätze benutzt

Die klassische Theorie kann nur sagen: „NP-vollständig“. *Das unterscheidet nicht zwischen* $2^{0{,}1n}$ *und* 2^n.

> *Unter der ETH lassen sich dagegen Aussagen der Form „dieses Problem braucht genau* $2^{\Theta(n)}$*“ beweisen* – **und das ist die Auskunft, die ein Praktiker braucht.**

Man nennt das Gebiet ***feinkörnige Komplexität****.* **Es untersucht Probleme *innerhalb* von** P **ebenso wie die Feinstruktur der harten:**

Problem	**obere Schranke**	**unter SETH**
Editierabstand	$\mathcal{O}(n^2)$	nicht $\mathcal{O}(n^{2-\varepsilon})$
Orthogonale Vektoren	$\mathcal{O}(n^2)$	nicht $\mathcal{O}(n^{2-\varepsilon})$
Durchmesser (Graph)	$\mathcal{O}(nm)$	nicht $\mathcal{O}(n^{2-\varepsilon})$

Der Editierabstand aus Band 1 (Kapitel 14) ist also vermutlich optimal gelöst – **das quadratische Verfahren lässt sich nicht wesentlich verbessern.**

Aufgabe 12.1.

(a) Entwickeln Sie den FPT-Algorithmus für die Knotenüberdeckung schrittweise.

(b) Untersuchen Sie die Kernbildung als eigenständige Technik.

(c) Diskutieren Sie, warum Clique vermutlich nicht FPT ist.

Lösung Schritt für Schritt.

(a) Knotenüberdeckung, parametrisiert.
Frage: Hat G *eine Knotenüberdeckung der Größe* $\leq k$*?*
Verfahren 1 – Erschöpfende Suche.

$$\binom{n}{k} = \mathcal{O}(n^k) \quad \text{Teilmengen prüfen.}$$

Bei $n = 10^6$, $k = 10$*:* 10^{60} – **unmöglich.** *Das ist XP, nicht FPT.*

Verfahren 2 – Verzweigen an einer Kante.

Die Schlüsselbeobachtung: *Für jede Kante (u, v) muss* ***mindestens einer*** *der beiden Endpunkte in der Überdeckung liegen.*

```
VC(G, k):
  if G hat keine Kanten then return wahr;
  if k = 0 then return falsch;
  wähle eine Kante (u,v);
  return VC(G - u, k-1) oder VC(G - v, k-1)
```

Die Analyse.

Der Suchbaum ist binär und hat Tiefe k – **also höchstens 2^k Blätter.** *Jeder Knoten kostet $\mathcal{O}(n+m)$ für das Entfernen.*

$$\boxed{T(n,k) = \mathcal{O}\big(2^k \cdot (n+m)\big) \quad \textbf{– FPT}}$$

k	2^k	**bei** $n + m = 10^6$
10	1024	10^9 – Sekunden
20	10^6	10^{12} – Stunden
40	10^{12}	unmöglich

Verfahren 3 – Bessere Verzweigung.

Man wähle statt einer beliebigen Kante die Ecke v mit ***höchstem Grad*** *$d \geq 3$.* **Entweder v ist in der Überdeckung – oder *alle* d Nachbarn sind es.**

Fall 1	v hinein: Restproblem mit $k - 1$
Fall 2	alle Nachbarn hinein: Restproblem mit $k - d$

Die Rekursionsgleichung

$$T(k) = T(k-1) + T(k-3)$$

hat die Lösung $\mathcal{O}(1{,}466^k)$ **– die Wurzel von $x^3 = x^2 + 1$.**

Ecken vom Grad ≤ 2 behandelt man gesondert: Grad 1 nimmt man den Nachbarn, Grad 2 löst man mit einer lokalen Regel. ✓

Verfahren	Basis	$k = 40$
naive Verzweigung	2	10^{12}
nach Grad	1,466	$10^{6,6}$
Stand 2026	**1,25**	$\mathbf{10^{3,9}}$

Von 10^{12} ***auf*** 10^4 **– *allein durch bessere Verzweigungsregeln.*** Das Wettrennen um die kleinste Basis ist ein eigener Zweig des Gebiets.

(b) Kernbildung.

Definition 12.3. Ein **Kern** (*kernel*) ist eine in polynomieller Zeit berechenbare Reduktion, die eine Instanz (I, k) in eine **äquivalente** Instanz (I', k') überführt mit

$$|I'| \leq g(k), \qquad k' \leq k$$

für eine Funktion g, die nur von k abhängt.

Man schrumpft die Eingabe auf eine Größe, die nur noch vom Parameter abhängt – unabhängig davon, wie groß n war.

Die Regeln von Buss für die Knotenüberdeckung.

R1	Hat eine Ecke Grad $> k$, so **muss** sie in die Überdeckung. Entferne sie, setze $k := k - 1$.
R2	Entferne isolierte Ecken.
R3	Bleiben danach mehr als k^2 Kanten, so ist die Antwort **nein**.

Begründung von R1.
*Wäre v mit Grad $> k$ nicht in der Überdeckung, so müssten **alle** seine $> k$ Nachbarn darin sein* – **mehr als** k **Ecken.** *Widerspruch.* ✓
Begründung von R3.
Nach R1 hat jede Ecke Grad $\leq k$. Eine Überdeckung mit k Ecken deckt also höchstens $k \cdot k = k^2$ Kanten ab. ✓

Nach Anwendung der Regeln hat der Graph höchstens k^2 Kanten und damit höchstens $2k^2$ nichtisolierte Ecken.

Kern der Größe $\mathcal{O}(k^2)$

Ein Zahlenbeispiel.

Stufe	Größe	Aufwand danach
Eingabe	$n = 10^6$	
nach Kernbildung ($k = 20$)	≤ 800 Ecken	
Verzweigung $1{,}25^{20}$		$\approx 10^4$

Die Kernbildung ist in der Praxis oft wirksamer als die Verzweigung. Sie ist reine Vorverarbeitung in polynomieller Zeit – und danach ist das Problem winzig.

Satz 12.4 (Nemhauser und Trotter, 1974)**.** *Für die Knotenüberdeckung gibt es einen Kern mit $\leq 2k$* ***Ecken*** *– über die Lösung eines linearen Programms.*

FPT und Kernbildung sind dasselbe

Satz 12.5. *Ein parametrisiertes Problem ist genau dann in FPT, wenn es einen Kern besitzt.*

„⇐“: Kern berechnen, dann erschöpfend suchen – Aufwand $g(k)$-abhängig. ✓
„⇒“: Sei ein FPT-Algorithmus mit $f(k) \cdot n^c$ gegeben. Man lasse ihn n^{c+1} Schritte laufen.

Er terminiert	Antwort bekannt; gib eine triviale Instanz aus
Er terminiert nicht	Dann ist $n^{c+1} < f(k)n^c$, also $n < f(k)$ – die Instanz ist **schon** klein genug

In beiden Fällen hat man einen Kern der Größe $f(k)$. ✓

Der Satz ist theoretisch schön und praktisch unbrauchbar – der so gewonnene Kern hat die Größe $f(k)$, also möglicherweise 2^{2^k}. **Die interessante Frage ist, ob es *polynomielle* Kerne gibt**, *und dafür gibt es eine eigene Härtetheorie* (Bodlaender u. a., 2009).

(c) Warum Clique vermutlich nicht FPT ist.

Schritt 1 – Der triviale Algorithmus.

$$\binom{n}{k} \cdot \mathcal{O}(k^2) = \mathcal{O}(n^k \cdot k^2) \quad \text{– XP, nicht FPT.}$$

Schritt 2 – Warum die Verzweigung hier versagt.

> *Bei der Knotenüberdeckung gab es eine* ***lokale Zwangsbedingung****: Für jede Kante muss ein Endpunkt gewählt werden.* **Das erlaubt die binäre Verzweigung mit Tiefe k.**
>
> *Bei der Clique gibt es keine solche Bedingung.* **Keine Ecke *muss* in die Clique** *– und es gibt keinen lokalen Anhaltspunkt, wo man ansetzen soll.*

Schritt 3 – Die formale Aussage.

Satz 12.6 (Downey und Fellows, 1995)**.** *Clique ist* W[1]*-vollständig unter FPT-Reduktionen.*

Also: Clique ist genau dann FPT, wenn FPT $=$ W[1] – **das Gegenstück zu Satz 4.4.**

Schritt 4 – Die schärfere Aussage unter ETH.

Satz 12.7 (Chen u. a., 2006)**.** *Unter der ETH gibt es keinen Algorithmus für Clique mit Laufzeit $f(k) \cdot n^{o(k)}$.*

> ***Der Exponent k ist also unvermeidlich.*** Der triviale Algorithmus $\mathcal{O}(n^k)$ ist im Wesentlichen optimal – bis auf Konstanten im Exponenten.

Das Beste, was man kennt, ist $\mathcal{O}(n^{0{,}792k})$ über schnelle Matrixmultiplikation (Nešetřil und Poljak, 1985) – **eine Verbesserung des Exponenten um einen Faktor, nicht der Größenordnung.**

Zwei Probleme, dieselbe klassische Härte, verschiedene Praxis

	Knotenüberd.	**Unabh. Menge**
klassisch	NP-vollständig	NP-vollständig
Reduktion	ineinander in $\mathcal{O}(n)$ (Kapitel 5)	
parametrisiert	**FPT**	**W[1]-vollständig**
Näherbarkeit	Faktor 2	$n^{1-\varepsilon}$

Dieselben zwei Probleme, die in Kapitel 5 als „dasselbe" erschienen, trennen sich hier scharf. Der Grund ist derselbe wie dort: Die Reduktion $S \mapsto V \setminus S$ erhält die Lösung, aber nicht den **Parameter**.

Eine Überdeckung der Größe k entspricht einer unabhängigen Menge der Größe $n - k$ **– aus einem kleinen Parameter wird ein riesiger.** ✓

***Die parametrisierte Theorie ist damit ein* feineres *Werkzeug als die klassische* – sie unterscheidet Probleme, die P gegen NP in einen Topf wirft.**
Und die Näherungstheorie ist ein drittes, wieder anders geartetes.

Downey und Fellows[6] sind die Begründer und ihr Buch das Standardwerk; Cygan und Mitautoren[5] ist die modernere Darstellung mit vielen Techniken; Impagliazzo[14] entwickelt die Sicht auf die Durchschnittsfallhärte.

13 Härte als Werkzeug

Das einzige Gebiet, das Schwierigkeit *braucht*

Überall sonst in diesem Band ist Härte ein Hindernis. **In der Kryptographie ist sie die Grundlage** – *ohne schwere Probleme gäbe es keine Verschlüsselung.*

Definition 13.1. Eine Funktion f heißt **Einwegfunktion**, wenn

leicht vorwärts	$f(x)$ ist in polynomieller Zeit berechenbar
schwer rückwärts	Für jeden polynomiellen Algorithmus A ist $\Pr_x[A(f(x)) \in f^{-1}(f(x))]$ vernachlässigbar klein

Zwei Feinheiten stecken in der zweiten Zeile. Erstens: Es genügt nicht der **schlechteste** Fall – die Umkehrung muss für **fast alle** x schwer sein. **Zweitens:** Auch *randomisierte* Angreifer dürfen nicht durchkommen.

P ≠ NP genügt *nicht* für Kryptographie

Das ist der wichtigste Punkt dieses Kapitels.

P ≠ NP sagt	Es gibt Probleme, die im **schlechtesten** Fall schwer sind
Kryptographie braucht	Probleme, die im **Durchschnitt** schwer sind

*Ein Verschlüsselungsverfahren nützt nichts, wenn es nur für **seltene** Schlüssel sicher ist.* **Es muss für *zufällig gewählte* Schlüssel sicher sein.**

Und der Zusammenhang ist ungeklärt.

Einwegfunktionen existieren	$\Longrightarrow$ P ≠ NP ✓
P ≠ NP	$\Longrightarrow$ **nichts**

Es könnte also sein, dass P ≠ NP *gilt und es trotzdem **keine** Kryptographie gibt –* **eine Welt, in der harte Probleme existieren, aber keine, die man *erzeugen* kann.**

Impagliazzos fünf Welten

RUSSELL IMPAGLIAZZO *beschrieb 1995 fünf mögliche Welten – je nachdem, welche der offenen Fragen wie ausgehen.* **Der Aufsatz gilt als der beste Einstieg in die Bedeutung dieser Fragen.**

Welt	Es gilt	Folge
Algorithmica	$P = NP$	Alles leicht; keine Kryptographie; Mathematik automatisierbar
Heuristica	$P \neq NP$, aber im Mittel leicht	Harte Instanzen existieren, sind aber nicht findbar
Pessiland	Im Mittel hart, **keine** Einwegfunktionen	**die schlechteste Welt**
Minicrypt	Einwegfunktionen ja, öffentliche Schlüssel nein	Symmetrische Verfahren möglich
Cryptomania	Public-Key funktioniert	**unsere vermutete Welt**

Pessiland ist die unangenehmste: Man kann schwere Instanzen weder lösen noch für etwas Nützliches gebrauchen. **Alle Nachteile der Härte, keiner ihrer Vorteile.**

Wir leben vermutlich in Cryptomania – **aber bewiesen ist das nicht, und man kann die Welten nicht einmal auseinanderhalten.**

Impagliazzo schrieb dazu 1995: „Wir können nicht beweisen, in welcher Welt wir leben, und das wird sich in absehbarer Zeit nicht ändern." **Dreißig Jahre später gilt der Satz unverändert.**

Aufgabe 13.1.

(a) Analysieren Sie RSA und die Annahme, auf der es beruht.
(b) Untersuchen Sie das Konzept des Nullwissensbeweises an einem Beispiel.
(c) Diskutieren Sie die Nach-Quanten-Kryptographie und ihre Härteannahmen.

Lösung Schritt für Schritt.

(a) RSA.
Schritt 1 – Das Verfahren.

1.	Wähle zwei große Primzahlen p, q; setze $n := pq$.
2.	Berechne $\varphi(n) = (p-1)(q-1)$.
3.	Wähle e teilerfremd zu $\varphi(n)$; berechne d mit $ed \equiv 1 \pmod{\varphi(n)}$.
4.	Öffentlich: (n, e). Geheim: d (und p, q).

$$\text{Verschlüsseln: } c = m^e \bmod n, \qquad \text{Entschlüsseln: } m = c^d \bmod n.$$

Schritt 2 – Warum es funktioniert.
Nach dem Satz von Euler (Mathematik-Band 1) *gilt* $m^{\varphi(n)} \equiv 1$ *für teilerfremdes* m. *Mit* $ed = 1 + k\varphi(n)$*:*

$$c^d = m^{ed} = m^{1+k\varphi(n)} = m \cdot \left(m^{\varphi(n)}\right)^k \equiv m \cdot 1^k = m. \qquad \checkmark$$

Schritt 3 – Ein kleines Zahlenbeispiel.

Größe	**Wert**	**Rechnung**
p, q	$11, 13$	
n	143	$11 \cdot 13$
$\varphi(n)$	120	$10 \cdot 12$
e	7	$\mathrm{ggT}(7, 120) = 1$ ✓
d	103	$7 \cdot 103 = 721 = 6 \cdot 120 + 1$ ✓

Nachricht $m = 9$*:*

$$c = 9^7 \bmod 143.$$

Schrittweise (schnelle Potenzierung, Band 0):

$$9^2 = 81, \quad 9^4 = 81^2 = 6561 = 45 \cdot 143 + 126 \equiv 126,$$

$$9^7 = 9^4 \cdot 9^2 \cdot 9 \equiv 126 \cdot 81 \cdot 9.$$

$$126 \cdot 81 = 10206 = 71 \cdot 143 + 53 \equiv 53, \qquad 53 \cdot 9 = 477 = 3 \cdot 143 + 48 \equiv 48.$$

$$\boxed{c = 48}$$

Probe – entschlüsseln mit $d = 103$*: Man rechnet* $48^{103} \bmod 143$ *und erhält 9.* ✓
Schritt 4 – Worauf die Sicherheit beruht.

Faktorisierung	Wer n zerlegt, kennt $\varphi(n)$ und damit d
RSA-Annahme	Aus (n, e, c) lässt sich m nicht effizient bestimmen

Die zweite Annahme ist formal* schwächer *als die erste – es könnte einen Weg geben, RSA zu brechen, ohne zu faktorisieren. **Bewiesen ist die Äquivalenz nicht.**

Die Sicherheitslage in drei Zeilen

Faktorisieren ist NP-schwer?	**Nein, vermutlich nicht** – es liegt in NP ∩ coNP
Faktorisieren ist bewiesen schwer?	**Nein** – niemand hat eine untere Schranke
RSA ist bewiesen sicher?	**Nein** – nur nicht gebrochen

Die gesamte Public-Key-Kryptographie ruht auf unbewiesenen Annahmen. Das ist bekannt, unvermeidlich und wird offen ausgesprochen – *ein Beweis würde* P $\neq$ NP *implizieren und wäre die Millennium-Million wert.*

Was man stattdessen hat, sind Reduktionsbeweise: *„Wer dieses Verfahren bricht, kann auch jenes Problem lösen."* **Die Sicherheit ist *relativ* zu einer Annahme, nie absolut.**

Die einzige beweisbar sichere Verschlüsselung ist das ***Einmalblatt*** **– und es braucht einen Schlüssel, der so lang ist wie die Nachricht.** SHANNON *bewies 1949, dass das unvermeidlich ist* (Band 7).

(b) Nullwissensbeweise.

Die Aufgabe: *Man will beweisen, dass man ein Geheimnis kennt* **– ohne es preiszugeben.** *Das klingt widersprüchlich und ist es nicht.*

Das Beispiel: 3-Färbung.
Peggy behauptet, eine 3-Färbung des Graphen G zu kennen. Victor soll das glauben, ohne die Färbung zu erfahren.

1. Peggy **permutiert die drei Farben zufällig** und legt jede Ecke verdeckt fest (etwa in einem Umschlag).
2. Victor wählt **eine zufällige Kante** (u, v).
3. Peggy öffnet die beiden Umschläge u und v.
4. Victor prüft: verschiedene Farben? Wenn nein, ist Peggy überführt.

Die drei Eigenschaften.
Vollständigkeit. Kennt Peggy eine Färbung, so besteht sie jede Runde. ✓
Korrektheit. Kennt sie keine, so ist mindestens eine Kante schlecht gefärbt:

$$\Pr[\text{ertappt}] \geq \frac{1}{m}.$$

Nach $t = m \cdot k$ Runden:

$$\Pr[\text{nie ertappt}] \leq \left(1 - \frac{1}{m}\right)^{mk} \leq \mathrm{e}^{-k}.$$

Bei $m = 1000$ Kanten und $k = 100$ also 10^5 Runden für Fehler 10^{-44}. ✓

Nullwissen. Victor sieht in jeder Runde **zwei verschiedene zufällige Farben**.

> *Da Peggy die Farben jedes Mal neu permutiert, ist diese Beobachtung von reinem Zufall nicht zu unterscheiden.* **Victor könnte das Protokoll *selbst simulieren*, ohne mit Peggy zu reden** – *er lernt also nichts.* ✓

*Das ist die formale Definition: Ein Protokoll ist **nullwissend**, wenn es einen **Simulator** gibt, der dieselbe Verteilung von Sichtweisen ohne Kenntnis des Geheimnisses erzeugt.*

> ***Da 3-Färbung NP-vollständig ist, folgt daraus: Jede** Aussage in* NP *hat einen Nullwissensbeweis* (GOLDREICH, MICALI, WIGDERSON, 1986) – *sofern Einwegfunktionen existieren, denn die „Umschläge" braucht man dafür.*

*Ein bemerkenswerter Fall, in dem die NP-Vollständigkeit **konstruktiv** genutzt wird:* **Man beweist etwas für ein Problem und erhält es für alle.**

Wo das eingesetzt wird.

Anmeldung	Passwortkenntnis ohne Passwortübertragung
Blockketten	`zk-SNARK`: Gültigkeit ohne Offenlegung der Beträge
Ausweise	Altersnachweis ohne Geburtsdatum

(c) Nach-Quanten-Kryptographie.
Was Shors Algorithmus bricht.

Verfahren	beruht auf	quantensicher?
RSA	Faktorisieren	**nein**
Diffie-Hellman	diskreter Logarithmus	**nein**
Elliptische Kurven	disk. Log. auf Kurven	**nein**
AES-256	symmetrisch	ja (Schlüssel verdoppeln)
Gitterverfahren	kürzester Gittervektor	vermutlich
Hashsignaturen	Kollisionsresistenz	ja
Codebasiert	Dekodierung	vermutlich

> ***Der gemeinsame Nenner der ersten drei Zeilen:*** *Sie beruhen alle auf dem* ***verborgenen Untergruppenproblem*** *in abelschen Gruppen* – **und genau das löst Shors Algorithmus** (Band 15).

Die Verfahren der unteren Hälfte beruhen auf Problemen, für die kein Quantenvorteil bekannt ist.
Der Stand August 2026.

Norm (NIST)	Verfahren	Grundlage
ML-KEM (2024)	Kyber	Gitter (Module-LWE)
ML-DSA (2024)	Dilithium	Gitter
SLH-DSA (2024)	SPHINCS+	Hashfunktionen
HQC (2025)	—	Codebasiert

Ein Kandidat, der binnen einer Stunde fiel

Im Juli 2022 wählte NIST vier Verfahren aus. Eines davon war `SIKE`, ein Verfahren auf Basis *supersingulärer Isogenien*.

> *Am 30. Juli 2022 – wenige Wochen später – veröffentlichten* Wouter Castryck *und* Thomas Decru *einen Angriff.* **Er brach die höchste Sicherheitsstufe auf einem einzelnen Rechnerkern in etwa einer Stunde.**

Der Angriff benutzte einen Satz von Ernst Kani *aus dem Jahr 1997 – reine Mathematik über abelsche Flächen, dreißig Jahre alt und niemandem in der Kryptographie aufgefallen.*

> *`SIKE` wurde zurückgezogen.* **Der Fall ist die beste Illustration dafür, was „unbewiesene Annahme“ bedeutet.**

> *Das Verfahren war jahrelang untersucht worden, hatte ein Auswahlverfahren überstanden und galt als eines der elegantesten Konzepte des Gebiets.* **Es fiel nicht an einem Implementierungsfehler, sondern an der mathematischen Grundannahme selbst.**

Die Lehre der Kryptographen daraus lautet Krypto-Agilität: *Systeme so bauen, dass sich das Verfahren austauschen lässt* – **denn man muss damit rechnen, dass eine Annahme fällt.**

*Und die zweite Lehre: Man setzt nicht auf **ein** Verfahren.* **NIST normierte deshalb bewusst Kandidaten aus *verschiedenen* mathematischen Familien** – *Gitter, Hashfunktionen, Codes.*

Ein Nachtrag zum Zeitdruck.

> *Man spricht von **Harvest now, decrypt later**: Verschlüsselte Daten lassen sich heute aufzeichnen und in zwanzig Jahren entschlüsseln, wenn es Quantenrechner gibt.*

*Für Daten mit langer Schutzfrist – Gesundheitsakten, Staatsgeheimnisse – ist die Umstellung deshalb **schon jetzt** dringlich,* **obwohl es die Maschinen noch nicht gibt.**

Goldreich[11] ist das Grundlagenwerk der Kryptographie; Katz und Lindell[16] sind die zugänglichere Darstellung; Impagliazzos Aufsatz[14] über die fünf Welten ist auf zwölf Seiten lesbar und lohnt sich für jeden.

Part V

Grenzen der Methode

14 Warum die Frage offen ist

Sätze darüber, welche Beweise nicht funktionieren

Nach fünfzig Jahren ohne Fortschritt liegt die Frage nahe, ***warum*** *niemand weiterkommt.* **Die Komplexitätstheorie hat darauf eine ungewöhnliche Antwort:** *Sie hat Sätze bewiesen, die ganze Beweisfamilien ausschließen.*

Schranke	Jahr	schließt aus
Relativierung	1975	alles, was unter Orakeln gültig bleibt – insbesondere Diagonalisierung
Natürliche Beweise	1994	konstruktive, breit anwendbare kombinatorische Argumente
Algebrisierung	2008	auch die Arithmetisierungstechniken der 1990er Jahre

Jede dieser Schranken entstand, nachdem eine Methode große Erfolge hatte – und zeigte dann, dass diese Erfolge nicht bis P gegen NP reichen können.

Man nennt sie ***Barrieren****.* **Es sind Metasätze: Aussagen über Beweise, nicht über Berechnungen.**

14.1 Die Relativierungsbarriere

Maschinen mit Orakel

Definition 14.1. Eine **Orakelmaschine** M^A darf in einem Schritt fragen, ob ein Wort in der Sprache A liegt, und erhält die Antwort umsonst. P^A und NP^A sind die entsprechenden Klassen.

> ***Der Punkt:*** *Fast alle klassischen Beweise – insbesondere alle Diagonalisierungen – bleiben gültig, wenn man beiden Seiten dasselbe Orakel gibt.* **Man nennt sie relativierend.**

Etwa der Zeithierarchiesatz: Sein Beweis simuliert Maschinen und kehrt Antworten um – mit Orakel geht das genauso. **Also gilt** $\mathrm{P}^A \subsetneq \mathrm{EXP}^A$ **für *jedes* A.** ✓

Satz 14.2 (Baker, Gill, Solovay 1975)**.** *Es gibt Orakel A und B mit*

$$\mathrm{P}^A = \mathrm{NP}^A \qquad \textit{und} \qquad \mathrm{P}^B \neq \mathrm{NP}^B.$$

> ***Damit ist jede relativierende Beweismethode erledigt.*** Würde sie $\mathrm{P} \neq \mathrm{NP}$ zeigen, so zeigte sie auch $\mathrm{P}^A \neq \mathrm{NP}^A$ **– im Widerspruch zur ersten Hälfte.** *Und umgekehrt für* $\mathrm{P} = \mathrm{NP}$.

Das Orakel A – Gleichheit erzwingen.
Man wähle A als eine PSPACE*-vollständige Sprache, etwa* QBF.

$\mathrm{P}^{\mathrm{QBF}} \supseteq \mathrm{PSPACE}$	Mit dem Orakel löst man jedes PSPACE-Problem in einem Schritt
$\mathrm{NP}^{\mathrm{QBF}} \subseteq \mathrm{PSPACE}$	Man probiert alle Zertifikate durch und beantwortet Orakelfragen selbst – alles in polynomiellem Platz

$$\Longrightarrow \mathrm{P}^{\mathrm{QBF}} = \mathrm{NP}^{\mathrm{QBF}} = \mathrm{PSPACE}. \qquad ✓$$

Das Orakel B – Ungleichheit erzwingen.
Man definiert

$$L_B := \{1^n : B \text{ enthält ein Wort der Länge } n\}.$$

$L_B \in \mathrm{NP}^B$ *für jedes B: Man rät das Wort und fragt das Orakel.* ✓

Nun baut man B durch Diagonalisierung so, dass $L_B \notin \mathrm{P}^B$:

1.	Zähle alle polynomiellen Orakelmaschinen $M_1, M_2, \ldots$ auf.
2.	Für jedes i wähle ein n, das größer ist als alles bisher Festgelegte.
3.	Lasse M_i^B auf 1^n laufen. Sie stellt höchstens n^i Orakelfragen – aber es gibt 2^n Wörter der Länge n.
4.	**Also bleibt ein Wort der Länge n unbefragt.**
5.	Akzeptiert M_i, so nimm **kein** Wort der Länge n in B auf. Verwirft sie, so nimm das unbefragte hinein.

In beiden Fällen irrt M_i – und die Änderung betrifft nur Wörter, nach denen sie nicht gefragt hat. *Ihr Lauf bleibt also derselbe.* ✓

Der Kern ist die Abzählung: $n^i \ll 2^n$ – **eine polynomiell beschränkte Maschine kann den exponentiell großen Raum nicht ausschöpfen.**

Was der Satz *nicht* sagt

Er sagt nicht, dass die Frage unentscheidbar oder unbeweisbar ist.

Er sagt: Ein Beweis muss die ***innere Struktur*** *von Berechnungen ausnutzen* – **etwas, das sich ändert, wenn man ein Orakel dazugibt.**

Und solche Beweise gibt es. *Der Satz* IP = PSPACE *(Shamir 1990) relativiert* ***nicht*** – **es gibt ein Orakel, unter dem er falsch ist.**

Er benutzt die ***Arithmetisierung****: Boolesche Formeln werden in Polynome übersetzt, und man rechnet mit ihrer algebraischen Struktur.* **Ein Orakel hat keine solche Struktur.**

Das war 1990 ein großer Hoffnungsschimmer: **Endlich eine nichtrelativierende Methode.** *Kapitel-Abschnitt 3 zeigt, wie diese Hoffnung 2008 begrenzt wurde.*

14.2 Natürliche Beweise

Der Erfolg der 1980er Jahre und sein Ende

Nach 1975 wandte man sich den ***Schaltkreisen*** *zu.* **Der Gedanke: Statt Maschinen zu betrachten, zeigt man, dass SAT keine kleinen Schaltkreise hat** – *denn* P*-Probleme haben polynomiell große.*

Ergebnis	Jahr	Klasse
Parität nicht in AC^0	1983	konstante Tiefe
Clique nicht in monotonem P	1985	Razborov
Untere Schranken für ACC^0	**2011**	Williams

Man kam gut voran – **und dann kam 1994 der Satz, der erklärte, warum es nicht weiterging.**

Definition 14.3. Ein Beweis einer unteren Schranke heißt **natürlich**, wenn er eine Eigenschaft $\mathcal{C}$ von Booleschen Funktionen benutzt mit

Nützlichkeit	Funktionen mit $\mathcal{C}$ haben keine kleinen Schaltkreise
Konstruktivität	Ob f die Eigenschaft $\mathcal{C}$ hat, ist effizient prüfbar (aus der Wertetabelle)
Breite	**Ein großer Anteil** aller Funktionen hat $\mathcal{C}$

Satz 14.4 (Razborov und Rudich, 1994)**.** *Existieren Einwegfunktionen mit exponentieller Sicherheit, so gibt es* ***keinen*** *natürlichen Beweis für* $\mathrm{NP} \not\subseteq \mathrm{P/poly}$.

Die Beweisidee – und sie ist bemerkenswert.

Aus Einwegfunktionen baut man ***Pseudozufallsfunktionen*** (GOLDREICH, GOLDWASSER, MICALI, 1986) – **Funktionen mit kleinen Schaltkreisen, die von echten Zufallsfunktionen nicht zu unterscheiden sind.**

Nun sei $\mathcal{C}$ eine natürliche Eigenschaft.

1. Wegen der **Breite** hat eine zufällige Funktion die Eigenschaft $\mathcal{C}$ mit erheblicher Wahrscheinlichkeit.
2. Wegen der **Nützlichkeit** hat eine Pseudozufallsfunktion sie **nicht** – sie hat ja kleine Schaltkreise.
3. Wegen der **Konstruktivität** lässt sich $\mathcal{C}$ effizient prüfen.

Damit ist $\mathcal{C}$ ein effizienter Unterscheider – er trennt Pseudozufall von echtem Zufall. **Also war die Funktion nicht pseudozufällig, und es gibt keine Einwegfunktionen.**

Die Ironie ist vollkommen: *Wer* $\mathrm{P} \neq \mathrm{NP}$ *mit einer natürlichen Methode beweisen wollte, würde damit zeigen, dass es keine Kryptographie gibt* – **also indirekt, dass die Welt *leichter* ist als gedacht.**

Und praktisch alle bekannten unteren Schranken sind natürlich in diesem Sinn – **einschließlich Razborovs eigenem Ergebnis von 1985.**

Ein Beweis, dass man den Beweis nicht führen kann

Alexander Razborov hatte 1985 die bis dahin stärkste untere Schranke bewiesen – *für monotone Schaltkreise, mit der* ***Approximationsmethode****.*

Die Fachwelt war überzeugt, der Durchbruch stehe bevor. **Razborov selbst versuchte jahrelang, das Ergebnis auf allgemeine Schaltkreise auszudehnen** – *und scheiterte.*

Statt weiter zu versuchen, untersuchte er zusammen mit Steven Rudich, ***warum*** **es nicht ging.**

Das Ergebnis von 1994 zeigte: Die eigene Methode – und alle verwandten – können prinzipiell nicht reichen.

Der Aufsatz heißt schlicht „Natural Proofs“ und erhielt 2007 den Gödel-Preis. **Es ist eines der wenigen Beispiele, in denen jemand einen Preis dafür bekommt, die Grenzen seines eigenen Programms nachgewiesen zu haben.**

Rudich *sagte später über die Wirkung:* **„Wir haben der Gemeinde eine Landkarte gegeben, auf der sehr viel Gebiet als Sumpf markiert ist.“**

Der eine bekannte Ausweg ist RYAN WILLIAMS' *Ergebnis von 2011 über die Klasse* ACC^0. **Sein Beweis ist *nicht* natürlich – er verletzt die Konstruktivität**, *indem er eine nichtkonstruktive Diagonalisierung mit algorithmischen Methoden verbindet.*

Williams nannte sein Programm „algorithmische Methode": Man gewinnt untere Schranken aus ***schnellen Algorithmen*** – **eine Umkehrung der üblichen Richtung.**

Aufgabe 14.1.

(a) Führen Sie die Konstruktion des Orakels B vollständig aus.
(b) Untersuchen Sie die Algebrisierungsbarriere.
(c) Diskutieren Sie, welche Wege offen bleiben.

Lösung Schritt für Schritt.

(a) Das Orakel B im Detail.
Schritt 1 – Die Aufzählung.
Sei $M_1, M_2, \ldots$ eine Aufzählung aller Orakelmaschinen mit Laufzeitschranke n^i für die i-te – **man erhält sie, indem man Maschinen und Polynome paart.**
Schritt 2 – Die Konstruktion in Stufen.
Wir bauen B schrittweise. Nach Stufe i ist über endlich viele Wörter entschieden; sei n_i die größte betroffene Länge.

---

1. Wähle $n > n_{i-1}$ so groß, dass $n^i < 2^n$.
2. Lasse M_i^B auf der Eingabe 1^n laufen. Für Orakelfragen zu bereits entschiedenen Wörtern nimm die Entscheidung; für alle anderen antworte vorläufig „nein".
3. Die Maschine stellt höchstens n^i Fragen. Da $2^n > n^i$, gibt es ein Wort w der Länge n, nach dem sie **nicht** gefragt hat.
4. **Fall A:** M_i akzeptiert. Dann nimm *kein* Wort der Länge n in B auf.
5. **Fall B:** M_i verwirft. Dann nimm w in B auf.

Schritt 3 – Warum M_i irrt.

Fall	M_i sagt	Wahrheit
A	akzeptiert, also $1^n \in L_B$	B hat kein Wort der Länge n, also $1^n \notin L_B$ falsch
B	verwirft, also $1^n \notin L_B$	$w \in B$ mit $\lvert w\rvert = n$, also $1^n \in L_B$ falsch

Entscheidend ist Schritt 3: *Die Änderung an w beeinflusst den Lauf von M_i nicht, weil sie w nie gefragt hat.* **Ihr Verhalten bleibt exakt dasselbe.** ✓

Schritt 4 – Der Schluss.
Jede Maschine M_i wird auf irgendeiner Eingabe widerlegt, also entscheidet keine L_B:

$$L_B \notin \mathrm{P}^B, \qquad L_B \in \mathrm{NP}^B \Longrightarrow \mathrm{P}^B \neq \mathrm{NP}^B.$$

Der Beweis ist im Kern ein Abzählargument: n^i Fragen gegen 2^n Möglichkeiten. **Und genau das ist seine Schwäche als Vorbild** – *ohne Orakel gibt es keine „unbefragten Wörter", an denen man drehen könnte.*

(b) Die Algebrisierungsbarriere.
Der Hintergrund.

Nach 1990 galt die ***Arithmetisierung*** *als der Ausweg aus der Relativierungsbarriere* – **sie hatte** IP = PSPACE **und den PCP-Satz geliefert** (Kapitel 10), *und beide relativieren nicht.*

Der Kunstgriff der Arithmetisierung.
Man übersetzt Boolesche Operationen in Polynome:

Boolesch	Polynom	auf $\{0,1\}$
$\neg x$	$1 - x$	✓
$x \wedge y$	$x \cdot y$	✓
$x \vee y$	$x + y - xy$	✓

Die Polynome stimmen auf $\{0,1\}$ mit der Logik überein – **aber sie sind auch auf größeren Körpern auswertbar.** *Und dort hat man plötzlich Struktur: Ein Polynom vom Grad d ist durch $d+1$ Werte festgelegt.*

Satz 14.5 (Aaronson und Wigderson, 2008)**.** *Es gibt* ***algebraische Orakel****, unter denen* $\mathrm{P} = \mathrm{NP}$ *gilt, und andere, unter denen* $\mathrm{P} \neq \mathrm{NP}$ *gilt.*

Ein algebraisches Orakel ist ein Orakel zusammen mit einer ***Fortsetzung auf einen Körper*** **– genau das, was die Arithmetisierung erzeugt.**

Damit sind auch die Methoden der 1990er Jahre unzureichend **– sie „algebrisieren", und algebrisierende Beweise können die Frage nicht entscheiden.**

Der Stand nach drei Barrieren.

Methode	**Barriere**	**Erfolge**
Diagonalisierung	Relativierung	Hierarchiesätze
Kombinatorik	natürliche Beweise	AC^0, monoton
Arithmetisierung	Algebrisierung	IP = PSPACE, PCP
???	—	**gesucht**

(c) Welche Wege offen bleiben.
Weg 1 – Nichtkonstruktive Beweise.

Die Barriere der natürlichen Beweise verlangt ***Konstruktivität****.* **Ein Beweis, der die Härteeigenschaft nicht effizient prüfbar macht, umgeht sie.**

WILLIAMS' *Ergebnis von 2011 ist von dieser Art* **– und der einzige nennenswerte Fortschritt seit 1994.**
Weg 2 – Nichtbreite Beweise.
Ein Beweis, der eine Eigenschaft benutzt, die nur ***sehr wenige*** *Funktionen haben, ist ebenfalls nicht natürlich.*

Das ist der Ansatz der ***geometrischen Komplexitätstheorie*** *von* KETAN MULMULEY *und* MILIND SOHONI *(seit 2001).* **Sie benutzt Darstellungstheorie und algebraische Geometrie, um Härte an** ***Symmetrieeigenschaften*** **festzumachen.**

Mulmuley selbst schätzte 2009, sein Programm brauche ***Jahrhunderte*** **– es setzt Vermutungen der algebraischen Geometrie voraus, die selbst offen sind.**

Weg 3 – Beweistheoretische Unabhängigkeit.

Es ist denkbar, dass P *gegen* NP *von den üblichen Axiomen* ***unabhängig*** *ist –* **wie die Kontinuumshypothese** (Band 2, Kapitel 13).

Bekannt	In schwachen Arithmetiktheorien ist die Aussage nicht beweisbar
Offen	Ob sie in ZFC unabhängig ist
Bemerkung	Die meisten halten das für unwahrscheinlich

Was wäre, wenn?

Fall 1: $\mathrm{P} = \mathrm{NP}$ ***mit praktikablem Algorithmus.***

Kryptographie bricht zusammen. Optimierung wird trivial. Mathematisches Beweisen wird automatisierbar – **genau das, was Gödel 1956 vorhersah** (Vorwort). *Die Folgen wären so weitreichend, dass man sie kaum abschätzen kann.*

Fall 2: $\mathrm{P} = \mathrm{NP}$ ***mit*** n^{100}**.**

Praktisch ändert sich ***nichts*** – die Kryptographie bliebe sicher, die Optimierung schwer. **Aber die begriffliche Grundlage des Fachs wäre umgestürzt.**

Fall 3: $\mathrm{P} \neq \mathrm{NP}$ ***bewiesen.***

Praktisch ändert sich ebenfalls wenig – man rechnet ohnehin damit. **Aber der Beweis brauchte eine völlig neue Methode**, *und die wäre vermutlich wertvoller als das Ergebnis.*

In allen drei Fällen ist die Methode wichtiger als die Antwort **– eine Lage, die in der Mathematik häufiger vorkommt, als man denkt.**

Eine Umfrage von William Gasarch (2019, 124 Antworten):

Meinung	**Anteil**	**1. Umfrage 2002**
$\mathrm{P} \neq \mathrm{NP}$	88 %	61 %
$\mathrm{P} = \mathrm{NP}$	12 %	9 %
unabhängig / unklar	wenige	Rest

Die Zuversicht ist gestiegen, obwohl es keinen Fortschritt gab **– ein Befund, über den man nachdenken kann.**

Baker, Gill und Solovay[3] sowie Razborov und Rudich[22] sind beide gut lesbar; Aaronson[1] gibt die beste Übersicht über alle drei Barrieren; Williams' Arbeit[28] zeigt den einen bekannten Ausweg; Wigderson[27] ordnet das Ganze ein.

Part VI

Aktueller Stand

15 Wo die Theorie heute steht

Was bewiesen ist und was nicht

Aussage	Status	Bemerkung
$P \subsetneq EXP$	bewiesen	Zeithierarchiesatz 1965
$L \subsetneq PSPACE$	bewiesen	Platzhierarchiesatz
$PSPACE = NPSPACE$	bewiesen	Savitch 1970
$NL = coNL$	bewiesen	Immerman-Szelepcsényi 1987
$IP = PSPACE$	bewiesen	Shamir 1990
PCP-Satz	bewiesen	1992, neu 2005
Primalität in P	bewiesen	AKS 2002
Ungerichteter Zusammenhang in L	bewiesen	Reingold 2004
P gegen NP	**offen**	seit 1971
NP gegen coNP	offen	
P gegen PSPACE	offen	
L gegen NL	offen	
P gegen BPP	offen	$=$ **vermutet**
P gegen NC	offen	Parallelisierbarkeit
Unique-Games-Hypothese	offen	seit 2002

Die obere Hälfte enthält keine einzige Trennung benachbarter Klassen. Alles Bewiesene betrifft entweder weit auseinanderliegende Klassen oder *Gleichheiten*. **Die Trennungen fehlen vollständig.**

15.1 Warum die Praxis anders aussieht

Die Löser, die es nicht geben dürfte

SAT *ist das Musterbeispiel eines harten Problems.* **Und zugleich löst man es routinemäßig für Instanzen mit Millionen Variablen.**

Jahr	Variablen	Meilenstein
1992	≈ 100	
2001	$\approx 10^4$	Chaff, Klauellernen
2010	$\approx 10^6$	
2026	$> 10^7$	Standard in der Industrie

Wodurch das gelingt.

CDCL	Aus jedem Widerspruch wird eine neue Klausel gelernt und der Suchbaum beschnitten
Nichtchronologisches Zurücksetzen	Man springt zur Ursache zurück, nicht zum letzten Zug
Aktivitätsheuristik	Variablen, die oft an Konflikten beteiligt sind, werden zuerst belegt
Neustarts	Regelmäßiges Aufgeben und Neubeginnen mit gelerntem Wissen

Keine dieser Techniken hat eine bessere Worst-Case-Schranke. CDCL entspricht dem Resolutionskalkül, und für den kennt man exponentielle untere Schranken (Kapitel 6).

Die Erklärung liegt darin, dass reale Instanzen ***Struktur*** *haben* – **sie kommen aus Schaltungsverifikation, Planung oder Programmanalyse und sind nicht zufällig.**

Wo die harten Instanzen liegen

Für zufällige 3-SAT-Formeln kennt man das Verhalten sehr genau.

Bei m Klauseln und n Variablen entscheidet das Verhältnis $\alpha = m/n$:

α	Erfüllbarkeit	Schwierigkeit
$< 4{,}0$	fast sicher ja	leicht
$\approx 4{,}267$	**Phasenübergang**	**sehr schwer**
$> 4{,}6$	fast sicher nein	leicht

Die harten Instanzen liegen in einem schmalen Band um den Phasenübergang – dort, wo etwa die Hälfte der Formeln erfüllbar ist. **Ober- und unterhalb ist es leicht.**

Der Wert 4,267 wurde mit Methoden der statistischen Physik vorhergesagt (Mézard, Parisi, Zecchina 2002) und 2015 von Ding, Sly *und* Sun *für große k bewiesen.*

Ein bemerkenswerter Fall, in dem Physik der Informatik ein Ergebnis lieferte – **die Kavitätsmethode stammt aus der Theorie der Spingläser.**

Was sich seit 2010 bewegt hat
Erstens: Feinkörnige Komplexität.

*Statt „polynomiell gegen exponentiell" fragt man nach dem **genauen** Exponenten* – **und beweist Schranken unter der Exponentialzeithypothese** (Kapitel 12).

*Das gibt Aussagen über Probleme **innerhalb** von* P*, wo die klassische Theorie schweigt.*
Etwa: Der Editierabstand braucht n^2, und das ist optimal.
Zweitens: Der eine Fortschritt bei den unteren Schranken.
Ryan Williams *zeigte 2011, dass* NEXP $\not\subseteq$ ACC^0 – **die erste neue untere Schaltkreisschranke seit zwanzig Jahren**, *und der erste Beweis, der alle drei Barrieren umgeht.*

*Seine „algorithmische Methode" dreht die übliche Richtung um: Aus einem **schnellen Algorithmus** für ein Erfüllbarkeitsproblem folgt eine **untere Schranke**.* **Der Fortschritt seither ist gering geblieben.**

Drittens: Quantenkomplexität.

Frage	Status	Bemerkung
BQP gegen P	offen	Trennung vermutet
NP $\subseteq$ BQP	offen	**stark bezweifelt**
BQP gegen PH	Orakeltrennung 2018	Raz, Tal
MIP* = RE	2020	Ji u. a.

Die letzte Zeile ist die erstaunlichste: ***Zwei verschränkte Beweiser können einem Prüfer*** **jede** ***aufzählbare Aussage belegen*** **– auch das Halteproblem.** *Daraus folgte nebenbei, dass die Connes-Vermutung aus der Operatoralgebra (1976) falsch ist.*

Ein Ergebnis der Komplexitätstheorie entschied ein Problem der Funktionalanalysis **– das kommt selten vor.**

Schlussgedanke: drei Einsichten

Erstens: Härte ist keine Eigenschaft, sondern eine Beziehung.

Kein einziges Ergebnis dieses Bandes sagt „Problem X ist schwer“. **Alle sagen: „*X* ist mindestens so schwer wie *Y*“** *– und Y ist es, weil es mindestens so schwer wie Z ist, und so fort bis* SAT.

Die ganze Theorie ruht auf einer Vermutung und einer Kette von Reduktionen. **Das ist keine Schwäche, sondern die einzige Bauform, die derzeit möglich ist.**

Zweitens: Die Klassifikation hängt davon ab, was man misst.

Maßstab	Knotenüberd.	Unabh. Menge
klassisch	schwer	schwer
Näherung	Faktor 2	aussichtslos
Parameter	FPT	W[1]-vollst.

Zwei Probleme, die klassisch dasselbe sind, trennen sich unter jedem feineren Maßstab. Die NP-Vollständigkeit ist ein grobes Sieb – nützlich, aber nicht das letzte Wort.

Drittens: Die Barrieren sind das eigentliche Ergebnis.

Fünfzig Jahre ohne Antwort wären enttäuschend, wenn nicht in dieser Zeit etwas anderes entstanden wäre: **eine präzise Theorie darüber, warum die Antwort ausbleibt.**

Relativierung, natürliche Beweise, Algebrisierung – **drei Sätze, die sagen, welche Werkzeuge nicht reichen.** *Das ist mehr, als die meisten offenen Probleme über sich selbst zu sagen haben.*

Und die Barriere der natürlichen Beweise hat eine Pointe, die den ganzen Band zusammenfasst: **Der Beweis, dass Probleme schwer sind, ist selbst schwer – und zwar aus demselben Grund.**

Wenn die Welt harte Probleme enthält, so enthält sie auch Pseudozufall – **und Pseudozufall macht Härte unsichtbar.** *Die Schwierigkeit versteckt sich selbst.*

Es wäre interessant zu wissen, wie stark im allgemeinen bei finiten kombinatorischen Problemen die Anzahl der Schritte gegenüber dem blossen Probieren verringert werden kann.

— Kurt Gödel an John von Neumann, *20. März 1956. Der Brief blieb bis 1988 unentdeckt, und die Frage ist bis heute unbeantwortet.*

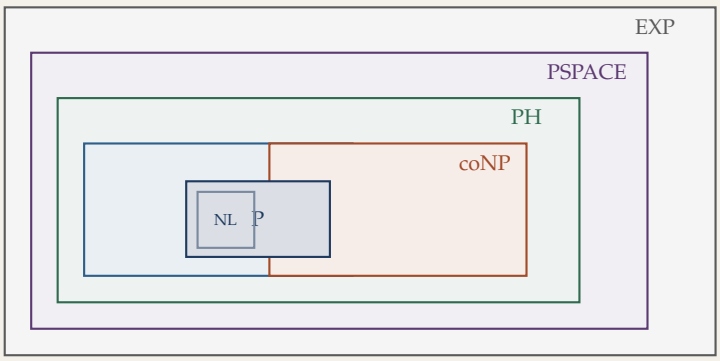
Part VII

Anhang

A Übersicht und Formelsammlung

A.1 Die Landkarte der Klassen

Was in was enthalten ist



*Keine der eingezeichneten Inklusionen ist als **echt** bewiesen* – **außer der äußersten Kette** $\mathrm{P} \subsetneq \mathrm{EXP}$ **und** $\mathrm{L} \subsetneq \mathrm{PSPACE}$**.**

Klasse	**Definition**	**vollständiges Problem**
L	$\mathrm{SPACE}(\log n)$	— (unter NC^1)
NL	$\mathrm{NSPACE}(\log n)$	REACH
P	$\bigcup \mathrm{DTIME}(n^k)$	Schaltkreisauswertung
NP	$\bigcup \mathrm{NTIME}(n^k)$	**SAT**
coNP	Komplemente von NP	TAUTOLOGIE
Σ_k^{P}	k Quantoren, $\exists$ zuerst	Σ_k-QBF
PH	$\bigcup_k \Sigma_k^{\mathrm{P}}$	keines (vermutlich)
PSPACE	$\bigcup \mathrm{SPACE}(n^k)$	**QBF**
EXP	$\bigcup \mathrm{DTIME}(2^{n^k})$	Schach ($n \times n$)
BPP	Zufall, Fehler $\leq \frac{1}{3}$	keines bekannt
BQP	Quantenzeit polynomiell	

Die beiden Zeilen ohne vollständiges Problem sind die semantischen Klassen – ihre Definition enthält eine Bedingung, die man einer Maschine nicht ansieht (Kapitel 11).

Die wichtigsten Sätze auf einen Blick

Satz	Jahr	Aussage
Zeithierarchie	1965	$\mathrm{DTIME}(f) \subsetneq \mathrm{DTIME}(g)$ bei $f \log f = o(g)$
Savitch	1970	$\mathrm{NSPACE}(f) \subseteq \mathrm{SPACE}(f^2)$
Cook-Levin	**1971**	**SAT ist NP-vollständig**
Karp	1972	21 vollständige Probleme
Ladner	1975	Zwischenklassen existieren, falls $\mathrm{P} \neq \mathrm{NP}$
Baker-Gill-Solovay	1975	Relativierungsbarriere
Immerman-Szelepcsényi	1987	$\mathrm{NL} = \mathrm{coNL}$
Shamir	1990	$\mathrm{IP} = \mathrm{PSPACE}$
PCP-Satz	1992	$\mathrm{NP} = \mathrm{PCP}(\log n, 1)$
Razborov-Rudich	1994	Barriere natürlicher Beweise
Impagliazzo-Wigderson	1997	Härte $\Longrightarrow \mathrm{P} = \mathrm{BPP}$
AKS	2002	Primalität in P
Reingold	2004	$\mathrm{USTCON} \in \mathrm{L}$
Aaronson-Wigderson	2008	Algebrisierungsbarriere
Williams	2011	$\mathrm{NEXP} \not\subseteq \mathrm{ACC}^0$

A.2 Die Beweistechniken

Wie man Härte zeigt

Reduktion	Bekanntes schweres A **auf** neues B abbilden; nie umgekehrt (Kap. 4)
Tableau	Rechnung als Tabelle, Korrektheit lokal in 2×3-Fenstern (Cook-Levin)
Diagonalisierung	Maschine bauen, die das Gegenteil jeder anderen tut (Hierarchiesätze)
Konfigurationszählung	Platz $f \Longrightarrow 2^{\mathcal{O}(f)}$ Konfigurationen $\Longrightarrow$ Zeitschranke
Lückenreduktion	Instanzen bauen, bei denen gut und schlecht weit auseinanderliegen (Kap. 10)
Nichtkollaps	Zeigen, dass eine Annahme PH kollabieren ließe

Und wie man Näherungsgüte beweist:

Untere Schranke	Der Algorithmus erzeugt nebenbei ein Objekt, das das Optimum von unten beschränkt (Matching bei der Knotenüberdeckung)
Kostenverteilung	Eigene Kosten auf die Elemente verteilen, dann pro Menge des Optimums abschätzen (Mengenüberdeckung)

Beiden gemeinsam ist, dass sie das Optimum nie berechnen. Man vergleicht mit einer Größe, die man kennt – und die zwischen der eigenen Lösung und dem Optimum liegt.

Zahlen zum Abschätzen

n	n^3	2^n	$1{,}47^n$	$n!$
20	$8 \cdot 10^3$	10^6	$10^{3{,}3}$	$2{,}4 \cdot 10^{18}$
40	$6{,}4 \cdot 10^4$	10^{12}	$10^{6{,}7}$	—
60	$2{,}2 \cdot 10^5$	10^{18}	10^{10}	—
100	10^6	10^{30}	$10^{16{,}7}$	—

*Bei 10^9 Operationen je Sekunde ist 10^{15} etwa **elf Tage**, 10^{18} etwa **dreißig Jahre**.* **Exakte exponentielle Verfahren sind also bis etwa $n = 50$ brauchbar** – *mit besserer Basis bis $n = 100$.*

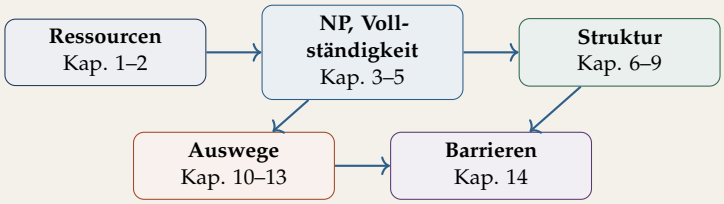
Beste bekannte Basen für exakte Verfahren:

Problem	**Basis**	**Bemerkung**
3-SAT	$\approx 1{,}307$	
Knotenüberdeckung	$1{,}25^k$	parametrisiert
Rundreise	2^n	Held-Karp, dyn. Prog.
Graphfärbung	2^n	
Unabh. Menge	$\approx 1{,}199$	

Unter der SETH lässt sich die Basis bei SAT *nicht unter 2 drücken, wenn die Klauselläng*e *wachsen darf* (Kapitel 12).

A.3 Die Landkarte des Bandes

Aufbau und Querverbindungen



Informatik-Band 0	Zahldarstellung, Speicherhierarchie, Selbstbezug
Informatik-Band 1	Algorithmen, $\mathcal{O}$-Notation, dynamische Programmierung
Informatik-Band 2	Turingmaschine, Reduktionen, Unentscheidbarkeit
Informatik-Band 7	Informationstheorie, Kompression
Informatik-Band 8	Kryptographie ausführlich
Informatik-Band 15	Quantenkomplexität, Shor, Grover
Mathematik-Band 1	Modulare Arithmetik, Euler, RSA-Grundlagen
Mathematik-Band 4	Erwartungswert, Chernoff-Schranken
Mathematik-Band 11	Endliche Körper, Polynome
Mathematik-Band 15	Graphen, Matchings, Flüsse, Schubfachprinzip

A.4 Glossar

Die Begriffe dieses Bandes

Algebrisierung	Barriere gegen arithmetisierende Beweise (2008)
Barriere	Satz darüber, welche Beweismethode nicht ausreicht
BPP	Polynomielle Zeit mit beidseitig beschränktem Fehler
Einwegfunktion	Leicht vorwärts, schwer rückwärts
ETH	3-SAT braucht $2^{\Omega(n)}$
FPT	Laufzeit $f(k) \cdot n^{\mathcal{O}(1)}$
Güte	Verhältnis von Näherung zu Optimum
Kern	Auf $g(k)$ geschrumpfte äquivalente Instanz
Kollaps	Zusammenfallen der Polynomialzeithierarchie
Lückenreduktion	Konstruktion mit Abstand zwischen Ja und Nein
natürlicher Beweis	Konstruktiv, nützlich und breit – und daher unzureichend
NP-schwer	Alles in NP ist darauf reduzierbar
NP-vollständig	NP-schwer und selbst in NP
Orakelmaschine	Darf Fragen an eine Sprache kostenlos stellen
PCP	Beweis, den man stichprobenartig prüft
PTAS	Näherung $1 + \varepsilon$ für jedes feste ε
pseudopolynomiell	Polynomiell im Zahlenwert, nicht in der Länge
Relativierung	Gültigkeit eines Beweises unter beliebigen Orakeln
semantische Klasse	Definition enthält eine unentscheidbare Bedingung
Selbstreduzierbarkeit	Suche lässt sich auf Entscheidung zurückführen
W[1]	Parametrisierte Härteklasse; Clique ist vollständig
Zertifikat	Kurzer, schnell prüfbarer Beleg für die Ja-Antwort

B Kommentierte Literatur

Achtundzwanzig Titel

Die Auswahl enthält neun Originalarbeiten – *darunter alle drei Barrieren und die Gründungsaufsätze von Cook, Karp und Levin.* **Die Reihenfolge ist alphabetisch.**

[1] Aaronson, S.: P $\stackrel{?}{=}$ NP. In: Open Problems in Mathematics, Springer 2016.

Einhundert Seiten über eine einzige Frage – was sie bedeutet, warum sie schwer ist, was die Barrieren aussagen, welche Angriffe gescheitert sind. Der beste Text zum Thema, frei verfügbar und ohne Formalismus lesbar.

[2] Arora, S.; Barak, B.: *Computational Complexity: A Modern Approach.* Cambridge 2009.

Das Standardlehrbuch der Gegenwart. Deckt alles ab, was in diesem Band steht, und wesentlich mehr – PCP, Derandomisierung, Schaltkreisschranken, Quantenkomplexität. Der Entwurf ist frei verfügbar.

[3] Baker, T.; Gill, J.; Solovay, R.: *Relativizations of the* P =?NP *Question.* SIAM J. Computing 4(4), 1975.

Zehn Seiten, die eine ganze Beweisfamilie erledigten. Erstaunlich elementar – der schwierigste Teil ist ein Abzählargument, das jeder nachvollziehen kann.

[4] Cook, S.: *The Complexity of Theorem-Proving Procedures.* STOC 1971.

Der Gründungsaufsatz. Elf Seiten, in denen der Begriff der NP-Vollständigkeit eingeführt und SAT als vollständig nachgewiesen wird. Cook stellt darin auch ausdrücklich die Frage P = NP.

[5] Cygan, M. u. a.: *Parameterized Algorithms.* Springer 2015.

Die moderne Darstellung der parametrisierten Komplexität – mit sämtlichen Techniken: Kernbildung, Verzweigung, Baumzerlegung, algebraische Verfahren. Sieben Autoren, über 600 Seiten, frei verfügbar.

[6] Downey, R. G.; Fellows, M. R.: *Fundamentals of Parameterized Complexity.* Springer 2013.

Von den Begründern des Gebiets. Ausführlicher und historischer als [5], mit vielen Bemerkungen dazu, warum die Begriffe so und nicht anders gewählt wurden.

[7] Edmonds, J.: *Paths, Trees, and Flowers.* Canadian J. Mathematics 17, 1965.

Der Matching-Algorithmus – und, in Abschnitt 2, die erste ausdrückliche Begründung dafür, warum polynomielle Zeit der richtige Begriff für „effizient“ ist. Die Gutachter wollten diesen Abschnitt streichen.

[8] Fortnow, L.: *The Golden Ticket:* P, NP, *and the Search for the Impossible.* Princeton 2013.

Allgemeinverständlich, ohne Formeln, mit vielen Geschichten aus dem Fach. Fortnow schreibt seit Jahrzehnten das einflussreichste Blog der Komplexitätstheorie und kennt alle Beteiligten.

[9] Garey, M. R.; Johnson, D. S.: *Computers and Intractability: A Guide to the Theory of NP-Completeness.* Freeman 1979.

Das meistzitierte Buch der Informatik. Der Anhang listet über 300 NP-vollständige Probleme mit Quellenangabe – nach fünfundvierzig Jahren immer noch das erste Nachschlagewerk, wenn man ein neues Problem einordnen will.

[10] Goldreich, O.: *Computational Complexity: A Conceptual Perspective.* Cambridge 2008.

Legt Wert darauf, *warum* die Begriffe so definiert sind. Wer wissen will, welche Entscheidung hinter jeder Definition steckt, findet es hier und sonst kaum.

[11] Goldreich, O.: *Foundations of Cryptography.* 2 Bände, Cambridge 2001/2004.

Das Grundlagenwerk der beweisbaren Sicherheit – Einwegfunktionen, Pseudozufall, Nullwissen, alles von Grund auf. Anspruchsvoll und die Referenz für jede formale Frage.

[12] Håstad, J.: *Some Optimal Inapproximability Results.* J. ACM 48(4), 2001.

Die scharfen Nichtnäherbarkeitsschranken für Max-3-Sat, Mengenüberdeckung und andere – gewonnen mit dem PCP-Satz. Håstad erhielt dafür 2011 den Gödel-Preis.

[13] Hopcroft, J. E.; Motwani, R.; Ullman, J. D.: *Introduction to Automata Theory, Languages, and Computation.* 3. Aufl., Pearson 2006.

Das Standardwerk für formale Sprachen; die letzten Kapitel führen an die Komplexitätstheorie heran. Für Band 2 dieser Reihe die Hauptquelle.

[14] Impagliazzo, R.: *A Personal View of Average-Case Complexity.* Structure in Complexity Theory, 1995.

Zwölf Seiten über die fünf möglichen Welten. Der klarste Text darüber, was von den offenen Fragen praktisch abhängt – und warum P $\neq$ NP allein für Kryptographie nicht genügt.

[15] Karp, R. M.: *Reducibility Among Combinatorial Problems.* In: Complexity of Computer Computations, Plenum 1972.

Die 21 Probleme. Der Aufsatz, der aus Cooks Satz ein Werkzeug machte und ganze Fachgemeinden erkennen ließ, dass sie am selben Problem gescheitert waren.

[16] Katz, J.; Lindell, Y.: *Introduction to Modern Cryptography.* 3. Aufl., CRC Press 2020.

Die zugängliche Darstellung der beweisbaren Sicherheit. Zeigt konsequent, dass jede Sicherheitsaussage eine Reduktion auf eine Annahme ist – und nie mehr.

[17] Kleinberg, J.; Tardos, É.: *Algorithm Design.* Addison-Wesley 2005.

Die Kapitel über NP-Vollständigkeit und Näherungsverfahren sind die bestmotivierten, die ich kenne. Sie zeigen nicht nur die Reduktionen, sondern wie man auf sie kommt.

[18] Ladner, R. E.: *On the Structure of Polynomial Time Reducibility.* J. ACM 22(1), 1975.

Der Existenzbeweis für Zwischenklassen. Technisch, aber kurz – und die Konstruktion der „Verzögerungsdiagonalisierung“ ist seither ein Standardwerkzeug.

[19] Levin, L. A.: *Universelle Suchprobleme.* Problemy Peredatschi Informazii 9(3), 1973.

Zwei Seiten auf Russisch, unabhängig von Cook. Levin behandelt Suchprobleme statt Entscheidungsprobleme und gibt gleich sechs vollständige an. Sein Zugang ist konstruktiver.

[20] Motwani, R.; Raghavan, P.: *Randomized Algorithms.* Cambridge 1995.

Das Standardwerk zu randomisierten Verfahren – Miller-Rabin, Karger, Schwartz-Zippel, probabilistische Methode. Setzt Wahrscheinlichkeitsrechnung voraus (Mathematik-Band 4).

[21] Papadimitriou, C. H.: *Computational Complexity.* Addison-Wesley 1994.

Der Klassiker vor [2]. Papadimitriou schreibt ungewöhnlich lebendig und ordnet die Ergebnisse historisch ein; die Kapitel über Logik und Komplexität sind bis heute unerreicht.

[22] Razborov, A. A.; Rudich, S.: *Natural Proofs.* J. Computer and System Sciences 55(1), 1997.

Der Beweis, dass die eigene erfolgreiche Methode nicht weit genug reicht. Eine der bemerkenswertesten Arbeiten des Fachs – und einer der wenigen Gödel-Preise für ein negatives Metaresultat.

[23] Savitch, W. J.: *Relationships Between Nondeterministic and Deterministic Tape Complexities.* JCSS 4(2), 1970.

Sieben Seiten für den Satz, der PSPACE = NPSPACE liefert. Die Idee – die Mitte des Weges raten statt der ganzen Folge – ist von einer Klarheit, die man selten findet.

[24] Sipser, M.: *Introduction to the Theory of Computation.* 3. Aufl., Cengage 2012.

Der beste Einstieg. Sipser führt Berechenbarkeit und Komplexität gemeinsam ein, mit sorgfältig ausgeführten Beweisen und ohne unnötigen Formalismus. Wenn Sie *ein* Buch zu diesem Band kaufen: dieses.

[25] Vazirani, V. V.: *Approximation Algorithms.* Springer 2001.

Das Standardwerk der Näherungsverfahren – systematisch nach Techniken geordnet (gierig, LP-Rundung, Primal-Dual, semidefinit). Der zweite Teil behandelt die Nichtnäherbarkeit.

[26] Wegener, I.: *Komplexitätstheorie.* Springer 2003.

Die deutschsprachige Darstellung. Knapp, präzise und mit sorgfältiger Begriffsbildung; Wegener legt besonderen Wert auf die praktische Bedeutung der Ergebnisse.

[27] Wigderson, A.: *Mathematics and Computation.* Princeton 2019.

Ein Überblick über das ganze Gebiet aus der Sicht eines Abel-Preisträgers – was die Komplexitätstheorie für Mathematik, Physik, Biologie und Ökonomie bedeutet. Frei verfügbar, über 400 Seiten, ohne Beweise, aber mit großer Übersicht.

[28] Williams, R.: *Non-uniform ACC Circuit Lower Bounds.* J. ACM 61(1), 2014.

Die erste neue untere Schaltkreisschranke seit zwanzig Jahren – und die einzige, die alle drei Barrieren umgeht. Der Beweis gewinnt die Schranke aus einem *schnellen Algorithmus*, was die übliche Richtung umkehrt.

Wenn Sie nur drei Bücher kaufen

[24]	**Sipser**	zum Lernen – der klarste Einstieg
[2]	**Arora, Barak**	zum Nachschlagen – alles Weitere
[9]	**Garey, Johnson**	zum Arbeiten – ist mein Problem schon bekannt?

Zum Nachdenken: [1] Aaronson *über die Frage selbst* und **[14] Impagliazzo** *über die fünf Welten* – **beide frei verfügbar und beide an einem Abend zu lesen.**

Und wer sehen will, wie das Fach entstand, liest ***[4] Cook 1971*** *und* ***[15] Karp 1972*** *im Original* – **zusammen keine vierzig Seiten.**